Notas de Teoría de la Computación

Miguel Angel Norzagaray Cosío

11 de noviembre de 2026

Índice general

1. Lenguajes Regulares **7**

1.1. Lenguajes 8

1.1.1. Alfabetos y cadenas 8

1.1.2. Lenguajes 13

1.1.3. Operaciones con lenguajes 14

1.2. Autómatas Finitos Determinísticos 15

1.2.1. Representación de AFD 16

1.2.2. Completitud de la función de transición 18

1.2.3. Autómata Procesando Cadenas 19

1.2.4. El lenguaje de un AFD 20

1.2.5. Descripción instantánea de un AFD 22

1.2.6. AFD mínimos 22

1.2.7. Análisis Léxico 23

1.2.8. Autómata finito no determinístico 25

1.2.9. Autómata finito con transiciones ϵ 31

1.3. Expresiones Regulares 37

1.3.1. Conversión de expresiones regulares a autómatas 41

1.3.2. Conversión de AFD a expresiones regulares 43

1.3.3. Simplificación de expresiones regulares 50

1.4. Propiedades de los lenguajes regulares 52

1.4.1. ¿Cómo saber que un lenguaje es regular? 52

1.4.2. Propiedades de cerradura 53

1.4.3. Propiedades de decisión 56

1.5. Limitaciones de los lenguajes regulares 57

2. Lenguajes Libres de Contexto **59**

2.1. Gramáticas Libres de Contexto 59

2.1.1. Derivaciones izquierdas y derivaciones derechas 62

2.1.2. Árboles de derivación 62

2.1.3. Formas normales 63

2.1.4. Simplificación de gramáticas 65

2.1.5. Ejemplos de aplicaciones 66

2.1.6. Gramáticas ambiguas 66

2.2. Autómatas de Pila 68

2.2.1. Descripción instantánea de los AP . . . 71
2.2.2. Lenguaje de un Autómata de Pila . . . 72
2.2.3. Equivalencia entre GLL y AP . . . 73
2.2.4. Limitaciones de los AP . . . 74
2.2.5. Autómatas de Pila Determinísticos (APD) . . . 75
2.3. Lenguajes y compiladores . . . 76
2.3.1. Eliminación de recursividad izquierda . . . 77
2.3.2. Analizadores Recursivos Descendentes . . . 78
2.3.3. Analizadores LL(1) . . . 78
2.4. Propiedades de los LLC . . . 81
2.4.1. Propiedades de decisión . . . 81
2.4.2. Propiedades de cerradura . . . 82
2.5. Limitaciones de los lenguajes libres de contexto . . . 83

3. Lenguajes Recursivamente Enumerables 85
3.1. Máquinas de Turing . . . 85
3.1.1. Descripción instantánea de una MT . . . 88
3.1.2. Lenguaje de una MT . . . 89
3.1.3. MT como calculadoras de funciones . . . 90
3.1.4. Máquinas de Turing no deterministas . . . 93
3.1.5. Máquinas de Turing con múltiples cintas . . . 95
3.1.6. Lenguajes sensibles al contexto . . . 96
3.1.7. La jerarquía de Chomsky . . . 98
3.2. Problemas de decisión . . . 98
3.2.1. Máquinas de Turing Universales . . . 99
3.2.2. Máquina de Turing Universal restringida . . . 100
3.2.3. Lenguajes recursivos . . . 104
3.2.4. Lenguajes diagonal y universal . . . 104
3.2.5. El problema de la parada . . . 106
3.2.6. Reducciones . . . 108
3.2.7. El teorema de Rice . . . 108

Introducción

La Teoría de la Computación constituye los cimientos que sostienen con firmeza los conocimientos del área de la computación. Comprende dos grande rubros, íntimamente ligados:

1. la teoría de los lenguajes y
2. la computabilidad y factibilidad.

Estas notas introductorias es material para usarse en el curso de Teoría de la Computación, en las carreras del Departamento Académico de Sistemas Computacionales de la UABCS. Abordan sólo la teoría de los lenguajes, aunque aportan los principios para el estudio de la computabilidad. Es de gran fortuna que muchos de los resultados importantes de la teoría de lenguajes que se utilizan en la práctica tiene demostraciones que son constructivas, lo que proporciona no sólo su veracidad, sino un algoritmo que permite realizar una tarea.

Cada vez que se define un concepto se *enfatiza* utilizando un tipo de letra que lo distinga. En algunos párrafos se ha colocado un símbolo $\star$ al margen, indicando que la idea es importante y que vale la pena su estudio cuidadoso.

Los ejercicios aparecen aparte en el Cuaderno de Ejercicios de Teoría de la Computación, indispensable para el buen aprovechamiento de esta asignatura.

Los conceptos y proposiciones sobre teoría de conjuntos e inducción matemática, pueden consultarse en el suplemento de Fundamentos Matemáticos. Dicho texto ofrecen el material necesario para estudiar lenguajes y autómatas. Es necesario para entender las proposiciones presentadas (y demostradas), que, en muchos casos, son medulares para el estudio de la computabilidad y el desarrollo del pensamiento abstracto del estudiante.

Estos materiales corresponden a un curso de Matemáticas Discretas y por tanto pueden emplearse ahí como material complementario. Como es de esperarse, muchos libros y notas sobre matemáticas discretas pueden ser de utilidad para este curso.

Material de utilidad fueron muchas referencias de todo tipo, desde libros clásicos, sus actualizaciones, así como notas de otros cursos. De manera particular, los Apuntes y Ejercicios de Gonzalo Navarro, de la Universidad de Chile, ofrecieron una perspectiva nueva para algunos temas.

Es ampliamente recomendable utilizar software como JFLAP (en este caso se utilizó la versión 7.0) para hacer ejercicios y experimentos.

Estas notas fueron elaboradas (y son mantenidas) utilizando LaTeX, la distribución MikTeX 2.9, editando con TeXnicCenter 2.0 Beta y VauCanSon-G 0.4 para los diagramas de los autómatas.

El código fuente en LaTeXestá disponible para otros docentes y estudiantes solicitándolo por correo electrónico a: `manc@uabcs.mx`.

Capítulo 1

Lenguajes Regulares

La teoría de lenguajes aporta al mundo de la computación no solo teoría elegante y formalismo indispensable para confiar en el trabajo del diseño de lenguajes y programación de computadoras, sino que además provee herramientas poderosas para el diseño mismo de lenguajes y su procesamiento para el trabajo de compilación.

Introducción

Desde hace mucho exiten máquinas que hacen cálculos de algún tipo. El mecanismo de Antikitera, la Pascalina, el telar de Jackard (dirigido por tarjetas perforadas), las máquinas de Babagge y de Hollerit (electromecánica), hasta la época moderna, con la llegada de las ideas de Turing y el diseño y contrucción de ENIAC, EDVAC y ACE (o su versión reducida la Pilot ACE).

Cuando se pensaba en el diseño de verdaderas computadoras (como la ACE, ya con programas residentes en memoria) se analizaba la manera óptima de escribir instrucciones. En un principio esto se hizo un tanto artesanalmente, aunque la descripción de algoritmos ya ocurría. Así, se llegó a la abstracción de los lenguajes, que son conjuntos de cadenas.

La descripción de lenguajes mediante gramáticas ya ocurría desde muchos siglos antes, pero no es hasta la década de 1950 cuando se desarrolla la Teoría de los Lenguajes Formales. Desde los primeros años surgieron diversos modelos matemáticos bien formalizados para la descripción de lenguajes. De todos, los más importantes son las gramáticas y las máquinas de estado finito.

Las *gramáticas* son un modelo matemático estructurado que definen la manera como las cadenas de un lenguaje se deben formar. La forma de definirlo es mediante reglas (gramaticales) de la forma $\alpha \to \beta$, donde α y β son cadenas formadas por símbolos terminales y variables, con al menos una variable en α. Con tales reglas, la gramática genera cadenas. Se emplean en el diseño de todo lenguaje de programación y son esenciales en el proceso de compilación, cuando se verifica que el código de un programa es correcto gramaticalmente y se puede

pasar a traducir a lenguaje máquina.

Las *máquinas de estado finito*, llamadas más cortamente autómatas, se representan como grafos dirigidos en los que las aristas están etiquetadas por los símbolos del alfabeto con los que se forman las cadenas, que son rutas en cada máquina, de un (único) estado inicial a un estado final. Dada una cadena, el autómata parte del estado inicial y la cadena se acepta si se llega a un estado final. Varias etapas de un compilador, como el análisis léxico y el sintáctico, se modelan con autómatas especializados.

Para cada clase de lenguajes que se estudiará hay un tipo de gramática y un tipo de autómata equivalentes: generan y aceptan, respectivamente, los mismos lenguajes. Cambiando sus propiedades es posible pasar de lenguajes sencillos a lenguajes con mayor poder de expresión.

1.1. Lenguajes

1.1.1. Alfabetos y cadenas

Sea Σ un conjunto finito no vacío de símbolos al que llamaremos *alfabeto*.

Aunque cualquier caracter puede ser considerado como símbolo, en general utilizaremos letras minúsculas y dígitos, como el conjunto $\{0, 1\}$ o $\{a, b, c\}$.

Ejemplo 1. *Los alfabetos ASCII o Unicode son ejemplos de alfabetos, de uso muy difundido en el mundo de la computación.*

Ejemplo 2. *El código morse asocia una secuencia única no vacía de rayas y puntos a cada letra y número. Aquí conviene fijar cuál es la unidad indivisible: puede tomarse como alfabeto el conjunto $\{\cdot, -\}$ (más el espacio que separa letras), y entonces cada letra es una cadena sobre él, o bien puede tomarse como alfabeto el conjunto de letras y números.*

Algunas propiedades y operaciones

Una *cadena* o *palabra* es el resultado de concatenar símbolos de un alfabeto. Por ejemplo, las cadenas 00101 y 11001 son cadenas sobre el alfabeto $\Sigma = \{0, 1\}$. Más formalmente, tenemos nuestra primera definición:

Definición 1. *La concatenación de dos símbolos de un alfabeto se escribe como $a \cdot b$.*

La concatenación, definida por ahora para dos símbolos, se extenderá para concatenar un símbolo con una cadena y luego dos cadenas, utilizándo siempre la misma notación, que por lo común se omite.

Aunque los alfabetos sean finitos, se puede generar una infinidad de cadenas a partir de ellos.

El sencillo concepto de cadena pueden formalizarse de manera inductiva, como se muestra a continuación.

Definición 2. *Una* cadena *es vacía, denotada por ϵ, o es un símbolo a del alfabeto, concatenado con una cadena, esto es, la cadena w se forma concatenando el símbolo a con la subcadena x, es decir, $w = a \cdot x$.*

Con esta definición, se construyen cadenas uniendo símbolos por la izquierda a cadenas ya formadas. Es importante notar que no se habla de agregar símbolos por la derecha, sólo se agregan por la izquierda.

Ejemplo 3. *La palabra* hola *se expresa formalmente como $h \cdot (o \cdot (l \cdot (a \cdot \epsilon)))$, o más simplemente $h \cdot o \cdot l \cdot a \cdot \epsilon$.*

Es razonable, a partir del ejemplo anterior, que sea preferible omitir el símbolo de concatenación y su asociatividad, escribiendo simplemente `hola`. Aunque cada cadena tiene la cadena vacía al final, se acostumbra omitirla, a menos que se requiera explícitamente, por ejemplo `hola`ϵ.

Definición 3. *La* longitud *de una cadena es la cantidad de símbolos que la componen y se denota por $|w|$ la longitud de la cadena w.*

Por ejemplo la cadena $w = 01110$ tiene longitud 5. La cadena vacía es la cadena que se compone de cero símbolos y se denota por ϵ (algunos autores prefieren λ u otros símbolos). Como se espera, la cadena vacía tiene longitud 0.

Ahora se procede a definir la concatenación de dos cadenas, a partir de la definición anterior. De nuevo se usa el método inductivo.

Definición 4. *La concatenación de dos cadenas r y s se define como*

$$r \cdot s = \begin{cases} s & si \quad r = \epsilon \\ a \cdot (w \cdot s) & si \quad r = a \cdot w \end{cases} \tag{1.1}$$

Esta definición establece que la concatenación de cadenas $r \cdot s$ se realiza separando uno a uno los símbolos iniciales de r y concatenando con s al terminar, mediante la definición 2. Nótese: siempre es de izquierda a derecha. El ejemplo siguiente lo ilustra.

Ejemplo 4. *Para concatenar «abc» con «def» se procede paso a paso como sigue.*

$$\begin{aligned} abc\epsilon \cdot def &= a \cdot (bc\epsilon \cdot def) \\ &= a \cdot (b \cdot (c\epsilon \cdot def)) \\ &= a \cdot (b \cdot (c \cdot (\epsilon \cdot def)) \\ &= a \cdot (b \cdot (c \cdot def)) \\ &= a \cdot (b \cdot (cdef)) \\ &= a \cdot (bcdef) \\ &= abcdef \end{aligned}$$

En cada paso se aplica la definición (1.1); su primer caso, $\epsilon \cdot s = s$, aparece en la cuarta línea. El símbolo ϵ se ha colocado al final de la cadena inicial para poder emplear la definición. En la segunda cadena, está de manera implícita.

Es sencillo ver que la concatenación de cadenas es asociativa y lo podemos anunciar como una proposición formal.

Proposición 1. *Sean r, s y t cadenas de Σ^*. Se cumple que*

$$(r \cdot s) \cdot t = r \cdot (s \cdot t) \tag{1.2}$$

Demostración. Por inducción sobre la longitud de r, con s y t arbitrarias.

Caso base: si $r = \epsilon$, por el primer caso de (1.1), $(\epsilon \cdot s) \cdot t = s \cdot t = \epsilon \cdot (s \cdot t)$.

Hipótesis de inducción: la igualdad se cumple para toda cadena r de longitud n o menor.

Sea $r = a \cdot x$ con $|r| = n + 1$, de modo que $|x| = n$. Entonces

$$\begin{aligned}
(r \cdot s) \cdot t &= ((a \cdot x) \cdot s) \cdot t \\
&= (a \cdot (x \cdot s)) \cdot t && \text{Definición (1.1)} \\
&= a \cdot ((x \cdot s) \cdot t) && \text{Definición (1.1)} \\
&= a \cdot (x \cdot (s \cdot t)) && \text{Hipótesis de inducción} \\
&= (a \cdot x) \cdot (s \cdot t) && \text{Definición (1.1)} \\
&= r \cdot (s \cdot t).
\end{aligned}$$

□

Ejemplo 5. *Dadas las cadenas $w_1 = aab$ y $w_2 = bcab$, su concatenación se escribe, de manera más económica como $w_1 \cdot w_2 = aabbcab$, evitando el detalle de las operaciones, evitando tanto los paréntesis como el operador $\cdot$, a menos que sea necesario para hacer algo explícito.*

En las definiciones anteriores se establece la forma de construir cadenas, concatenando símbolos a cadenas, comenzando (o terminando) con la cadena vacía. Es natural pensar que la cadena vacía pudiera concatenarse con cualquiera y dar como resultado la cadena original, es decir, $w\epsilon = \epsilon w = w$, pero esto debe probarse y no obrar por intuición.

Proposición 2. *Para toda cadena w, se cumple que $w \cdot \epsilon = w$.*

Demostración. Es sencillo aplicar inducción sobre la longitud de w.

Como caso base, $|w| = 0$ cuando se trata de la cadena vacía y por el primer caso de la definición (1.1), $w \cdot \epsilon = \epsilon \cdot \epsilon = \epsilon$.

Como hipótesis de inducción, suponemos que para toda cadena w de longitud menor o igual que n se cumple que $w\epsilon = w$. Lo que hay que demostrar es que toda cadena de longitud $n + 1$ también lo cumple.

Sea $w = a \cdot x$ una cadena de longitud $n + 1$, con a un símbolo y x una subcadena de longitud n.

$$\begin{aligned}
w \cdot \epsilon &= (a \cdot x) \cdot \epsilon \\
&= a \cdot (x \cdot \epsilon) && \text{Definición 1.1} \\
&= a \cdot x && \text{Hip. de Inducción} \\
&= w
\end{aligned}$$

□

Si bien no es directamente asociado con las cadenas, es importante detenerse para analizar una la técnica de definición y demostración muy común en el estudio de los lenguajes formales. Ya se ha empleado la definición y demostración inductiva. Esto se puede aplicar junto con la definición de operaciones cerradas y será fácil de ver con un ejemplo práctico.

Ejemplo 6. *Considérense las cadenas de paréntesis bien emparejados, como* (()()). *En una expresión aritmética como* $(a-(b+a))/(c-a)$ *los demás símbolos no afectan el emparejamiento, así que nos concentramos sólo en los paréntesis: sea* $\Sigma = \{(,)\}$. *Una cadena* w *está* bien emparejada *si tiene tantos paréntesis que abren como paréntesis que cierran y, además, ningún prefijo de* w *tiene más paréntesis que cierran que paréntesis que abren. Definimos inductivamente el conjunto* P *de la siguiente manera.*

1. $\epsilon \in P$.
2. *Si la cadena* $w \in P$, *entonces* $(w) \in P$.
3. *Si* r *y* s *son cadenas de* P, *entonces su concatenación* $r \cdot s$ *también es cadena de* P.

Se trata de operaciones cerradas, es decir, cada que hacemos algo válido con cadenas del conjunto, concatenarlas o ponerles paréntesis, la nueva cadena sigue estando en el conjunto. Puede demostrarse por inducción que toda cadena de P *está bien emparejada.*

Demostración. La inducción se hace sobre el número de reglas aplicadas para construir w (a esto se le llama *inducción estructural*).

Caso base: $w = \epsilon$. No tiene paréntesis, así que ambas condiciones se cumplen por vacuidad.

Hipótesis de inducción: las cadenas r y s, construidas con menos aplicaciones de las reglas, están bien emparejadas. Denotemos por $A(u)$ y $C(u)$ la cantidad de paréntesis que abren y que cierran en una cadena u.

2. La cadena (r) tiene un paréntesis que abre y uno que cierra más que r, así que $A((r)) = C((r))$. Un prefijo propio no vacío de (r) es de la forma $(u$, con u prefijo de r, y cumple $A((u) = A(u) + 1 > C(u) = C((u)$. El prefijo vacío y la cadena completa también cumplen la condición.

3. En $r \cdot s$ los conteos se suman, así que $A(rs) = C(rs)$. Un prefijo de rs es un prefijo de r, que cumple la condición por hipótesis, o es de la forma rv con v prefijo de s, y entonces $A(rv) = A(r) + A(v) \geq C(r) + C(v) = C(rv)$.

El recíproco, que toda cadena bien emparejada pertenece a P, también es cierto y se deja como ejercicio. □

Definición 5. *Se define y denota como* Σ^k *al conjunto de todas las cadenas de longitud* k *sobre el alfabeto* Σ.

Ejemplo 7. *Por ejemplo, si* $\Sigma = \{0, 1\}$*, entonces*

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}.$$

Definición 6. *El conjunto de todas las cadenas que se pueden formar a partir de un alfabeto* Σ *se denota por* Σ^**, conocida como la estrella de Kleen o cerradura de Kleen. De esta manera,*

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cdots$$

Dado que Σ^* incluye la cadena vacía, si se le desea excluir del lenguaje se escribe Σ^+, lo que elimina Σ^0 de Σ^*. Debe entenderse que el lenguaje vacío no contiene cadenas, por lo que la cadena vacía no pertenece al lenguaje vacío.

Ejemplo 8. *Si el alfabeto* Σ *son las letras de la A a la Z, entonces* Σ^* *son todas las combinaciones de letras, incluyendo (de manera mayoritaria) las que no tienen significado en ningún lenguaje natural de ningún país.*

A una cadena $w \in \Sigma^*$ se le pueden invertir sus símbolos, de tal forma de convertir `abcd` en `dcba`. Esta operación es llamada reversa y se define formalmente a continuación.

Definición 7. *Sea* w *una cadena. Su reversa se define y denota como*

$$\text{rev}(w) = \begin{cases} \epsilon & si \quad w = \epsilon \\ \text{rev}(x) \cdot a & si \quad w = a \cdot x \end{cases} \tag{1.3}$$

donde $a \in \Sigma$ *es un símbolo y* $x \in \Sigma^*$ *es una subcadena (sufijo) de* w *que contiene todos los símbolos menos 1.*

Ejemplo 9. *Calculemos la reversa de abc:*

$$\begin{aligned} \text{rev}(abc) &= \text{rev}(a \cdot bc) \\ &= \text{rev}(bc) \cdot a \\ &= \text{rev}(b \cdot c) \cdot a \\ &= (\text{rev}(c) \cdot b) \cdot a \\ &= (\text{rev}(c \cdot \epsilon) \cdot b) \cdot a \\ &= ((\text{rev}(\epsilon) \cdot c) \cdot b) \cdot a \\ &= ((\epsilon \cdot c) \cdot b) \cdot a \\ &= cba \end{aligned}$$

Proposición 3. *Dadas dos cadenas* r *y* s*, se cumple que*

$$\text{rev}(r \cdot s) = \text{rev}(s) \cdot \text{rev}(r).$$

Demostración. Por inducción sobre la longitud de r, con s arbitraria.

Caso base: si $r = \epsilon$, entonces

$$\text{rev}(\epsilon \cdot s) = \text{rev}(s) = \text{rev}(s) \cdot \epsilon = \text{rev}(s) \cdot \text{rev}(\epsilon),$$

usando la proposición 2 y la definición (1.3).

Hipótesis de inducción: $\text{rev}(r \cdot s) = \text{rev}(s) \cdot \text{rev}(r)$ para toda cadena r de longitud n o menor y toda cadena s.

Sea $r = a \cdot x$ con $|r| = n + 1$, de modo que $|x| = n$. Entonces

$$\begin{aligned}
\text{rev}(r \cdot s) &= \text{rev}((a \cdot x) \cdot s) \\
&= \text{rev}(a \cdot (x \cdot s)) && \text{Proposición 1} \\
&= \text{rev}(x \cdot s) \cdot a && \text{Definición (1.3)} \\
&= (\,\text{rev}(s) \cdot \text{rev}(x)) \cdot a && \text{Hipótesis de inducción} \\
&= \text{rev}(s) \cdot (\,\text{rev}(x) \cdot a) && \text{Proposición 1} \\
&= \text{rev}(s) \cdot \text{rev}(a \cdot x) && \text{Definición (1.3)} \\
&= \text{rev}(s) \cdot \text{rev}(r).
\end{aligned}$$

Como la afirmación vale para toda s, no hace falta una inducción adicional sobre la longitud de s. □

1.1.2. Lenguajes

Con el concepto de cadenas bien entendido, entramos a las amplias aguas de los lenguajes y comenzamos con su definición.

Definición 8. *Un* lenguaje *es un conjunto arbitrario de cadenas definidas sobre un alfabeto fijo* Σ.

Es claro que Σ^*, todas las cadenas que es posible construir a partir de los símbolos de Σ, es un lenguaje, pero generalmente un lenguaje (de interés) L es un subconjunto propio de Σ^* que cumple con características especiales. Se dice que todo $L \subseteq \Sigma^*$ es un lenguaje sobre Σ. El conjunto vacío es un lenguaje sin cadenas y es distinto de $\{\epsilon\}$, el lenguaje que posee sólo a la cadena vacía. Ambos se consideran lenguajes sobre cualquier alfabeto.

Por sencillo que parezca, el concepto siguiente es fundamental en la Teoría de la Computación.

Definición 9. *Un* **problema** *es toda pregunta que pretende decidir si una cadena pertenece o no a un lenguaje.* ★

Esta definición corresponde a los llamados *problemas de decisión*, cuya respuesta es sí o no. Muchos otros problemas, como calcular un valor u optimizar, pueden reformularse como problemas de decisión mediante una codificación adecuada, aunque esa traducción no siempre es inmediata. Formalmente: sea Σ un alfabeto y L un lenguaje sobre Σ. Entonces, un *problema* en L es:

$$\text{Dada una cadena } w \in \Sigma^* \text{ decidir si ocurre o no que } w \in L. \tag{1.4}$$

De hecho, hablar sobre lenguajes o problemas es la misma cosa, todo depende del punto de vista como se estén abordando. Si es estudian lenguajes, las cadenas importan por sí mismas, mientras que en computabilidad cada cadena representa una *instancia* del problema (por ejemplo, la codificación de un grafo o de un programa) y el lenguaje es el problema mismo. Décadas de uso han probado que esta es una manera adecuada de enfrentar las preguntas importantes de la Teoría de la Computación, que permite además clasificarlas según la dificultad para determinar su veracidad o no.

Para definir un lenguaje sobre el alfabeto Σ es posible utilizar diversas notaciones. Una vez definido el lenguaje, se puede emplear cualquiera de las formas de descripción de conjuntos, como a continuación se muestra.

Ejemplo 10. *Varios lenguajes sobre alfabetos distintos.*

$L_1 = \{aab, abb, aca, bac\}$ *sobre el alfabeto* $\Sigma = \{a, b, c, d\}$
$L_2 = \{w | w$ *es una cadena binaria que inicia con 1*$\}$ *sobre* $\Sigma = \{0, 1\}$
$L_3 = \{w | w$ *es un programa válido en lenguaje C*$\}$
$L_4 = \{a^n b^{n+1} \mid n \geq 0\}$ *sobre* $\Sigma = \{a, b\}$
$L_5 = \{$`for`, `goto`, `gosub`, `next`, `return`$\}$
$L_6 = \{100, 105, 110, 115, \ldots, 190, 195\}$ *sobre* $\Sigma = \{0, 1, \ldots, 9\}$

L_1, L_5 y L_6 son lenguajes finitos, mientras que L_2, L_3 y L_4 son infinitos[1]. En el caso de L_3, las cadenas son programas en lenguaje C, por lo que los caracteres con que se forman no son símbolos simples, sino compuestos, como palabras reservadas u operadores como `+=`. L_5 es una colección de palabras reservadas de una versión antigua de Basic y puede suponerse que el alfabeto son los caracteres individuales o que las palabras mismas son parte del alfabeto, que es lo que generalmente se supone en lenguajes de programación.

Con la definición dada de lenguaje, ahora es posible proceder a clasificarlos, partiendo de los más simples a los más complejos. Es posible hacer esto de dos formas:

- definiendo las reglas que generan el lenguaje L o
- definiendo una máquina que sólo acepta las cadenas de L.

Ambos enfoques son equivalentes, necesarios y complementarios.

1.1.3. Operaciones con lenguajes

Siendo los lenguajes conjuntos, es natural que se hereden las operaciones unión, intersección y complemento, considerando el lenguaje Σ^* como el uni-

[1] Aunque en la práctica, los programas válidos en C son un conjunto finito debido a las limitaciones técnicas de los compiladores, pero en la teoría de lenguajes siempre se supone medios de almacenamiento infinitos.

verso. Estas operaciones se aplican tal y como se acostumbran. Para cada operación, en el contexto de los lenguajes se le da una interpretación y notación conveniente.

En el caso de la unión de dos lenguajes L_1 y L_2, se acostumbra utilizar como notaciones equivalentes $L_1 \cup L_2$ y $L_1 + L_2$, preferentemente esta última en teoría de lenguajes formales.

Adicionalmente, para lenguajes se definen la concatenación y la cerradura. Sean L_1 y L_2 lenguajes.

Definición 10. *La* concatenación *de los lenguajes L_1 y L_2 se define como la concatenación de cada pareja posible de cadenas de ambos.*

$$L_1 L_2 = \{r \cdot s / r \in L_1 \quad y \quad s \in L_2\}$$

En caso de concatenar un lenguaje consigo mismo, equivale a escribir $LL = L^2$. Debe ser claro que tanto L como L^2 son subconjuntos de Σ^*. De aquí deriva la definición de

Definición 11. *La* potencia *n de un lenguaje L es la concatenación de n copias de L. Inductivamente, $L^0 = \{\epsilon\}$ y $L^{n+1} = L\,L^n$ para $n \geq 0$.*

Ejemplo 11. *Si $\Sigma = \{0, 1\}$ y $L = \{00, 1\}$, entonces*

$$L^0 = \{\epsilon\} \tag{1.5}$$
$$L^1 = \{00, 1\} \tag{1.6}$$
$$L^2 = \{0000, 001, 100, 11\} \tag{1.7}$$
$$\vdots \tag{1.8}$$

Definición 12. *La* cerradura *del lenguaje L se define como la unión de todas sus potencias:*

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Es importante entender que el conjunto vacío $\emptyset$ es distinto del lenguaje que sólo posee la cadena vacía $\{\epsilon\}$.

1.2. Autómatas Finitos Deterministicos

Un *autómata finito determinístico* (AFD) es un formalismo o modelo matemático que simula una máquina que lee, de izquierda a derecha y uno por uno, los símbolos de una cadena finita escrita en una cinta de entrada. La máquina *acepta o no* la cadena en la cinta de entrada, significando que dicha cadena pertenece o no al lenguaje. De esta forma, las cadenas aceptadas pertenecen al lenguaje y las no aceptadas están fuera de este.

Formalmente, un AFD consta de:

1. Q un conjunto *finito* de estados,
2. Σ un alfabeto fjo,
3. $\delta : Q \times \Sigma \to Q$ la función de transición,
4. $q_0 \in Q$ el estado inicial y
5. $F \subseteq Q$ el conjunto de los estados finales o de aceptación (puede ser vacío).

De esta manera, un AFD es una quíntupla $(Q, \Sigma, \delta, q_0, F)$ junto con una cinta de entrada donde se escribe una cadena de caracteres que será procesada.

La palabra determinístico se refiere a que δ es una función, pues a cada pareja (q, x) le asocia un único estado en Q. De esta manera, la transición siempre es fija desde un estado con el mismo símbolo. Las restricciones en la definición del AFD determinan el tipo de lenguajes que es capaz de procesar. Más adelante, se definirán autómatas más poderosos que son capaces de aceptar una cantidad más amplia de lenguajes.

Estrictamente, δ es una función total: está definida para toda pareja $(q, a) \in Q \times \Sigma$. Por comodidad, en estas notas también se permite que δ no esté definida para algunas parejas (una función parcial); en ese caso, si el autómata no tiene transición para el símbolo que lee, se detiene y rechaza la cadena. En la sección 1.2.2 se verá que todo AFD parcial puede completarse sin cambiar el lenguaje que acepta, así que ambas convenciones son equivalentes.

Si el estado inicial q_0 es a su vez uno de los estados finales ($q_0 \in F$), entonces eso significa que el AFD acepta la cadena vacía ϵ pues no se requiere ningún símbolo para llegar a un estado de aceptación.

Es posible que existan estados a los que no posible llegar a partir del estado inicial. Estos se consideran *estados inalcanzables* y pueden ser omitidos del autómata.

1.2.1. Representación de AFD

Hay dos maneras de representar AFD, con *tablas de transición*, escribiendo una matriz donde a cada fila le corresponde un estado y a cada columna un símbolo de Σ, o con un *diagrama de transición*, dibujando un grafo que represente los estados y las transiciones entre ellos.

La tabla 1.1 muestra las dos versiones de representación del mismo autómata. Se usa la convención de que el estado inicial se indica con $\mapsto$ y los estados finales con $*$. El guión (símbolo –) se emplea para las indicar transiciones no definidas. En el diagrama, la flecha volante que llega a q_0 indica el estado inicial y el círculo de doble línea indica estados finales.

Las tablas de transiciones son las adecuadas al momento de programar computadoras que simulen autómatas. Un archivo de texto con la tabla de transiciones es más práctico, pero el diagrama es más efectivo para reconocimiento humano. Unos ejemplos mostrarán más sobre las representaciones.

Tabla

δ	0	1
$\mapsto q_0$	–	q_1
$*q_1$	q_1	–

Diagrama

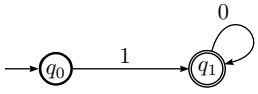


Tabla 1.1: Dos formas de representar las transiciones para reconocer las potencias de 2 en binario.

Tabla

δ	0	1
$\mapsto *q_0$	q_1	q_0
q_1	q_0	q_1

Diagrama

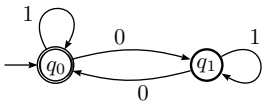


Tabla 1.2: Tabla y diagrama de transiciones de un autómata que reconoce cadenas de ceros y unos que tienen una cantidad par de ceros.

Ejemplo 12. *Sea el AFD* $(Q, \Sigma, \delta, q_0, F) = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$*, con dos transiciones definidas como* $\delta(q_0, 1) = q_1$ *y* $\delta(q_1, 0) = q_1$*. Este autómata de 2 estados acepta las cadenas de ceros y unos que comienzan con un 1 y terminan con puros ceros, es decir, las potencias de 2 en binario. Este autómata es el que se representa en la tabla 1.1. El estado* q_0 *es inicial y* q_1 *el único final.*

En este ejemplo, si se intenta con las cadenas 1, 10, 100, 1000 y todas las de esa forma, se llega al estado q_1 *y se aceptan las cadenas. Si la cadena de entrada es 10010, al intentar procesar el segundo 1 el autómata queda sin poder proceder, pues no hay transición en el estado final para el símbolo 1, por lo que la cadena no se acepta.*

Ejemplo 13. *Sea el AFD* $(Q, \Sigma, \delta, q_0, F) = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_0\})$*, con dos transiciones definidas como* $\delta(q_0, 1) = q_0$*,* $\delta(q_0, 0) = q_1$*,* $\delta(q_1, 1) = q_1$ *y* $\delta(q_1, 0) = q_0$*. Este autómata acepta las cadenas de ceros y unos que tienen una cantidad par de ceros. Este autómata es el que se representa en la tabla 1.2. De los dos estados, el* q_0 *es inicial y final a la vez.*

Para este caso, las cadenas 00, 100, 010, 001 y todas sus combinaciones serán aceptadas. Incluso la cadena vacía se acepta, pues el estado inicial es final a la vez.

El ejemplo 13 es interesante pues muestra una técnica común en el diseño de autómatas. El estado q_0 identifica cadenas con cantidad par de ceros, mientras que el estado q_1 reconoce las de cantidad impar. Así, es posible tener estados para identificar cadenas con varias características a la vez. El siguiente ejemplo es común en el reconocimiento de lenguajes.

Ejemplo 14. *Consideremos el languaje formado por todas las cadenas de símbolos en el alfabeto* $\Sigma = \{a, b, c, \ldots, y, z\}$ *que contienen la subcadena aei al menos*

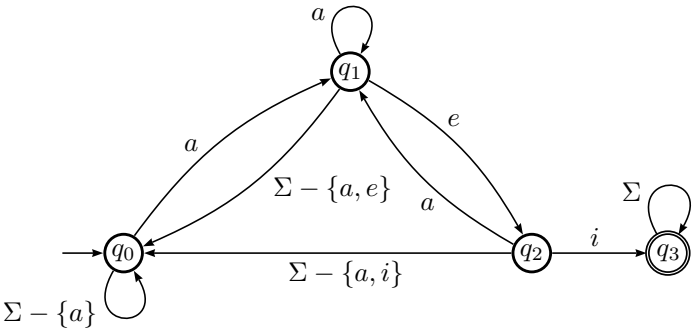


Figura 1.1: AFD que acepta cadenas que contienen la subcadena «*aei*».

una vez. Por ejemplo las cadenas kaeiyl, aei, aaeeaeiaei, kihraejaei y similares.

En este caso, el AFD que acepta tal lenguaje debe tener algún camino de estados que asegure la presencia de la subcadena de interés, al tiempo de permitir la formación de cualquier cadena válida. La figura 1.1 muestra el AFD, donde la ruta $q_0 \to q_1 \to q_2 \to q_3$, con los símbolos a, e e i, identifica la subcadena aei. Una vez en q_3, cualquier símbolo mantiene la aceptación.

1.2.2. Completitud de la función de transición

Como se observa de la representación en la tabla 1.1, la función de transición no está definida para los casos $\delta(q_0, 0)$ y $\delta(q_1, 1)$. Estas entradas en la tabla de transición solo tienen un guión. Eso motiva una definición importante.

Definición 13. *Una función de transición es completa si está definida para toda combinación de valores de su dominio $Q \times \Sigma$.*

Retomemos el ejemplo 12 y completemos la función de transición en el ejemplo 15.

Ejemplo 15. *Al AFD del ejemplo 12 se le completa agregando un estado no final q_2 y las transiciones*

$$\begin{aligned}\delta(q_0, 0) &= q_2\\ \delta(q_1, 1) &= q_2\\ \delta(q_2, 0) &= q_2\\ \delta(q_2, 1) &= q_2.\end{aligned}$$

Las últimas dos transiciones se agregan debido a que de no hacerlo, δ estaría incompleta para el estado q_2. Este nuevo AFD se representa en la tabla 1.3.

Completar la función de transición de un AFD es importante para ciertas operaciones sobre lenguajes que se estudiarán más adelante.

Tabla

δ	0	1
$\mapsto q_0$	q_2	q_1
$*q_1$	q_1	q_2
q_2	q_2	q_2

Diagrama

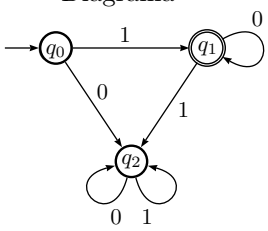


Tabla 1.3: Autómata completo, presentado incompleto en el ejemplo 12.

1.2.3. Autómata Procesando Cadenas

Sea la cadena 11010 escrita en la cinta de entrada. Un AFD inicia su ejecución en el estado q_0, y lee el primer símbolo en la cadena de entrada, el 1. Si $\delta(q_0, 1) = q_i$, entonces se cambia al estado q_i en espera de la lectura del siguiente símbolo, el segundo 1. Si no está definida la transición $\delta(q_0, 1)$ entonces la cadena se rechaza. El proceso continúa, leyendo cada símbolo en la cinta de entrada y cambiando de estado conforme la función de transición, hasta que la cadena termina. Si la cadena se termina y el estado actual, digamos q_f, es uno de los estados finales, es decir, $q_f \in F$, entonces la cadena 11010 es *aceptada* por el AFD.

Si en el transcurso de la ejecución se llega a un estado a partir del cuál no está definida la transición para el símbolo que se lee, el autómata se detiene y rechaza la cadena. Si la cadena se termina pero el autómata no está en uno de los estados finales, la cadena tampoco es aceptada.

Aunque la función de transición procesa caracter por caracter, en general es más cómodo pensar que procesa cadenas completas, por ejemplo $\delta(q_0, 11010)$ y que el resultado sea el estado al que se llega con la cadena. Esto puede formalizarse inductivamente. Sea w una cadena formada por símbolos de Σ tal que

$$w = xa$$

con $a \in \Sigma$ el último símbolo de w. A la subcadena x se le llama prefijo. Sea $\hat{\delta} : Q \times \Sigma^* \to Q$ una función auxiliar definida recursivamente como

$$\hat{\delta}(q, \epsilon) = q \tag{1.9}$$

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a) \tag{1.10}$$

El resultado de procesar la cadena w es el estado $q_k = \hat{\delta}(q_i, w)$. Lo que hace la regla (1.10) es procesar la subcadena x a partir de q_i con $\hat{\delta}$ para llegar al estado q_j y luego utilizar δ para procesar el símbolo a y llegar al estado destino con la transición normal $\delta(q_j, a) = q_k$. Esta transición genérica se esquematiza en la figura 1.2.

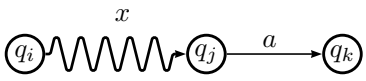


Figura 1.2: La regla (1.10) procesa la cadena x desde q_i, pasando por $|x|$ transiciones hasta llegar al estado q_j. Luego transita con $\delta(q_j, a) = q_k$.

Ejemplo 16. *Considérense de nuevo el lenguaje L_6 del ejemplo 10, las cadenas de 3 dígitos que representan múltiplos de 5 desde 100 hasta 195. Sea el autómata AFD $A = (Q, \Sigma, \delta, q_0, F)$ que acepta cadenas de ese tipo. Del estado q_0, se requiere una transición que procese el primer dígito, que debe ser un 1, con $\delta(q_0, 1) = q_1$. Luego del estado q_1 debe ser posible pasar al estado q_2 con cualquier dígito. Del estado q_2 se debe poder pasar al estado final q_3 con un 0 o con un 5.*

Es deseable pensar en las transiciones que sucesivamente tienen que darse para una cadena, por ejemplo 125. $\hat{\delta}(q_0, 125)$ se calcula como sigue, utilizando las reglas (1.9) y (1.10).

La cadena $w = 125$ se descompone primero como $x = 12$ y $a = 5$, luego se repite el proceso, descomponiendo la cadena 12 como $x = 1$ y $a = 2$. Finalmente, la cadena $w = 1$ se descompone como $x = \epsilon$ y $a = 1$ siendo el caso base. A partir de allí, sólo se sustituyen las transiciones definidas para δ.

Los pasos son:

$$\begin{aligned}
\hat{\delta}(q_0, 125) &= \delta(\ \hat{\delta}(q_0, 12)\ , 5) && x = 12 \quad a = 5 \\
&= \delta(\delta(\ \hat{\delta}(q_0, 1)\ , 2), 5) && x = 1 \quad a = 2 \\
&= \delta(\delta(\delta(\ \hat{\delta}(q_0, \epsilon), 1), 2), 5) && x = \epsilon \quad a = 1 \\
&= \delta(\delta(\delta(\ q_0, 1), 2), 5) && \textit{por la regla (1.9)} \\
&= \delta(\delta(q_1, 2), 5) && \delta(q_0, 1) = q_1 \\
&= \delta(q_2, 5) && \delta(q_1, 2) = q_2 \\
&= q_3
\end{aligned}$$

★ Viendo que es formalmente posible definir cómo un AFD procesa cadenas, en lo sucesivo nos referiremos indistintamente a δ como si procesara símbolos individuales o cadenas completas.

1.2.4. El lenguaje de un AFD

De manera empírica se ha mencionado que el AFD acepta cadenas, sin mencionar el lenguaje que esto define. Es momento de formalizar.

Definición 14. *El* lenguaje que acepta el AFD $A = (Q, \Sigma, \delta, q_0, F)$, *denotado con $L(A)$, se define como*

$$L(A) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \in F\} \tag{1.11}$$

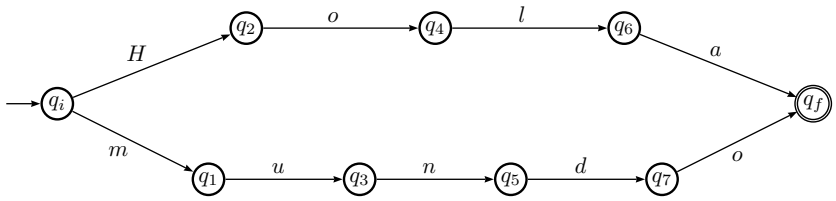


Figura 1.3: Autómata que reconoce un lenguaje con sólo dos cadenas.

que son todas las cadenas del alfabeto que al ser procesadas, inciando en el estado q_0, terminan en algún estado final del AFD. Se dice que $L(A)$ es un lenguaje regular.

Con esta definición, entonces el lenguaje definido en el ejemplo 12 es un lenguaje regular pues es el aceptado por un AFD. Cualquiera que sea el AFD definido, el conjunto de cadenas que partiendo del estado inicial alcancen un estado final al procesar el último símbolo es un lenguaje regular.

Ejemplo 17. *El autómata que reconoce el lenguaje {`Hola,mundo`} es el de la figura 1.3. De esta manera, es posible definir autómatas que acepten las palabras reservadas de un lenguaje.*

Los AFD se utilizan constantemente en la práctica. Desde la verificación de la entrada válida de un usuario o los primeros pasos que todo compilador da, al realizar lo que se conoce como el análisis léxico, que es el reconocimiento de cada elemento individual en el código de un programa, como operadores, palabras reservadas, identificadores del usuario, constantes, delimitadores de ámbito y todo aquello de que consta cierto lenguaje. Cada elemento de algún tipo es llamado token.

Ejemplo 18. *Un identificador es una cadena alfanumérica que se utiliza para nombrar una función, variable, etiqueta o macro. Todo identificador comienza con una letra o con el símbolo _ y puede continuar con letras, dígitos y _. Si comienza con _ (simple, doble o más), debe contener al menos una letra*[2]*. Suponiendo que no hay restricciones en la longitud de los identificadores, se puede hacer un AFD T que reconozca sólo cadenas válidas.*

Sea L el conjunto de las letras válidos para usarse en un identificador y D el conjunto de los números del 0 al 9. Sea el alfabeto $\Sigma =$ `Letra` $\cup$ `Dígito` $\cup \{_\}$, es decir, toda letra, dígito (del cero al nueve) y el guión bajo.

$T = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_2\})$, con las transiciones $\delta(q_0, _) = q_1$; $\delta(q_0, x) = q_2$ para toda letra $x \in$ `Letra`; $\delta(q_1, x) = q_1$ para toda $x \in$ `Dígito` $\cup \{_\}$; $\delta(q_1, x) =$

[2]Existe toda una variedad de reglas y convenciones bien establecidas, además de las variantes que cada vendedor de compiladores ofrece, sobre cómo nombrar identificadores. Por ejemplo, en C los identificadores que comienzan con doble guión bajo, o con guión bajo seguido de mayúscula, están reservados para la implementación (el compilador y su biblioteca estándar).

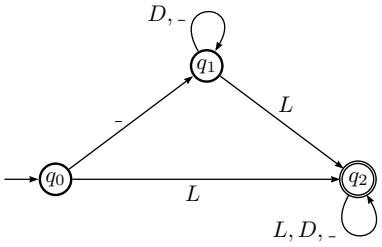


Figura 1.4: Autómata que acepta identificadores de un lenguajes de programación. La D representa al conjunto de los dígitos y L el de las letras. El estado q_1 indica que todavía no aparece ninguna letra.

q_2 para toda $x \in$ `Letra`, y $\delta(q_2, x) = q_2$ para toda $x \in \Sigma$. El estado q_1 recuerda que la cadena empezó con _ y que todavía no aparece ninguna letra; q_2 indica que ya se tiene un identificador válido. A este autómata le corresponde el diagrama de transiciones de la figura 1.4.

1.2.5. Descripción instantánea de un AFD

Cuando un autómata está procesando una cadena, es posible conocer el estado que guarda la lectura de los símbolos. Sea la cadena que se procesa

$$w = a_1 a_2 a_3 \cdots a_r$$

con $a_i \in \Sigma$ y con i de 1 a r. La descripción instantánea del autómata se compone del estado actual q_j y el resto de la cadena que está por ser procesada.

Es claro que al inicio, en el estado inicial q_0 y con toda la cadena w por ser procesada, la descripción instantánea es

$$q_0 a_1 a_2 a_3 \cdots a_r$$

y si existe la transición $\delta(q_0, a_1) = q_k$, entonces la siguiente descripción instantánea es

$$q_k a_2 a_3 \cdots a_r$$

donde el símbolo inicial a_1 ya fue procesado.

Para un AFD y cierta cadena w, hay una única sucesión de descripiciones instantáneas, que termina con $q_f \epsilon$, con q_f uno de los estados finales, en caso de que la cadena sea aceptada.

1.2.6. AFD mínimos

Es posible construir muchos AFD que acepten el mismo lenguaje. Lo ideal es hacer uso del más sencillo de estos. Por lo general, se da preferencia a los que tienen menos estados, por lo que se requiere un criterio para reducir el número de estados de un AFD sin afectar el lenguaje que acepta. El criterio es el siguiente.

Definición 15. *Sea $A = (Q, \Sigma, \delta, q_0, F)$ un AFD con δ completa. Dos estados p y q de A son* equivalentes *si para toda cadena $w \in \Sigma^*$ se cumple que $\hat{\delta}(p, w) \in F$ si y sólo si $\hat{\delta}(q, w) \in F$.*

Esto significa que, partiendo de p o de q, cada cadena lleva a aceptar en ambos casos o en ninguno. En particular, con $w = \epsilon$ se ve que dos estados equivalentes son ambos finales o ambos no finales. Si existe una cadena w que lleva a aceptar desde uno de ellos y no desde el otro, se dice que w *distingue* a p y q.

Cuando dos estados son equivalentes, pueden sustituirse por un único estado. La equivalencia de estados es una relación de equivalencia (reflexiva, simétrica y transitiva), así que cada clase de estados equivalentes puede sustituirse por un único estado. Antes de hacerlo conviene eliminar los estados inalcanzables, que no afectan al lenguaje.

Algoritmo de minimización

Supóngase que δ es completa y que ya se eliminaron los estados inalcanzables. El siguiente algoritmo, conocido como *llenado de tabla*, encuentra todas las parejas de estados equivalentes.

1. Se construye una tabla con una casilla para cada pareja no ordenada $\{p, q\}$ de estados distintos.
2. Se marca toda pareja formada por un estado final y uno no final, pues la cadena ϵ los distingue.
3. Se repite el siguiente paso hasta que ya no se marque ninguna pareja nueva: se marca $\{p, q\}$ si existe un símbolo $a \in \Sigma$ tal que la pareja $\{\delta(p, a), \delta(q, a)\}$ ya está marcada. La justificación es que si w distingue a $\delta(p, a)$ y $\delta(q, a)$, entonces aw distingue a p y q.
4. Las parejas que quedan sin marcar son equivalentes. Cada clase de estados equivalentes se sustituye por un único estado, que hereda las transiciones de sus miembros.

El AFD que se obtiene es mínimo y, salvo el nombre de los estados, es único: es el que describe el teorema de Myhill-Nerode (proposición 7). Por ejemplo, en el AFD del código 1.1, completado con el estado de error, los estados 3 y 4 son equivalentes, pues ninguno es final y ambos van al estado 6 con un dígito y al error con cualquier otro símbolo. Por una razón similar también son equivalentes los estados 2 y 5, así que el autómata mínimo tiene dos estados menos.

1.2.7. Análisis Léxico

En análisis léxico es un problema al que se enfrenta tanto un compilador como un editor de textos, un corrector ortográfico, un buscador de bases de datos, un navegador de internet y toda aplicación que utiliza cadenas de caracteres para identificar patrones o similitudes. Es la aplicación más directa de los AFD.

El código de un analizador léxico se basa directamente en la tabla de transiciones del AFD, solo que esta debe completarse con las salidas adecuadas para

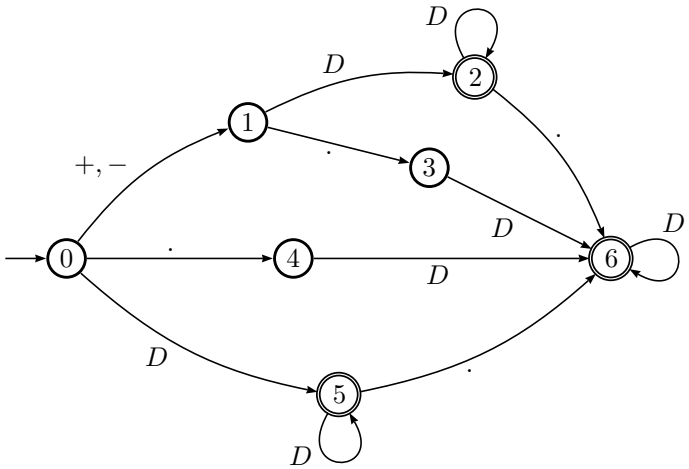


Figura 1.5: Autómata Finito Determinístico que acepta números flotantes, del ejemplo 19. En la transición del estado 0 al 1, por etiqueta $+, -$ debe entenderse que cuaquier signo es válido para la transición. La D hace referencia a cualquier dígito.

cada entrada de la tabla donde no hay transición definida, lo que hasta ahora se ha marcado con el signo –.

Lo que hace el analizador léxico es asignar a una variable el índice del estado actual y actualizarla cada ciclo hasta que se termine la cadena y comprobar que se terminó en un estado final.

Estrictamente, esto describe un *reconocedor*: decide si una cadena completa pertenece al lenguaje. Un analizador léxico real recibe todo el texto del programa y debe dividirlo en tokens. Para ello sigue la regla del *prefijo más largo* (en inglés, *maximal munch*): avanza mientras exista transición, recuerda el último estado final por el que pasó y, cuando ya no puede avanzar, entrega como token el prefijo más largo que fue aceptado y vuelve a empezar en el estado inicial a partir del símbolo siguiente. Así, en `x+=12` se reconoce `+=` como un solo token, y no como `+` seguido de `=`.

Ahora ya se ve la primera aplicación práctica de las teorías descritas hasta el momento. Programas sencillos pero con gran poder se pueden desarrollar haciendo uso de teoría bien desarrollada.

Ejemplo 19. *En muchos lenguajes se utilizan números de punto flotante, tales como 2.3, -7.23, +0.15, 456.23 o .12 y son consideradas constantes. Esas cadenas son un lenguaje regular, ya que son aceptadas por el AFD de la figura 1.5.*

A partir de la tabla de transiciones se construye el analizador del código 1.1. La tabla de transiciones ha sido codificada en el arreglo bidimensional `Tabla`*. El código es simple, ya que todo el conocimiento de cómo son las cadenas del*

lenguaje está codificado en la tabla de transiciones (el arreglo `Tabla`*).*

En el programa completo, cuya entrada es un arreglo de cadenas que representan flotantes, ante la entrada `123.45 -.56 rt45 6y4 2*3 654`, el programa muestra:

```
Analizando 123.45 ... se acepta
Analizando -.56 ... se acepta
Analizando rt45 ... no se acepta
Analizando 6y4 ... no se acepta
Analizando 2*3 ... no se acepta
Analizando 654 ... se acepta
```

Claro, este es un ejemplo con fines didácticos, bastante simple. Un compilador toma en cuenta notación científica, sufijo, longitud de cadena y detalles que oscurecerían la idea del analizador, por lo que se omiten en el ejemplo.

Otra forma de terminar, en el caso de un AFD que tenga varios estados finales, es definiendo una transición para el caracter que simula el fin de la cadena, el caracter nulo `\0`, que lleva a un único estado final. El código del analizador habría que cambiarlo ligeramente y agregar una columna más a la tabla de transiciones.

En el caso más general de un analizador léxico que de procesar un archivo completo, las posibilidades del `switch` deben considerar los todos los caracteres, además de que los símbolos + y - pueden ser entendidos como operadores unarios y no parte del número.

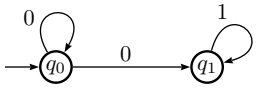
Lo que es importante notar es que si se define otro lenguaje regular, con su propia tabla de transiciones, lo único que hay que cambiar en el código del analizador es el arreglo `Tabla` y los casos del `switch`, así que cada analizador léxico tiene esa misma forma. Es un proceso tan mecánico que existen generadores automáticos de analizadores léxicos, por ejemplo Lex, Flex o Ragel. No se necesita escribir códigos como el del ejemplo anterior, sólo describir el lenguaje con una expresión regular, para que el generador construya un código que es bastante eficiente. ★

Un buen ejercicio es extender este lenguaje para que acepte notación científica.

1.2.8. Autómata finito no determinístico

Los AFD son el modelo más básico de una máquina que acepta cadenas para definir un lenguaje. Con la intención de darle poder, se pueden cambiar algunas cosas, particularmente el comportamiento de la función de transición δ. Las variantes que más interesan son las siguientes.

Consideremos el siguiente fragmento de un autómata que tiene transiciones distintas para $\delta(q_0, 0) = q_0$ y $\delta(q_0, 0) = q_1$.



Código 1.1: Analizador léxico para números flotantes simples reconocidos por el autómata de la figura 1.5. La tabla de transiciones se ha ampliado para poder considerar la salida en caso de un error. Encontrar el fin de cadena (caracter nulo) es lo que da fin al ciclo, es decir, el `case` de la línea 27 manda directo fuera del ciclo (línea 32).

```
1  int Tabla[][3]={ {5, 1, 4}, {2,-1, 3},
2                   {2,-1, 6}, {6,-1,-1},
3                   {6,-1,-1},{5,-1,6},
4                   {6,-1,-1}};
5  int AnalizarFlotante(const char *Cad)
6  {
7          char c;
8          int Estado=0, Simbolo, Error=-1;
9          int Digito=0, Signo=1, Punto=2;
10
11         do {
12                 c = *Cad++;
13                 switch(c) {
14                         case '0': case '1':
15                         case '2': case '3':
16                         case '4': case '5':
17                         case '6': case '7':
18                         case '8': case '9':
19                                 Simbolo = Digito;
20                                 break;
21                         case '-': case '+':
22                                 Simbolo = Signo;
23                                 break;
24                         case '.':
25                                 Simbolo = Punto;
26                                 break;
27                         case '\0':
28                                 continue;
29                         default:
30                                 return NO_SE_ACEPTA;
31                 }
32                 Estado = Tabla[Estado][Simbolo];
33                 if ( Estado == Error )
34                         return NO_SE_ACEPTA;
35         } while  ( c != '\0' );
36
37         if ( Estado == 2  ||  Estado == 5  ||
38              Estado == 6 )
39                 return SE_ACEPTA;
40
41         return NO_SE_ACEPTA;
42 }
```

¿Cómo decide el autómata qué hacer? Si la cadena inicia con 0, ¿continúa la ejecución en q_0 o se pasa a q_1? Es mejor pensar que el autómata no decide qué camino seguir, sino que continúa al mismo tiempo por ambas transiciones. Así, lo correcto es pensar que la transición es $\delta(q_0, 0) = \{q_0, q_1\}$, llegando a los dos estados con el mismo símbolo. ★

Este tipo de autómata finito es NO determinístico y es similar al AFD, pero con la capacidad de estar al mismo tiempo en varios estados. Esta imagen de *estar en varios estados a la vez* es sólo una forma de razonar: el autómata no ejecuta procesos en paralelo, sino que acepta una cadena si *existe* al menos una forma de elegir las transiciones que lleve a un estado final. Por eso también se dice que un autómata no determinístico *adivina* el camino correcto.

Además, para evitar conflictos con la definición de función, se define $\delta(q, 0) = \{p, q\}$, es decir, hay una única transición de $(q, 0)$ al conjunto $\{p, q\} \in 2^Q$.

Para hacer esto es necesario modificar la definición de AFD, particularmente la definición de la función de transición δ de la página 16. Ahora, como de un estado podemos ir a varios con el mismo símbolo, se define otro tipo de función de transición, una que puede ir a un conjunto de estados, por lo que su rango es el conjunto potencia de Q:

$$\delta : Q \times \Sigma \to 2^Q \tag{1.12}$$

Como ahora no está determinado a qué estado pasa el AFD con cada símbolo, llamamos al modelo *Autómata Finito No Deterministico*, o AFN. La nueva función de transición permite ir a cualquier cantidad de estados con el mismo símbolo como en

$$\delta(q, a) = \{p_1, p_2, \ldots, p_k\} \subseteq Q$$

por eso es que el rango de δ es 2^Q, el conjunto de todos los subconjuntos de Q. El conjunto puede ser vacío: $\delta(q, a) = \emptyset$ indica que no hay transición.

Habiendo cambiado la naturaleza de la función de transición, es necesario extenderla de nuevo para que procese cadenas de caracteres, como se hizo en la sección 1.2.3 para poder definir el lenguaje aceptado por un AFD. La extensión que se hizo con $\hat{\delta}$ lleva por un único camino de estados con una cadena, terminando en un sólo estado, pero en este caso no es así. A partir de cada estado es posible seguir varias transiciones, llegando a un conjunto de estados tras procesar la cadena.

Para que la función de transición extendida procese cadenas, sea como antes $w \in \Sigma^*$ una cadena formada por símbolos de Σ tal que $w = xa$, con $a \in \Sigma$ el último símbolo de w. Este caso se ilustra en la figura 1.6.

Sea $\hat{\delta} : Q \times \Sigma^* \to 2^Q$ la función de transición extendida, definida inductivamente, con el caso base para la cadena vacía:

$$\hat{\delta}(q, \epsilon) = \{q\}$$

y como cada transición resulta en un conjunto de estados, sea

$$\hat{\delta}(q, x) = \{p_1, p_2, \ldots, p_k\}$$

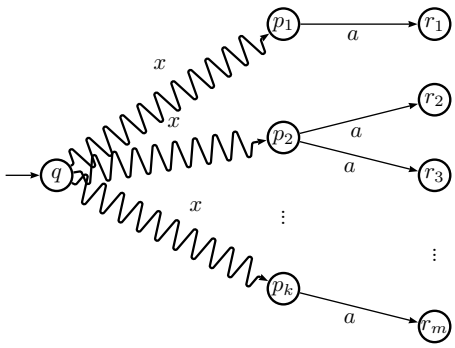


Figura 1.6: Paso inductivo de la extensión de δ a $\hat{\delta}$. Para procesar $\hat{\delta}(q, w)$ se reduce la cadena $w = xa$ en un símbolo y primero se procesa la subcadena $\hat{\delta}(q, x)$ y luego se procesa $\delta(p_i, a)$.

tal conjunto de estados a los que es posible llegar cuando se procesa la cadena x. Finalmente se tiene

$$\begin{aligned}\hat{\delta}(q, w) &= \bigcup_{i=1}^{k} \delta(p_i, a) \\ &= \{r_1, r_2, \ldots, r_m\}\end{aligned} \tag{1.13}$$

El lenguaje aceptado por un AFN $N = (Q, \Sigma, \delta, q_0, F)$ es similar al de un AFD, pero en este caso, como con cada cadena se pueden seguir diversas rutas con las transiciones, se considera que una cadena es aceptada si al menos una de esta rutas llega a un estado final al terminar de procesar la cadena. De esta forma,

$$L(N) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \cap F \neq \emptyset\} \tag{1.14}$$

Con esta capacidad de elegir entre varios caminos, los AFN parecen más poderosos que los AFD. Si esto fuera cierto, entonces, al ser más poderosos, los lenguajes aceptados por los AFD serían menos que los lenguajes aceptados por los AFN. Habría al menos un lenguaje aceptado por un AFN que no podría ser aceptado por un AFD.

★ Aunque el razonamiento anterior nos haría avanzar de los Lenguajes Regulares (los aceptados por los AFD) a un conjunto mayor, lo cierto es que para cada lenguaje que es aceptado por algún AFN, existe un AFD que también lo acepta, por lo que ambos modelos, el determinístico y el no determinístico son quivalentes. Si bien no son más poderosos, sí son más fáciles de construir.

La prueba de la equivalencia entre AFD y AFN es un algoritmo para la conversión de uno en otro. Dado un AFD, construir un AFN que acepta exactamente el mismo lenguaje y viceversa, dado el AFN, construir el AFD.

Equivalencia entre AFD y AFN

El hecho de que los AFN NO son más poderosos que los AFD, es decir, NO amplían el conjunto de lenguajes que se pueden reconocer es muy importante. Su equivalencia se plasma de manera formal.

Proposición 4. *El conjunto de lenguajes aceptados por los AFD es el mismo que los aceptados por los AFN.* ★

Demostración. Lo que hay que hacer para mostrar que $L(AFD) = L(AFN)$ es mostrar equivalencias entre los dos autómatas, es decir, dado un AFN N arbitrario mostrar un AFD A equivalente (que acepte el mismo lenguaje) y viceversa. Es necesario construir, en ambos sentidos, un autómata adecuado y demostrar que sí acepta cada cadena que el original acepta y ni una más.

Primer caso: Es evidente que todo AFD A es un AFN N en el que no hay transiciones múltiples para un mismo estado y símbolo, es decir, cada transición $\delta(q, a) = p$ se convierte en $\delta(q, a) = \{p\}$ en el nuevo AFN, pues cada estado q en el AFD se convierte en el conjunto $\{q\}$ para el AFN N. Las transiciones son las mismas y los estados finales conservan su condición. Por ello, el lenguaje $L(A) = L(A_N)$.

Segundo caso: La construcción de un AFD a partir de un AFN requiere proceder con más cuidado: Sea $N = (Q_N, \Sigma, \delta_N, q_N, F_N)$ un AFN. El nuevo AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ se construye de la siguiente manera.

- El alfabeto debe ser el mismo para ambos, pues se trata del mismo conjunto de cadenas.
- El estado inicial coincide, $q_D = \{q_N\}$, como es de esperarse.
- Q_D es el conjunto potencia de Q_N, $Q_D = 2^{Q_N}$. Es posible que no todos los subconjuntos se conviertan en estados alcanzables y serán eliminados del autómata final.
- Los estados finales del AFD F_D son todos aquellos estados de 2^{Q_N} que incluyen un estado final del AFN, es decir, $F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$.
- Para cada subconjunto de estados $S \subseteq Q_N$ y símbolo de entrada $a \in \Sigma$,

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a) \tag{1.15}$$

  y esto representa la manera de construir (y seleccionar) los estados alcanzables. Con palabras: para cada subconjunto S de estados, se revisa cada estados $p \in S$y a qué estados se llega con $\delta_N(p, a)$ y se calcula su unión.

Ahora falta demostrar que $L(D) = L(N)$. Para ello se probará, por inducción sobre la longitud de w, que para *toda* cadena $w \in \Sigma^*$

$$\hat{\delta}_D(q_D, w) = \hat{\delta}_N(q_N, w).$$

Es importante que la afirmación se pruebe para toda cadena y no sólo para las aceptadas: en el paso inductivo se usa para el prefijo x de w, y un prefijo de una cadena aceptada no tiene por qué ser aceptado.

Caso base: $|w| = 0$, es decir, $w = \epsilon$. Por las definiciones de $\hat{\delta}$ en ambos autómatas, $\hat{\delta}_D(q_D, \epsilon) = q_D = \{q_N\} = \hat{\delta}_N(q_N, \epsilon)$.

Hipótesis inductiva: la igualdad se cumple para toda cadena de longitud n o menor.

Sea w una cadena con $|w| = n + 1$, que se descompone como $w = xa$, con $a \in \Sigma$ el último símbolo de w. Por hipótesis inductiva, $\hat{\delta}_D(q_D, x)$ y $\hat{\delta}_N(q_N, x)$ son el mismo conjunto de estados; sea $\{p_1, p_2, \ldots, p_k\}$ tal conjunto. Entonces

$$\begin{aligned}
\hat{\delta}_D(q_D, w) &= \delta_D(\hat{\delta}_D(q_D, x), a) && \text{definición de } \hat{\delta} \text{ en un AFD} \\
&= \delta_D(\{p_1, p_2, \ldots, p_k\}, a) && \text{hipótesis inductiva} \\
&= \bigcup_{i=1}^{k} \delta_N(p_i, a) && \text{construcción (1.15)} \\
&= \hat{\delta}_N(q_N, w) && \text{definición de } \hat{\delta} \text{ en un AFN}
\end{aligned}$$

Finalmente, por la definición de F_D,

$$w \in L(D) \iff \hat{\delta}_D(q_D, w) \in F_D \iff \hat{\delta}_N(q_N, w) \cap F_N \neq \emptyset \iff w \in L(N).$$

□

Reiteremos la importancia del anterior resultado: pese a que parece ganarse poder con el NO determinismo, un autómata no gana poder para aceptar lenguajes, en el sentido de que el conjunto de lenguajes que se pueden reconocer no es mayor que el que ya se reconocía con los AFD.

Ejemplo 20. *Considere el AFN N representado en la figura 1.7 y a partir de allí se construirá un AFD equivalente. Los estados q_0, q_1 y q_2 tienen transiciones no deterministas pues con algún símbolo caen en más de un estado.*

El nuevo autómata se forma a partir de los estados y transiciones originales, agregando algunos más de la siguiente manera. De cada transición no determinista surgen los nuevos estados, por ejemplo a partir de $\delta(q_0, 0) = \{q_0, q_1\}$ se define el un estado nuevo con etiqueta $\{q_0, q_1\}$, que se agrega en una nueva fila de la tabla de transiciones. En la tabla 1.7b ya se ha completado este proceso, agregando 4 estados más. Una manera más económica es construir sólo los subconjuntos alcanzables: se parte de $\{q_0\}$ y se agregan únicamente los subconjuntos que aparecen como destino de alguna transición. Así no se generan filas, como las de q_1 y q_3, que después hay que eliminar por inalcanzables.

Luego, de cada nuevo estado, por decir, $\{q_0, q_1\}$, se calcula qué estados es posible alcanzar estando en q_0 y q_1 con el símbolo 0. Del primer renglón de la tabla se ve que de nuevo se llega $\{q_0, q_1\}$, pues q_1 no tiene transición con 0. Con el símbolo 1, de q_0 se llega a q_2 mientras que de q_1 se llega a $\{q_1, q_3\}$, por lo que se completa el quinto renglón de la tabla:

$$\begin{aligned}
\delta(\{q_0, q_1\}, 1) &= \delta(\{q_0\}, 1) \cup \delta(\{q_1\}, 1) \\
&= \{q_2\} \cup \{q_1, q_3\} \\
&= \{q_1, q_2, q_3\}
\end{aligned}$$

Es importante notar que durante la creación del nuevo autómata, algunos de los estados originales no se requieren más. El mismo estado final q_3 ya no se necesita, al igual que el estado q_1 pues partiendo del estado inicial ya no es alcanzado por ninguna transición.

Los nuevos estados que contienen alguno de los estados finales del autómata original se convierten en estados finales, en este caso, cualquiera que contenga a q_3. El AFD final, equivalente al de la figura 1.7a se muestra en la figura 1.8.

La construcción de subconjuntos puede ser costosa. Un AFN con n estados da lugar a un AFD con hasta 2^n estados, y hay lenguajes para los que esa explosión es inevitable. El ejemplo clásico es el lenguaje de las cadenas binarias cuyo n-ésimo símbolo, contado desde el final, es un 1. Un AFN con $n+1$ estados lo acepta *adivinando* en qué posición empiezan los últimos n símbolos, mientras que cualquier AFD que lo acepte necesita al menos 2^n estados, porque debe recordar los últimos n símbolos leídos. Así, el no determinismo no aumenta el poder de los autómatas, pero sí puede reducir enormemente su tamaño.

1.2.9. Autómata finito con transiciones ϵ

Otra forma de darle versatilidad a un AFD es permitir que pueda pasar de un estado a otro sin procesar símbolos en la cadena de entrada. Esto se logra modificando de nuevo la función de transición δ para que permita transiciones con la cadena vacía, como $\delta(q, \epsilon) = p$. Este tipo de transiciones permite describir autómatas de manera sencilla.

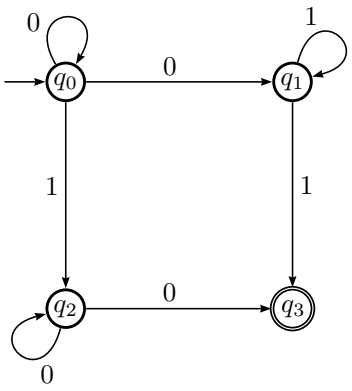
Ejemplo 21. *Un sencillo AFN-ϵ se muestra en la figura 1.9. Este autómata acepta exactamente el lenguaje $1(\epsilon + 0) + 10^*1(\epsilon + 0)$: las cadenas 1 y 10, y las cadenas que empiezan con 1, siguen con cualquier cantidad de ceros, tienen un segundo 1 y, opcionalmente, terminan con un 0.*

Ejemplo 22. *Considérese en autómata que acepta números reales como 123.32, $-.098$ o $+15.$, todos válidos como constantes numéricas en el código de un lenguaje de programación. El alfabeto es $\Sigma = \{0, 1, 2, \ldots, 9, ., +, -\}$. Sea D el conjunto de los dígitos, como en el ejemplo 18. El diagrama de transiciones de este autómata se muestra en la figura 1.10.*

En los autómatas con transiciones ϵ o AFN$-\epsilon$, la definición formal es casi la misma que en los AFN, excepto que la función de transición es $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$, que permite pasar de un estado a otro con la cadena vacía, es decir, sin procesar ningún símbolo de la cinta de entrada.

Para formalizar la definición de los AFN$-\epsilon$, particularmente para definir el lenguaje que aceptan, es necesario extender la función de transición, no solo definir de nuevo su dominio. Para esto, primero se define inductivamente la cerradura de un estado q, escrita como Clos$\{q\}$.

Caso base $q \in \text{Clos}\{q\}$
Inducción si $p \in \text{Clos}\{q\}$ y $r \in \delta(p, \epsilon)$, entonces $r \in \text{Clos}\{q\}$



(a) Diagrama con los estados originales.

δ	0	1
$\mapsto q_0$	$\{q_0, q_1\}$	q_2
q_1	–	$\{q_1, q_3\}$
q_2	$\{q_2, q_3\}$	–
$*q_3$	–	–
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_1, q_2, q_3\}$
$*\{q_1, q_3\}$	–	$\{q_1, q_3\}$
$*\{q_2, q_3\}$	$\{q_2, q_3\}$	–
$*\{q_1, q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_1, q_3\}$

(b) Tabla. Arriba de la línea punteada están las transiciones originales. Abajo los 4 nuevos estados.

Figura 1.7: Autómata Finito No Determinístico. En la tabla de transiciones, debajo de la línea puntueada, aparecen los nuevos estados que se requieren.

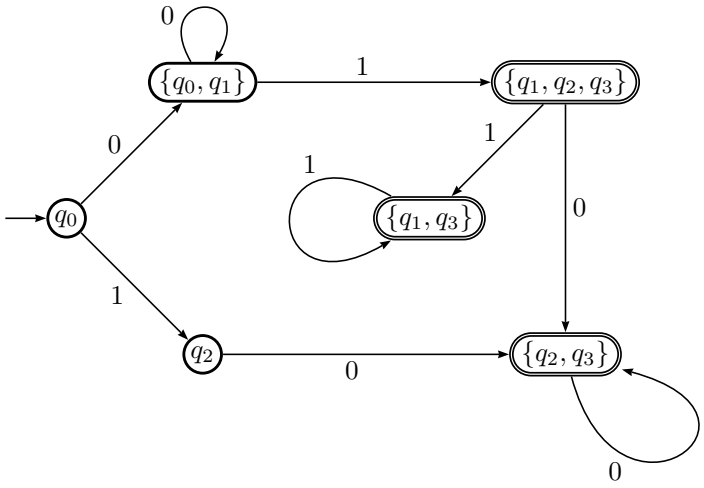


Figura 1.8: Autómafa Finito Determinístico equivalente al de la figura 1.7a. Su tabla de trasiciones se obtiene de la tabla 1.7b luego de eliminar los estados inaccesibles: q_1 y q_3.

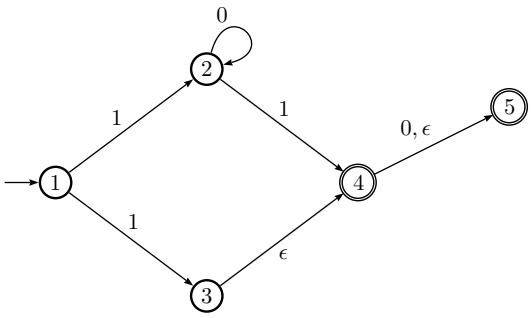


Figura 1.9: Autómata Finito No determinístico con transiciones ϵ. Acepta el lenguaje $1(\epsilon + 0) + 10^*1(\epsilon + 0)$.

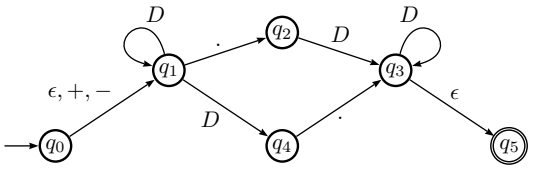


Figura 1.10: Autómata finito no determinístico con transiciones ϵ (AFN-ϵ) que acepta cadenas que representan constantes reales.

De esta manera, la cerradura[3] de un estado q es el conjunto de estados a los que se puede llegar desde q mediante transiciones épsilon.

La figura 1.10 tiene dos transiciones épsilon, una a partir de q_0 y otra de q_3. Por lo tanto, la cerradura de estos estados es $\text{Clos}\{q_0\} = \{q_0, q_1\}$ y $\text{Clos}\{q_3\} = \{q_3, q_5\}$, para el resto de los estados, su cerradura tiene sólo el mismo estado. Si las transiciones épsilon son muchas en un autómata, entonces las cerraduras serán mayores en general.

¿Qué significa que un AFN$-\epsilon$ acepte su entrada? Primero es necesario extender la función de transición a $\hat{\delta}$, modificando δ para que procese cadenas y no sólo símbolos. Ya se han realizado dos extensiones similares, una para los AFD y otra para los AFN, en este caso la extensión debe tomar en cuenta las transiciones ϵ sin que estas contribuyan a la cadena w que se procesa.

El caso base de la definición es $\hat{\delta}(q, \epsilon) = \text{Clos}(q)$, con lo que sólo se avanza por las transiciones etiquetadas con ϵ.

Ahora, para el paso inductivo, sea $w \in \Sigma^*$ una cadena tal que $w = xa$, con $x \in \Sigma^*$ y a el último símbolo de w. Nótese que $a \neq \epsilon$ pues $\epsilon \notin \Sigma$. Entonces, $\hat{\delta}$ se calcula de la siguiente manera.

1. Sea $\hat{\delta}(q, x) = \{p_1, p_2, \ldots, p_k\}$ el conjunto de estados que se alcanzan desde q siguiendo la cadena etiquetada con x. Este camino puede tener varias

[3]Este es un caso particular de la *cerradura transitiva*, definida para relaciones binarias.

trasiciones ϵ.

2. Sea $\bigcup_{i=1}^{k} \delta(p_i, a) = \{r_1, r_2, \ldots, r_m\}$, es decir, se siguen todas las posibles transiciones con el símbolo a desde lo estados en $\hat{\delta}(q, x)$. Esto significa que cada r_j es un estado al que se puede llegar mediante un camino etiquetado con x y luego con a.

3. Por último, se siguen las transiciones ϵ a partir de cada r_j, con

$$\hat{\delta}(q, w) = \bigcup_{j=1}^{m} \text{Clos}(r_j).$$

Ahora ya se puede definir el lenguaje aceptado por un AFN$-\epsilon$, como

$$L(A) = \{w \in \Sigma^* | \hat{\delta}(q_0, w) \cap F \neq \emptyset\} \tag{1.16}$$

Aunque en apariencia, los lenguajes definidos en (1.16) y (1.14) son el mismo, no deben confundirse, pues cada uno utiliza una $\hat{\delta}$ distinta.

Equivalencia entre AFD y AFN$-\epsilon$

Antes de demostrar que los AFD son equivalentes a los AFN$-\epsilon$, estableceremos la manera de eliminar las transiciones$-\epsilon$ y luego la equivalencia será más sencilla de probar. La construcción se asemeja a la de subconjuntos de la sección 1.2.8.

Sea $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ un AFN$-\epsilon$ y construiremos un AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ empleando la cerradura de los estados para eliminar las transiciones. Como antes, se trata del mismo alfabeto Σ. La construcción es:

1. $Q_D = 2^{Q_E}$, el conjunto de todos los subconjuntos de Q_E. En la práctica sólo se construyen los subconjuntos alcanzables desde q_D.

2. $q_D = \text{Clos}\{q_E\}$, el subconjunto de todos los estados a los que se puede llegar desde q_E mediante transiciones ϵ.

3. F_D es el subconjunto de estados en los que al menos uno es un estado final de E, es decir:

$$F_D = \left\{S \in 2^{Q_E} \quad \text{y} \quad S \cap F_E \neq \emptyset\right\}$$

4. La función de transición se calcula de la siguiente forma. Sea $a \in \Sigma$ y $S = \{p_1, p_2, \ldots, p_k\}$ un estado de D, es decir, un subconjunto de Q_E.

   Sea $\{r_1, r_2, \ldots, r_m\} = \bigcup_{i=1}^{k} \delta_E(p_i, a)$, todos los estados a los que se puede llegar partiende los p_i con el símbolo a. La función de transición puede ahora definirse como

$$\delta_D(S, a) = \bigcup_{j=1}^{m} \text{Clos}(r_j)$$

Con esta construcción podemos establecer la equivalencia entre los AFD y los AFN$-\epsilon$. Como antes, en el caso de los AFN, es necesario probar que dado un AFN$-\epsilon$ arbitrario, esposible construir un AFD equivalente y viceversa.

Proposición 5. *Un lenguaje L es aceptado por un AFD D si y sólo si es aceptado por algún AFN$-\epsilon$ E.*

Demostración. Sean dos autómatas, $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ un AFN$-\epsilon$ y un AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$. La demostración debe hacerse en dos partes, como en toda proposición si y sólo si. La primera parte, donde si un lenguaje es aceptado por un AFD entonces existe un AFN$-\epsilon$ es fácil.

$\Rightarrow$) A partir del AFD D es fácil construir un AFN$-\epsilon$ E que acepta el mismo lenguaje. El conjunto de estados es el mismo, el estado inicial sigue siendo igual así como los estados finales. La función de transición es casi la misma: cada transición $\delta_E(q, a) = \{\delta_D(q, a)\}$ pues el codominio de δ_E es 2^Q; además, $\delta_E(q, \epsilon) = \emptyset$ para todo estado q. Una cadena $w \in \Sigma^*$ hará el mismo recorrido por los estados de cada autómata, llegando a un estado de F_E si $w \in L(E)$, por lo que E acepta el mismo lenguaje que D.

$\Leftarrow$) Dado el AFN$-\epsilon$ E, sea D el AFD obtenido con la construcción anterior. Se probará por inducción sobre $|w|$ que para toda cadena $w \in \Sigma^*$ (no sólo para las aceptadas, por la misma razón que en la proposición 4)

$$\hat{\delta}_D(q_D, w) = \hat{\delta}_E(q_E, w).$$

Caso base: si $|w| = 0$, entonces $w = \epsilon$ y $\hat{\delta}_D(q_D, \epsilon) = q_D = \text{Clos}(q_E) = \hat{\delta}_E(q_E, \epsilon)$.

Caso inductivo: supongamos que la igualdad se cumple para una cadena x, es decir, $\hat{\delta}_D(q_D, x) = \hat{\delta}_E(q_E, x)$, y sea $\{p_1, p_2, \ldots, p_k\}$ ese conjunto de estados. Sean $a \in \Sigma$, $w = xa$ y

$$\{r_1, r_2, \ldots, r_m\} = \bigcup_{i=1}^{k} \delta_E(p_i, a). \tag{1.17}$$

Por la definición de $\hat{\delta}$ en un AFN$-\epsilon$,

$$\hat{\delta}_E(q_E, w) = \bigcup_{j=1}^{m} \text{Clos}(r_j). \tag{1.18}$$

Por otra parte, la construcción define $\delta_D(\{p_1, p_2, \ldots, p_k\}, a)$ precisamente con (1.17) y (1.18). Por lo tanto,

$$\hat{\delta}_D(q_D, w) = \delta_D(\hat{\delta}_D(q_D, x), a) = \delta_D(\{p_1, p_2, \ldots, p_k\}, a) = \hat{\delta}_E(q_E, w).$$

Finalmente, $w \in L(D)$ si y sólo si $\hat{\delta}_D(q_D, w) \cap F_E \neq \emptyset$, si y sólo si $w \in L(E)$, lo que concluye la demostración. □

Como ejemplos, tomaremos los autómatas de las figuras 1.9 y 1.10 para eliminar las transiciones épsilon.

δ_D	0	1
$\mapsto\{1\}$	–	$\{2,3,4,5\}$
$*\{2,3,4,5\}$	$\{2,5\}$	$\{4,5\}$
$*\{2,5\}$	$\{2\}$	$\{4,5\}$
$*\{4,5\}$	$\{5\}$	–
$\{2\}$	$\{2\}$	$\{4,5\}$
$*\{5\}$	–	–

Tabla 1.4: Tabla de transiciones del AFD equivalente al AFN-ϵ de la figura 1.9, obtenida con la construcción de subconjuntos. Sólo aparecen los subconjuntos alcanzables desde el estado inicial.

Ejemplo 23. *En el caso del autómata de la figura 1.9, sólo dos estados tienen transiciones épsilon, de tal manera que las cerraduras quedan:*

$$\begin{aligned}\text{Clos}(3) &= \{3,4,5\}\\ \text{Clos}(4) &= \{4,5\}\,.\end{aligned}$$

El estado inicial del AFD es $\text{Clos}(1) = \{1\}$*. Si estamos en el estado 1 y se recibe un símbolo 1, se puede pasar al estado 2 o al estado 3, y desde 3, mediante transiciones* ϵ*, a 4 y a 5. Por eso*

$$\delta_D(\{1\},1) = \text{Clos}(2)\cup\text{Clos}(3) = \{2,3,4,5\}\,.$$

De la misma manera, desde $\{2,3,4,5\}$ *con un 0 se llega a* $\text{Clos}(2)\cup\text{Clos}(5) = \{2,5\}$ *y con un 1 a* $\text{Clos}(4) = \{4,5\}$*. Al continuar sólo con los subconjuntos que van apareciendo se obtiene la tabla 1.4. Los estados finales son los que contienen a 4 o a 5.*

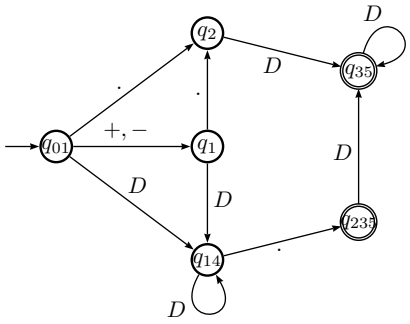
Ejemplo 24. *Considérese de nuevo el autómata del ejemplo 22 que acepta números reales. El diagrama de transiciones de este autómata se muestra en la figura 1.10. Es necesario observar que las cerraduras importantes son:*

$$\begin{aligned}&\text{Clos}(q_0) = \{q_0,q_1\} \quad \textit{(el nuevo estado inicial)}\\ &\text{Clos}(q_3) = \{q_3,q_5\}\end{aligned}$$

Por facilidad las vamos a referir como $q_{0,1}$ *y* $q_{3,5}$*. Obsérvese la figura 1.10. Desde* q_0 *sólo hay transiciones con un signo (hacia* q_1*) y con* ϵ*; con un punto o con un dígito no hay transición. Pero en el nuevo estado inicial* $q_{0,1}$ *se deben revisar las transiciones posibles para cada estado de la cerradura. Así, con un dígito ahora puede quedarse en* q_1 *(donde llegó por la transición* ϵ*) o avanzarse a* q_4*, por lo que se obtiene*

$$\delta(q_{0,1},D) = q_{1,4}$$

y $q_{1,4}$ *es un nuevo estado qué agregar al autómata que se está construyendo. De*



(a) AFD (parcial) sin transiciones ϵ que acepta constantes reales.

δ	.	S	D
$\mapsto q_{0,1}$	q_2	q_1	$q_{1,4}$
q_1	q_2	–	$q_{1,4}$
q_2	–	–	$q_{3,5}$
$q_{1,4}$	$q_{2,3,5}$	–	$q_{1,4}$
$*q_{3,5}$	–	–	$q_{3,5}$
$*q_{2,3,5}$	–	–	$q_{3,5}$

(b) Arriba de la línea punteada están las transiciones originales. Abajo los 3 nuevos estados alcanzados por las cerraduras.

Figura 1.11: AFD equivalente al de la figura 1.10. En la tabla de transiciones, debajo de la línea puntueada, aparecen los nuevos estados, que son las cerraduras correspondientes. Las cerraduras se han representado como subíndices.

la misma manera se obtienen

$$\delta(q_2, D) = q_{3,5}$$
$$\delta(q_{1,4}, .) = q_{2,3,5}$$
$$\delta(q_{1,4}, D) = q_{1,4}$$
$$\delta(q_{2,3,5}, D) = q_{3,5}.$$

Los nuevos estados finales son los que incluyen al estado final original. Al elmininar las transiciones ϵ del autómata aplicando la cerradura con respecto a ϵ se obtiene el diagrama de transiciones de la figura 1.11.

Las transiciones épsilon no agregan poder a los autómatas, pero son útiles para facilitar el diseño, tal como ocurre con el no determinismo.

1.3. Expresiones Regulares

Así como un lenguaje L se ha definido especificando la máquina (AFD) que acepta sólo cadenas $w \in L$, también es posible definirlo especificando una regla (o reglas) que indiquen cómo se pueden formar las cadenas de L.

Al ser un lenguaje un conjuto formado por cadenas, es válido realizar operaciones entre conjuntos o lenguajes, incluyendo operaciones especiales para conjuntos de cadenas. En particular, interesan 3 operaciones, que ya habían sido presentadas en la sección 1.1.3 (pág. 14). Sean L, L_1 y L_2 lenguajes cualesquiera.

1. La *unión* de 2 lenguajes L_1 y L_2 es al conjunto formado por las cadenas que están en alguno de los dos lenguajes, es decir, $L_1 \cup L_2 = \{w | w \in L_1 \text{ o } w \in L_2\}$.

2. La *concatenación* de lenguajes se denota por L_1L_2 y se define como el conjunto de cadenas que se forman concatenando cualquier cadena de L_1 con otra cualquier cadena de L_2, es decir, $L_1L_2 = \{w = w_1w_2 | w_1 \in L_1 \text{ y } w_2 \in L_2\}$.

3. La *cerradura* de un lenguaje, denotada por L^*, está formada por todas las concatenaciones de una cantidad finita (incluso cero) de cadenas de L, tomadas en cualquier orden y con repeticiones, es decir, $L^* = \bigcup_{i \geq 0} L^i$, donde $L^0 = \{\epsilon\}$, $L^1 = L$ y $L^i = LLL \cdots L$ (la concatenación de i veces L), para $i > 1$.

Ejemplo 25. *Si* $L_1 = \{a, ab, aab\}$ *y* $L_2 = \{bb, bba\}$*, entonces*

$$L_1 \cup L_2 = \{a, ab, aab, bb, bba\} \tag{1.19}$$

$$L_1L_2 = \{abb, abba, abbb, abbba, aabbb, aabbba\} \tag{1.20}$$

$$L_2L_1 = \{bba, bbab, bbaab, bbaa, bbaaab\} \tag{1.21}$$

$$L_1^* = \{\epsilon\} \cup \{w\alpha \mid w \in L_1 \text{ y } \alpha \in L_1^*\} \tag{1.22}$$

En el caso de (1.22), se trata de un conjunto infinito pues es una concatenación arbitraria, es decir, la cantidad que sea de cadenas tomadas de L_1*. Sin el término* $\{\epsilon\}$ *la definición quedaría sin caso base. Obsérvese también que* L_2L_1 *tiene sólo cinco cadenas, aunque hay seis parejas posibles, porque* $bba \cdot ab = bb \cdot aab = bbaab$*.*

Siguiendo con la idea de los lenguajes como conjuntos de cadenas, se acostumbra representar la unión de dos lenguajes L_1 y L_2 como $L_1 \cup L_2 = L_1 + L_2$.

Un *lenguaje regular*, definido en (1.11), es el que es aceptado por un AFD. Siempre es posible encontrar una manera de describir algebraicamente el mismo lenguaje, con lo que se conoce como *expresiones regulares*, o ER, utilizando ciertas reglas algebraicas bien definidas. La definición es de tipo inductivo, por lo que se requieren casos base (tres) y luego los casos inductivos (cuatro), donde se ve la manera de combinar ER. Sea Σ un alfabeto.

1. **(Caso base 0)** $\emptyset$ es una expresión regular.

$$L(\emptyset) = \emptyset \tag{1.23}$$

2. **(Caso base 1)** La cadena vacía, ϵ, es una expresión regular.

$$L(\epsilon) = \{\epsilon\} \tag{1.24}$$

3. **(Caso base 2)** Cada símbolo de Σ es una expresión regular.

$$L(a) = \{a\} \text{ con } a \in \Sigma \tag{1.25}$$

4. Si E es una expresión regular, entonces (E) también lo es.

$$L((E)) = L(E) \tag{1.26}$$

5. Si E es una expresión regular, su repetición arbitraria E^* también lo es.

$$L(E^*) = (L(E))^* \tag{1.27}$$

6. Si E_1 y E_2 son expresiones regulares, entonces su concatenación E_1E_2 también lo es.

$$L(E_1E_2) = L(E_1)L(E_2) \tag{1.28}$$

7. Si E_1 y E_2 son expresiones regulares, entonces $E_1 + E_2$ también lo es.

$$L(E_1 + E_2) = L(E_1) \cup L(E_2) \tag{1.29}$$

El operador $+$, que también se utiliza para la unión de lenguajes, tiene la precedencia más baja. Se utiliza para indicar que se toma una de las dos expresiones, es decir, la expresión regular $01 + 11$ produce dos cadenas, ya sea la 01 o bien la 11, pero no la cadena 0111, que es el resultado de la concatenación. El operador $+$ es conmutativo.

El operador $*$ tiene la máxima precedencia y afecta a la expresión regular inmediata que le antecede, por lo que ab^* produce las cadenas a, ab, abb, $abbb$ y no $abab$, $ababab$ que se generan con la expresión $(ab)^*$. La concatenación tiene una precedencia intermedia: $01^* + 1$ se lee como $(0(1^*)) + 1$.

El lenguaje producido por la expresión regular E se denota por $L(E)$. Por economía, muchas veces se escribe sólo la expresión regular para dar a entender el lenguaje que genera. Se dice que $L(E) \in ER$, es decir, el lenguaje generado por E pertenece al conjunto de los lenguajes generados por expresiones regulares.

Ejemplo 26. *¿Cuál es el lenguaje generado por la expresión $a(b+c)$? Apliquemos las reglas de la definición.*

$$\begin{aligned} L(a(b+c)) &= L(a)L(b+c) \\ &= L\{a\}\left(L(b) \cup L(c)\right) \\ &= \{a\}\left(\{b\} \cup \{c\}\right) \\ &= \{a\}\left(\{b, c\}\right) \\ &= \{a\}\{b, c\} \\ &= \{ab, ac\} \end{aligned}$$

Ejemplo 27. *¿Cuál es el lenguaje generado por la expresión $(10)^*00$? Hay que generar las cadenas de $(10)^*$ y luego concatenarles 00:*

$$\begin{aligned} L((10)^*00) &= L((10)^*)L(00) \\ &= \{\epsilon, 10, 1010, 101010, \ldots\}\,\{00\} \\ &= \{00, 1000, 101000, 10101000, \ldots\} \end{aligned}$$

Aunque en el ejemplo anterior se hizo de manera detallada, lo común es irse de manera directa a las cadenas. A fin de cuentas, $(10)^*00$ son las repeticiones arbitrarias de la cadena 10 concatenadas con 00, pero el detalle ayuda a entender.

Ejemplo 28. *¿Cuál es el lenguaje generado por la expresión* $(0+1)^*$*?*

$$\begin{aligned} L((0+1)^*) &= (\, L(0+1)\,)^* \\ &= \{0,1\}^* \\ &= \{\epsilon, 0, 1, 00, 01, 10, 11, \ldots\} \end{aligned}$$

Se trata de todas las cadenas que se pueden formar con ceros y unos.

Ejemplo 29. *La expresión regular* $\emptyset^*$ *genera el lenguaje que sólo contiene a la cadena vacía. Esto es así pues*

$$L(\emptyset^*) = \{\epsilon\} \cup \emptyset \cup \emptyset\emptyset \cup \emptyset\emptyset\emptyset \cup \cdots$$

y la única cadena que se genera es la vacía.

La expresión regular $\emptyset$ funciona como neutro ante la unión y cero ante concatenación pues si E es una expresión regular, entonces

$$E + \emptyset = E \tag{1.30}$$

$$E\emptyset = \emptyset \tag{1.31}$$

son operaciones que se consideran propiedades y deben recordarse. De manera análoga, ϵ es el neutro de la concatenación: $\epsilon E = E\epsilon = E$.

Ejemplo 30. *¿Cuál es el lenguaje generado por la expresión* $(0^*1^*)^*$*? Primero apliquemos las reglas de cerradura y concatenación.*

$$\begin{aligned} L((0^*1^*)^*) &= (\, L(0^*1^*)\,)^* \\ &= (\, L(0^*)L(1^*)\,)^* \\ &= (\, \{\epsilon, 0, 00, \ldots\}\{\epsilon, 1, 11, \ldots\}\,)^* \end{aligned}$$

Hay muchas maneras de enlistar la concatenación de $L(0^*)L(1^*)$*, pero hay que notar que dos cadenas que están ahí son* $\epsilon 1 = 1$ *y* $0\epsilon = 0$*, por lo que la cerradura de la concatenación resulta equivalente a la expresión del ejemplo 28,* $(0+1)^*$*.*

Ejemplo 31. *La expresión* $b(ac)^*d$ *genera cadenas que siempre comienzan con b, seguidas de una cantidad arbitraria de parejas ac y terminando con una letra d, como bd, bacd, bacacacd y así sucesivamente.*

Ejemplo 32. *Se pueden describir las cadenas generadas con la expresión regular* $(1+0)10^*$*, indicando que se producen cadenas que inicien con un cero o un uno, seguido siempre de un 1 y terminando con una cantidad arbitraria de ceros, como 11000, 0100, 01, 11 o 11000000.*

Ejemplo 33. *El lenguaje* $L(\ (01)^* + (123+0)^* + 1(23)^*\ )$ *equivale a la unión de los lenguajes*

$$(01)^* \cup (123+0)^* \cup 1(23)^*$$

que a su vez podría separarse aún más.

Ejemplo 34. *En el cuadro 1.1 se mostró un AFD que acepta las cadenas que representan potencias de 2 en binario. Una expresión regular equivalente es* 10^*.

Ejemplo 35. *Para el lenguaje* L_6*, del ejemplo 10, las cadenas de múltiplos de 5 desde 100 a 195 pueden generarse con la expresión regular* $1(0+1+2+3+\cdots+9)(0+5)$.

Ejemplo 36. *El lenguaje* $L(1^*+2^*)$ *se forma con todas las cadenas que sólo tienen unos o sólo tienen doses. La cadena vacía pertenece a este lenguaje.*

Ejemplo 37. *Otro ejemplo clásico es el lenguaje de las cadenas que alternan unos y ceros, como 010, 010101, 1, 1010 y todas en las que no hay repeticiones sucesivas. Una expresión regular que las genera parece ser algo como*

$$(01)^*$$

pero esta sólo incluye las que comienzan con 0, terminan con 1 y tienen longitud par. Para completar el lenguaje, se requiere agregar las simétricas $(10)^*$ *y las de longitud impar. La expresión regular resultante es*

$$(01)^* + (10)^* + 0(10)^* + 1(01)^*$$

que puede simplificarse usando la cadena vacía como:

$$(1+\epsilon)(01)^*(0+\epsilon)$$

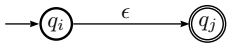
de donde se ve que las expresiones regulares no son únicas para cada lenguaje.

Muchas expresiones regulares son equivalentes, en es sentido que definen el mismo lenguaje. Por ejemplo, como $(0+1)^*$ son todas las posibles cadenas de ceros y unos, el lenguaje $(0+1)^*+1^*$ no aporta nada pues $1^* \subset (0+1)^*$, por lo que son iguales. Debe ser claro que $1+11+111+\cdots = 11^*$. Existe toda un algebra de expresiones regulares.

1.3.1. Conversión de expresiones regulares a autómatas

Construir a ojo un autómata que acepte el lenguaje generado por una ER puede requerir pericia, pero existe un algoritmo seguro para hacer la conversión cualquiera que sea la ER. La idea es construir el autómata por bloques, de manera inductiva, siguiendo la definición de las ER. Esta técnica se conoce como *construcción de Thompson*, por Ken Thompson, quien la publicó en 1968 para implementar la búsqueda con expresiones regulares en un editor de textos. El resultado es un AFN-ϵ; si se requiere un AFD, se le aplica después la construcción de la sección 1.2.9.

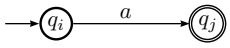
Cada autómata que se construye tiene un único estado inicial, al que no llega ninguna transición, y un único estado final, del que no sale ninguna transición. Esta propiedad se conserva en cada paso y es la que garantiza que las construcciones de la figura 1.13 sean correctas.



(a) Caso base de la expresión regular ϵ.



(b) Caso base de la expresión regular $\emptyset$.



(c) Caso base de la expresión regular a.

Figura 1.12: Casos base de la conversión de ER a AFN-ϵ.

Hay tres casos base, que consisten en las expresiones regulares ϵ, $\emptyset$ y a, con $a \in \Sigma$ un símbolo del alfabeto Σ. Los autómatas correspondientes a estos casos base se muestran en la figura 1.12.

A partir de estos simples autómatas como casos base, es posible construir autómatas más complejos, mediante construcciones inductivas. Sean E y F expresiones regulares. Para cada uno de los operaciones de suma, concatenación y cerradura se tiene un paso inductivo, esquematizados en la figura 1.13.

Adición de expresiones regulares. Para la suma de expresiones regulares $E + F$, en la que se aplica la suma de lenguajes, definida en la página 39 (punto 7), se construye un nuevo autómata a partir de los autómatas de E y F. Un nuevo primer estado es conectado con transiciones ϵ a los anteriores estados iniciales de los autómatas de E y F. De los estados finales de E y F se colocan transiciones ϵ al nuevo único estado final. Ver figura 1.13a.

$$L(E + R) = L(E) + L(F) \tag{1.32}$$

Concatenación de expresiones regulares. Se aplica la concatenación de lenguajes, definida en la página 39 (punto 6). Para la concatenación de las expresiones E y F, como EF, el estado final de E se conecta mediante una transición ϵ con el estado inicial de F y ambos dejan de ser estados especiales. Ver figura 1.13b.

$$L(ER) = L(E)L(F) \tag{1.33}$$

Cerradura de una expresión regular. Se aplica la cerradura de lenguajes, definida en la página 39 (punto 5). La cerradura E se construye haciendo un

nuevo estado inicial y otro nuevo estado final. El estado inicial se conecta con el anterior estado inicial de E y con el estado final. El estado final de E se conecta con su estado inicial original, para repetir, y con el nuevo estado final, para terminar. Ver figura 1.13c.

$$L(E^*) = (L(E))^* \tag{1.34}$$

Ejemplo 38. *Sea la expresión regular* $1 + 10$*, que genera un lenguaje con dos cadenas:* $L(1 + 10) = \{1,\ 10\}$*. La expresión emplea los símbolos 1 y 0, cuyos autómatas corresponden al caso base 1.12c. Además de ese caso base, están dos operaciones: primero la concatenación de 1 con 0 y luego la suma de 1 y 10, que se realiza como dicen los casos inductivos. Se ilustra en la figura 1.14.*

1.3.2. Conversión de AFD a expresiones regulares

Se ha mencionado que AFDs y Expresiones Regulares son dos formalismos equivalentes para definir Lenguajes Regulares. Esta proposición es suficientemente fuerte como para requerir una prueba mas detallada. Para mostrar la equivalencia, son necesarias dos cosas:

- Partir de un AFD arbitrario y construir una expresión regular que genere el mismo lenguaje que el AFD acepta.
- Dada una expresión regular E cualquiera, construir un AFD que solo acepte $L(E)$.

La gran ventaja de la Teoría de la Computación es que ofrece pruebas constructivas, que nos dan algoritmos para resolver problemas prácticos. La equivalencia entre AFD y expresiones regulares es también constructiva.

Sea $A = (Q, \Sigma, \delta, q_0, F)$ un AFD con n estados, que numeramos como $q_1, q_2, \ldots, q_n$, con q_1 el estado inicial. A partir de A se construirá una expresión regular R tal que $L(A) = L(R)$. Se entenderá por $R_{ij}^{(k)}$, la expresión regular que etiqueta el camino entre los estados q_i y q_j y que pasa por estados cuyo índice no es mayor que k. Se construirá inductivamente aumentando k de uno en uno.

Caso base: No hay estados intermediarios, $k = 0$. En ese caso, $R_{ij}^{(0)}$ solo son las expresiones de las rutas directas del nodo q_i al nodo q_j. Hay dos posibles casos:

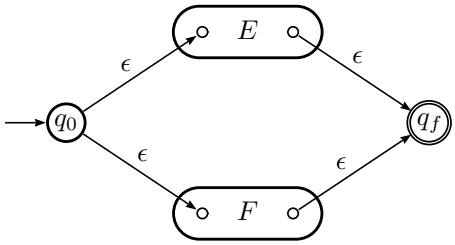
1. Un arco del estado i al estado j, con $i \neq j$.
2. Un camino de longitud 0, que consiste sólo del estado q_i.

Si $i \neq j$ entonces sólo el caso 1 es posible. Debe revisarse entonces el autómata A para buscar transiciones de q_i a q_j con algún símbolo a y hay tres posibilidades, que nos dan las primeras 3 reglas de construcción:

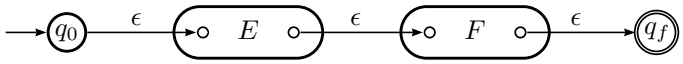
(a) Si no hay transición posible, entonces $R_{ij}^{(0)} = \emptyset$.

(b) Si hay tal transición, entonces $R_{ij}^{(0)} = a$.

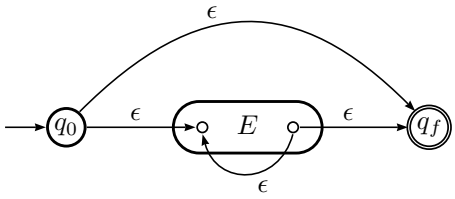
(c) Si hay varias transiciones, con símbolos $a_1, a_2, \ldots, a_r$ entonces $R_{ij}^{(0)} = a_1 + a_2 + \cdots + a_r$.



(a) La suma $E+F$. Las transiciones ϵ conectan a los autómatas de E y F.



(b) La concatenación EF.



(c) La cerradura E^*.

Figura 1.13: Pasos inductivos para la construcción modular de autómatas a partir de expresiones regulares.

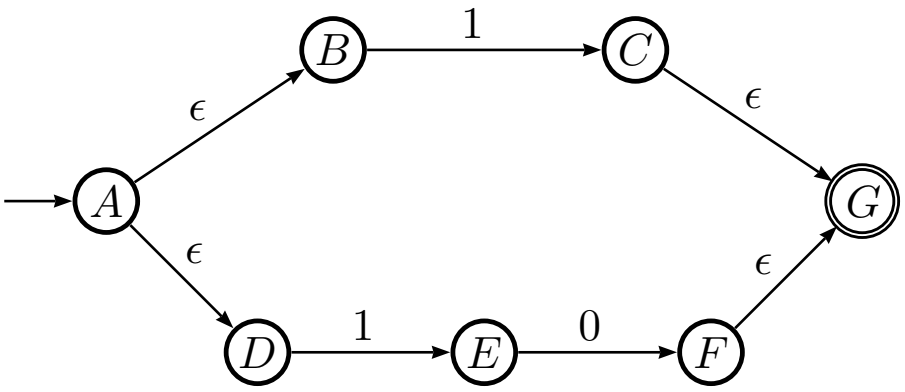


Figura 1.14: Conversión de $1 + 10$ a AFN.

En caso del camino de longitud cero y cuando $i = j$, entonces se trata de todas las transiciones de i sobre sí mismo. En este caso, el camino de longitud cero se representa por la expresión ϵ, la cadena vacía. Por eso, es necesario agregar ϵ a todas las expresiones que ya se tienen del caso base y construir ϵ, $a + \epsilon$ y $\epsilon + a_1 + a_2 + \cdots + a_r$.

Caso inductivo: Sea $R_{ij}^{(k)}$ la expresión que etiqueta las rutas del nodo i al nodo j y que no pasan por un nodo mayor que k. Hay dos casos que deben considerarse:

1. El camino no pasa por el estado q_k, así que la etiqueta está en el lenguaje de $R_{ij}^{(k-1)}$ por lo que ya se tiene.
2. El camino pasa por el estado k al menos una vez. Esta cadena se subdivide en 3 partes, primero del nodo i al k, luego los posibles caminos de k a k y por último de k a j.

Considerando los dos casos anteriores se obtiene la relación con la que se genera la expresión que se necesita (la cuarta regla de construcción):

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \; + \; R_{ik}^{(k-1)} \, (R_{kk}^{(k-1)})^* \, R_{kj}^{(k-1)} \tag{1.35}$$

para la etiqueta de los caminos que van del estado i al j sin pasar por estados con índice mayor a k.

Con las 4 reglas anteriores se pueden comenzar a construir expresiones partiendo de los índices más pequeños, de tal forma que cada una dependa sólo de expresiones con índices menores y que estén ya disponibles.

Finalmente, la expresión regular que se desea, equivalente al AFD A se forma con la unión de los lenguajes $R_{1j}^{(n)}$ para cada estado final $q_j \in F$.

El problema con este algoritmo es que genera cadenas (expresiones regulares) indeseablemente largas, por lo que es propenso a errores.

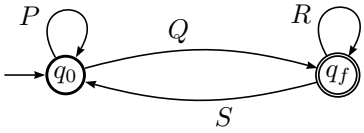


Figura 1.15: El caso al que se desea llegar en la reducción de estados. La expresión regular es $P^*QR^*(SP^*QR^*)^*$.

Eliminación de estados

Otra técnica para obtener la expresión regular a partir de un autómata es conocida como *eliminación de estados*.

En este caso, cada que se elimina un estado s es necesario revisar todas las rutas que pasan por s, de tal manera que si hay una ruta del estado q al p pasando por s, entonces se sustituye el estado s por una transición etiquetada por las expresiones regulares equivalentes a todas las rutas involucradas.

Al final de la eliminación de estados se espera tener el estado inicial y un único estado final, por lo que el caso más complicado será el de la figura 1.15. Se entiende que las etiquetas de las transiciones son expresiones regulares construidas a partir de la eliminación de estados. La expresión final es

$$P^*QR^*(SP^*QR^*)^*$$

Un autómata con tres estados se presenta en la figura 1.16, donde el estado s se elimina.

Obsérvese que es posible que los estados p y q (en la misma figura 1.16) sean el mismo estado. Hay dos cosas que considerar. Primeramente, no hay un orden determinista en cuanto a qué estados deben eliminarse primero. Por otra parte, puede ser complicado visualizar todas las rutas que pasan por un estado que se desea eliminiar. Los detalles de la conversión se muestran en los siguiente ejemplos, que ilustran los casos posibles. Refiérase el lector a la figura 1.16 cada que necesite.

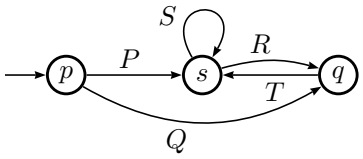
Ejemplo 39. *La figura 1.17 muestra un autómata y la secuencia de sus transformaciones. En la figura 1.17b se ha eliminado el estado q_2, lo que requiere poner la ruta $q_3 - q_2 - q_1$ (símbolos 01) en una transición de q_3 a q_1.*

El segundo estado que se elimina es q_1, el único posible pues el estado inicial y el final son intocables. Al eliminar este estado, las rutas $q_0 - q_1 - q_3$ (10^*1) y $q_3 - q_1 - q_3$ (010^*1) deben agregarse a las transiciones $q_0 - q_3$ y $q_3 - q_3$ respectivamente.

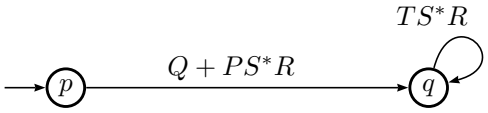
Del autómata final se lee la expresión regular

$$(0 + 10^*1)\ (010^*1)^*\ (1(0 + 10^*1)(010^*1)^*)^*$$

de las 3 transiciones resultantes. Corresponde a la forma de la figura 1.15 con $P = \emptyset$, $Q = 0 + 10^*1$, $R = 010^*1$ y $S = 1$; recuérdese que $\emptyset^* = \epsilon$.



(a) Autómata original. El estado s va a desaparecer.



(b) Autómata ya transformado, luego de eliminar el estado s. La etiqueta TS^*R corresponde a ir de q a s, recorrer el ciclo en s y regresar a q.

Figura 1.16: Eliminación del estado s. Las etiquetas P, Q, R, S y T representan expresiones regulares. Si q es el único estado final, la expresión final es $(Q + PS^*R)(TS^*R)^*$.

Ejemplo 40. *En la figura 1.18 se tiene un autómata como el del ejemplo anterior, pero con otra configuración de aristas, que hacen más interesante la eliminación de estados. La figura 1.18a es el autómata original y 1.18b es el autómata cuando se elimina el estado q_1.*

Por q_1 pasan varias rutas:

1. $q_0 - q_1 - q_2$*:* 00 *que se agrega a* $q_0 - q_2$*,*
2. $q_0 - q_1 - q_3$*:* 01 *que forma la transición* $q_0 - q_3$ *y*
3. $q_2 - q_1 - q_2$*:* 10 *que se agrega a* $q_2 - q_2$*.*

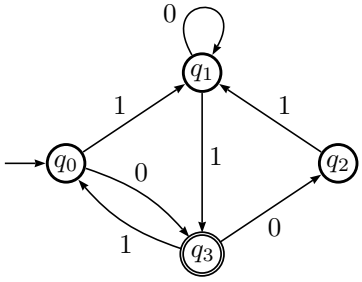
Eliminar q_2 requiere menos operaciones, agregando un ciclo $q_3 - q_3$. La expresión regular final es

$$(01 + (1 + 00)(0 + 10)^*(1 + 11))(01 + 00(0 + 10)^*(1 + 11))^*$$

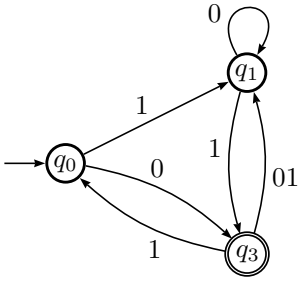
que se obtiene directamente de la figura 1.18c.

Ejemplo 41. *La figura 1.19 muestra un autómata y la secuencia paso a paso de su transformación a expresión regular. En la figura 1.19a está el autómata original. Como se busca terminar con un autómata con transición a un único estado final, se agrega tal estado en 1.19b y q_3 y q_4 dejan de ser estados finales.*

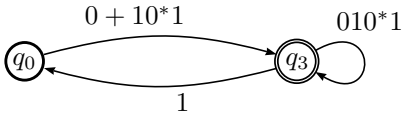
Siendo el orden de eliminación arbitrario, en la figura 1.19c se ha eliminado el estado q_3. Las rutas afectadas son del estado q_1 a q_5 y de q_1 a q_2, que están en las nuevas transiciones dibujadas. Al eliminar q_4 el proceso se repite, lo que se muestra en la figura 1.19d.



(a) Autómata original.

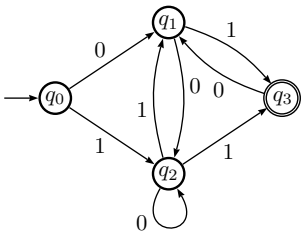


(b) Se elimina el estado q_2. Aparece ruta de q_3 a q_1.

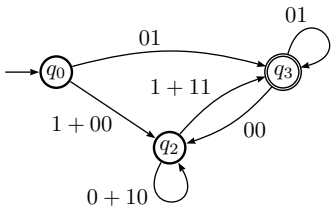


(c) Se elimina q_1.

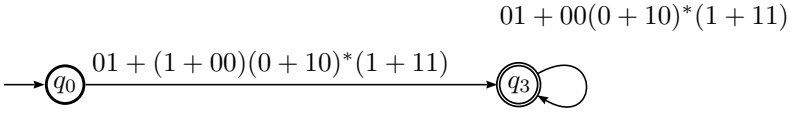
Figura 1.17: Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.



(a) Autómata original.



(b) Se elimina el estado q_1. Aparece ruta de q_0 a q_3 y todas las otras son modificadas.



(c) Se elimina q_2.

Figura 1.18: Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.

En la figura 1.19e, la eliminación de q_1 modifica las rutas $q_0-q_1-q_2$ (10^*10), $q_0-q_1-q_5$ (10^*1), $q_2-q_1-q_2$ $(1+010^*10)$ y $q_2-q_1-q_5$ $(0+010^*1)$, por lo que estas nuevas transiciones deben suplir tales caminos.

Al final, la eliminación de q_2 ya no es problema. En la figura 1.19f se ha sustituido R por la subexpresión 10^*1 para ahorrar espacio.

Cuando estos ejemplos se realizan con JFLAP hay dos cosas que tomar en cuenta. Primero, JFLAP guiara al usuario paso a paso en la transformación, impidiendo operaciones que no se necesitan. No hay problema si se hace algo y no se puede deshacer, siempre es posible cerrar la ventana e iniciar desde cero. Por otra parte, para aplicar el algoritmo de reducción, JFLAP requiere que haya transiciones entre cada pareja de estados, por lo que se agregarán transiciones etiquetadas con $\emptyset$ donde falte. Esto crea algo de confusión visual, pero siempre es posible mover los estados para leer adecuadamente.

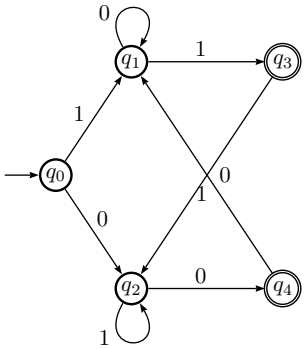
1.3.3. Simplificación de expresiones regulares

El algoritmo para formar expresiones regulares a partir de un AFD genera expresiones muy largas y es conveniente simplificar durante el proceso, ya sea que se haga a mano o se programe por computadora. A continuación se muestran algunas reglas algebraicas de las expresiones regulares que son útiles para este propósito.

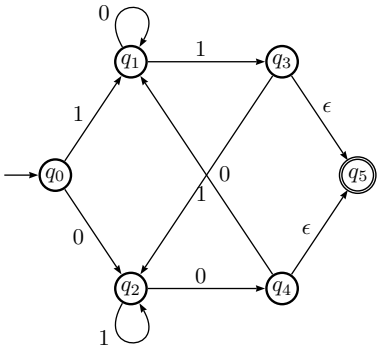
Sean R, E_1, E_2 y E expresiones regulares sobre un alfabeto Σ y $a \in \Sigma$. Se cumplen las siguientes operaciones.

$$R + R = R \tag{1.36}$$
$$E_1 + E_2 = E_2 + E_1 \tag{1.37}$$
$$E + (E_1 + E_2) = (E + E_1) + E_2 \tag{1.38}$$
$$E(E_1E_2) = (EE_1)E_2 \tag{1.39}$$
$$R(E_1 + E_2) = RE_1 + RE_2 \tag{1.40}$$
$$a + R^*a = R^*a \tag{1.41}$$
$$(\epsilon + R)^* = R^* \tag{1.42}$$
$$(\epsilon + R)R^* = R^* \tag{1.43}$$
$$\epsilon + R^* = R^* \tag{1.44}$$
$$\emptyset^* = \epsilon \tag{1.45}$$
$$(R + E)^* = (R^*E^*)^* \tag{1.46}$$
$$R + RE = R(\epsilon + E) \tag{1.47}$$
$$\emptyset + R = R + \emptyset = R \tag{1.48}$$
$$\emptyset R = R\emptyset = \emptyset \tag{1.49}$$

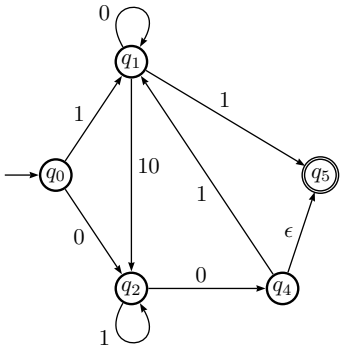
Algunas de estas reglas son más evidentes que otras, pero todas (y las que se deriven) pueden ser útiles. La regla (1.49) es particularmente útil para las enormes expresiones generadas por la ecuación (1.35).



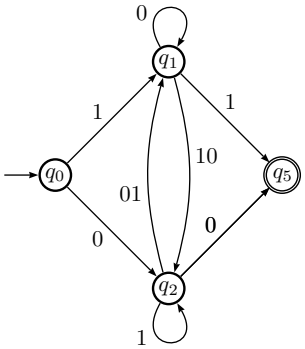
(a) Autómata original.



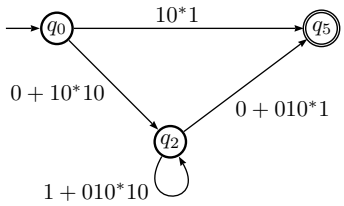
(b) Se agrega un estado final único.



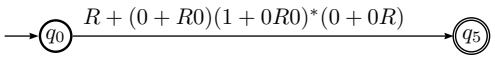
(c) Se elimina q_3.



(d) Se elimina q_4.



(e) Se elimina q_1.



(f) Se elimina q_2. $R = 10^*1$ y se ha sustituido por cuestión de espacio.

Figura 1.19: Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.

1.4. Propiedades de los lenguajes regulares

Aunque los ejemplos y operaciones ya presentados en secciones previas son en sí mismos propiedades de los lenguajes regulares, en este apartado se incluyen algunos casos notables, para introducir una clasificación básica de las propiedades de dichos lenguajes.

1.4.1. ¿Cómo saber que un lenguaje es regular?

Si un lenguaje es regular debiera ser simple probarlo pues basta mostrar el autómata que lo acepta. Pero no poder construir un autómata o una expresión regular no significa que no sea regular: la ausencia de prueba no es prueba de ausencia. Por eso se requiere un criterio más fuerte. Tal criterio es el lema del bombeo. Esto se aplica al caso de lenguajes infinitos, pues todo lenguaje finito es regular.

Proposición 6 (Lema del bombeo para lenguajes regulares)**.** *Supongamos un lenguaje L que sea regular e infinito. Entonces existe una constante n, que depende del lenguaje L, tal que para toda cadena $w \in L$, con $|w| \geq n$, se puede descomponer w en tres cadenas $w = xyz$ que cumplen con*

1. *$y \neq \epsilon$,*
2. *$|xy| \leq n$ y*
3. *para toda $k \geq 0$, la cadena $xy^k z$ también pertence a L.*

El lema dice que en un lenguaje regular infinito, cada cadena suficientemente larga (de longitud al menos n) tiene, cerca de su inicio, una subcadena no vacía que puede repetirse cuantas veces se quiera, o eliminarse, y la cadena resultante sigue perteneciendo al lenguaje. La razón es el principio del palomar: si A es un AFD con n estados que acepta L, al procesar los primeros n símbolos de w visita $n + 1$ estados, así que alguno se repite. El tramo de la cadena entre las dos visitas es y, y corresponde a un ciclo del autómata que puede recorrerse cualquier número de veces.

Ejemplo 42. *Sea el lenguaje $L = L(10^*)$, formado por cadenas de un uno y ceros (las potencias de 2 en binario). En la demostración del lema, n es el número de estados de un AFD que acepta L; el AFD de la tabla 1.1 tiene 2 estados, así que puede tomarse $n = 2$. Una cadena que pertenece a ese lenguaje es $w = 100$ y cumple con $|w| \geq 2$. Se puede separar dejando un símbolo para cada subcadena $w = xyz$: $x = 1$ y $y = z = 0$. Así, para cualquier k, se cumple que $xy^k z \in L$ pues*

$$
\begin{aligned}
xy^0 z &= 1\epsilon 0 = 10 \\
xy^1 z &= 100 \\
xy^2 z &= 1000 \\
&\vdots
\end{aligned}
$$

y es clara la pertenencia. Lo mismo ocurre con cualquier cadena $w = 10^m$ *con* $m \geq 1$*: basta tomar* $x = 1$*,* $y = 0$ *y* $z = 0^{m-1}$*.*

Para un ejemplo de un lenguaje no regular, vaya a la página 57.

El lema da una condición *necesaria* para que un lenguaje sea regular, pero no suficiente: todo lenguaje regular la cumple, y también hay lenguajes no regulares que la cumplen. Por eso el lema sólo sirve en una dirección: si un lenguaje *no* cumple el lema, entonces no es regular; si lo cumple, no se puede concluir nada. Un criterio que sí caracteriza exactamente a los lenguajes regulares es el teorema de Myhill-Nerode. Primero, es necesario definir una relación de equivalencia entre cadenas.

Definición 16. *Sea* L *un lenguaje sobre* Σ*. Dos cadenas* $x, y \in \Sigma^*$ *están relacionadas, denotado por* $x \equiv_L y$*, si para toda cadena* $z \in \Sigma^*$ *ocurre que* $xz \in L$ *si y sólo si* $yz \in L$*. Si están relacionadas se dice que son indistinguibles.*

Debe ser claro que las cadenas se *distinguen* si existe al menos una cadena z tal que xz y yz no estén al mismo tiempo en L.

Proposición 7 (Teorema de Myhill-Nerode)**.** *Un lenguaje* L *es regular si y sólo si la relación* $\equiv_L$ *determina una cantidad finita de clases de equivalencia. Más aún, la cantidad de clases es igual al número de estados del AFD mínimo, con función de transición completa, que acepta* L*.*

Para aplicarlo es necesario construir todas las clases de equivalencia para mostrar que es una cantidad finita. Otra aplicación útil y común de este resultado es la minimización de AFD.

1.4.2. Propiedades de cerradura

Estas propiedades son de la forma siguiente: dados dos lenguajes regulares, determinar si cierta operación entre ellos resulta en un lenguaje L que también es regular. Aunque en algunos casos esto sea obvio o intuitivo, es necesario comprobar formalmente la veracidad de cada cerradura.

Como se trata de lenguajes regulares, para cada uno hay un AFD que lo acepta, así que basta ver la manera de construir un AFD para el lenguaje L resultante. Esto se hace partiendo de los AFD originales.

Algunos ejemplos a continuación, planteados como proposiciones. Los detalles o las pruebas enteras son excelentes ejercicios para estudiantes diligentes.

Proposición 8. *La unión de dos lenguajes regulares es regular.*

Demostración. Sean L_1 y L_2 dos lenguajes regulares. Se trata de demostrar que el lenguaje $L = L_1 \cup L_2$ es también regular. Siendo regulares, existe un AFD que los acepta, digamos A_1 y A_2 respectivamente. El autómata que acepta a L se construye creando un nuevo estado inicial y conectándolo mediante transiciones épsilon a los estados iniciales de A_1 y A_2, que pierden tal condición. También se agrega al nuevo autómata su único estado final, al que se conectan

con transiciones épsilon los estados finales de A_1 y A_2, que también dejan de ser estados finales. El autómata obtenido es un AFN-ϵ, que por la proposición 5 tiene un AFD equivalente. También puede construirse un AFD directamente con la construcción producto de la proposición siguiente, con ambas funciones de transición completas y tomando como finales las parejas en las que al menos uno de los dos estados es final.

Otra manera equivalente es partir de las expresiones regulares que generan a cada lenguaje. Sean E_1 y E_2 tales expresiones. La expresión $E_1 + E_2$ genera a L, es decir,

$$L_1 \cup L_2 = L(E_1) \cup L(E_2) = L(E_1 + E_2)$$

□

Proposición 9. *La itersección de dos lenguajes regulares es regular.*

Demostración. Esta demostración requiere un poco más de trabajo. Sean L_1 y L_2 dos lenguajes regulares. Se trata de demostrar que el lenguaje $L = L_1 \cap L_2$ es también regular.

Siendo regulares, existen AFDs que los aceptan, digamos $L_1 = L(A_1)$ y $L_2 = L(A_2)$. Sean $A_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ y $A_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. En caso de que los alfabetos de fueran distintos, tomaríamos Σ como la unión de los alfabetos. Construiremos un autómata cuyos estados sean parejas de estados de $A_!$ y A_2 respectivamente, que simule el comportamiento de ambos autómatas al mismo tiempo.

El AFD A que acepta L se construye como

$$A = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$$

y $\delta((p, q), a) = (\, \delta_1(p, a),\, \delta_2(q, a)\,)$. Debe ser claro que si una de las transiciones $\delta_1(p, a)$ o $\delta_2(q, a)$ no existe, la transición entera $\delta((p, q), a)$ no existe.

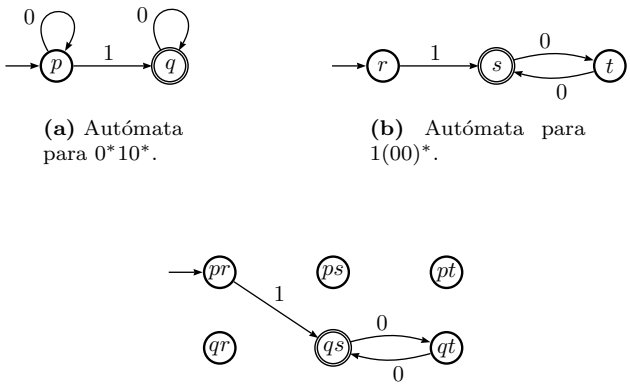
Primeramente, debe extenderse δ para que procese cadenas, pues está definida sólo para símbolos de Σ. Puede probarse por inducción que $\hat{\delta}((q_1, q_2), w) = (\, \hat{\delta}_1(q_1, w),\, \hat{\delta}_2(q_2, w)\,)$.

El AFD A aceptará la cadena w si y sólo si $\hat{\delta}((q_1, q_2), w)$ es una pareja de estados de aceptación, es decir, w se acepta siempre y cuando A_1 y A_2 la acepten, lo que significa que está en la intersección del lenguaje. □

Para ilustrar esta construcción, va un ejemplo.

Ejemplo 43. *La figura 1.20 muestran dos autómatas y su intersección. Las etiquetas que se han dado a los estados describen el resultado del producto cartesiano, solo cambiando la notación (p, r) por pr.*

La intersección tiene una transición del estado pr a qs porque hay transiciones de p a q en el primer AFD (subfigura 1.20a) y de r a s en el segundo (subfigura 1.20b). No así con la transición del estado p sobre sí mismo. No aparece en la intersección porque no hay una equivalente de r a r.



(a) Autómata para 0^*10^*.

(b) Autómata para $1(00)^*$.

(c) Intersección de los autómatas.

Figura 1.20: Dos autómatas y su intersección. Se presentan todos los estados generados, pero solo las transiciones coincidentes.

El complemento de un lenguaje regular también es regular y la construcción es bastante simple. Es de gran utilidad para determinar si dos expresiones regulares generan el mismo lenguaje, un problema algo más complejo de lo que parece.

Proposición 10. *El complemento de un lenguaje regular es regular.*

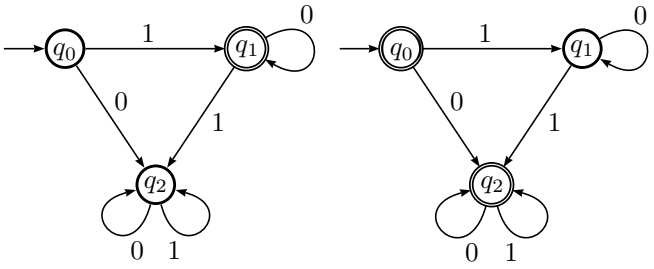
Demostración. Sea L un lenguaje regular, aceptado por el AFD $A = (Q, \Sigma, \delta, q_0, F)$. Si δ no es completa, primero se completa como en la sección 1.2.2: se agrega un estado no final q_s, llamado *sumidero*, y se dirigen hacia él todas las transiciones que faltan, incluidas las del propio q_s con cada símbolo. Esto no cambia el lenguaje aceptado.

Con δ completa, el AFD $B = (Q, \Sigma, \delta, q_0, Q - F)$ acepta exactamente L^C: para cada cadena w, el estado $\hat{\delta}(q_0, w)$ existe y es el mismo en A y en B, y es final en B si y sólo si no es final en A.

Es indispensable completar δ antes de intercambiar los estados finales. En un AFD parcial, una cadena que se queda sin transición es rechazada por A, y también sería rechazada por B, aunque pertenece a L^C. □

El siguiente ejemplo ilustra la construcción del complemento de un lenguaje regular, dado su AFD. Se trata del mismo ejemplo del lenguaje 10^* que aparece en la página 17 como AFD incompleto y que se completa en la página 18.

Ejemplo 44. *La figura 1.21 muestra el AFD completo para $L = L(10^*)$ (subfigura 1.21a), que es el de la tabla 1.3, y su complemento (subfigura 1.21b), que acepta L^C. Observe que el sumidero q_2 se convierte en estado final: el complemento acepta, por ejemplo, todas las cadenas que empiezan con 0.*



(a) Autómata completo para 10^*.

(b) Autómata complemento para 10^*.

Figura 1.21: Un autómata y su complemento.

La construcción anterior no es válida en caso de que se trate de un AFN. En un AFN, una cadena se acepta si *algún* camino termina en un estado final; al intercambiar estados finales y no finales, una cadena con un camino que acepta y otro que no sería aceptada por ambos autómatas. Para complementar un AFN, primero hay que convertirlo en AFD.

Otras operaciones que también son cerradas para los AFD son:

1. Diferencia de lenguajes $L_1 - L_2$
2. Reflexión, invirtiendo cada cadena
3. Cerradura de Kleen L^*
4. Concatenación
5. Homomorfismo, que es la sustitución de cada símbolo de Σ por una cadena de otro alfabeto.

Se sugiere al lector analizar y discutir en equipo estas propiedades. Su demostración requiere recursos similares a las de las que ya se mostraron.

1.4.3. Propiedades de decisión

Las preguntas típicas de decisión son problemas de la forma ¿la cadena w está en el lenguaje L? Esta es precisamente la definición de problema. A esta se suman dos preguntas que también son de decisión, primero si cierto que el lenguaje es vacío o si es infinito. De otra forma, una pregunta (sobre una propiedad) es de decisión si se responde con un SI o un NO.

Estas últimas dos preguntas de decisión son válidas cuando se describe un lenguaje o se trata de el resultado de una o más operaciones entre lenguajes y no está claro si el conjunto resultante es vacío o infinito.

En términos generales, para comprobar si un lenguaje regular es no vacío basta con encontrar una ruta del estado inicial a un estado final. Si lo que se tiene es una expresión regular, el único riesgo es cuando el conjunto vacío forma parte de tal expresión, por sus propiedades como lenguaje, por lo que se requiere simplificar la expresión.

Para saber si un lenguaje regular es infinito, se toma un AFD que lo acepta

y se busca un ciclo que sea alcanzable desde el estado inicial y desde el cual se pueda llegar a un estado final. No basta cualquier ciclo: el estado sumidero de un AFD completo tiene ciclos aunque el lenguaje sea finito. De manera equivalente, si el AFD tiene n estados, el lenguaje es infinito si y sólo si acepta alguna cadena w con $n \leq |w| < 2n$.

También es decidible si dos lenguajes regulares son iguales: $L_1 = L_2$ si y sólo si la diferencia simétrica $(L_1 \cap L_2^C) \cup (L_1^C \cap L_2)$ es vacía, y esa diferencia es un lenguaje regular por las propiedades de cerradura. Otra forma es minimizar ambos AFD y comprobar que son iguales salvo el nombre de los estados.

1.5. Limitaciones de los lenguajes regulares

Las expresiones regulares y su equivalente en máquina abstracta, los AFD, son el modelo de cómputo más simple y por tanto, como es de esperarse, no son capaces de reconocer o aceptar todos los posibles lenguajes. Hay una infinidad de lenguajes regulares, pero es una infinidad numerable, porque cada uno se describe con una expresión regular, que es una cadena finita. En cambio, el conjunto de todos los lenguajes sobre Σ, que es 2^{Σ^*}, no es numerable. Así que la mayoría de los lenguajes no son regulares.

Ejemplo 45. *El lenguaje $L = \{01, 0011, 000111, 00001111, \ldots\}$ no es regular. Esto significa que no hay AFD que lo acepte o expresión regular que produzca tales cadenas.*

Demostración. Es claro que no encontrar un autómata que acepte el lenguaje 0^n1^n no significa que no exista: la ausencia de prueba no es prueba de ausencia. Usaremos el lema del bombeo.

Supongamos, para llegar a una contradicción, que L es regular, y sea n la constante del lema. Esa constante la da el lema, no la elegimos nosotros, así que el argumento debe funcionar para cualquier valor de n. Tomemos $w = 0^n1^n \in L$, que cumple $|w| = 2n \geq n$. El lema garantiza que $w = xyz$ con $y \neq \epsilon$, $|xy| \leq n$ y $xy^kz \in L$ para toda $k \geq 0$. Como $|xy| \leq n$ y los primeros n símbolos de w son ceros, x y y constan sólo de ceros: $y = 0^j$ con $j \geq 1$. Entonces, con $k = 0$, la cadena $xz = 0^{n-j}1^n$ tiene menos ceros que unos, así que $xz \notin L$. Esto contradice el lema, de modo que L no es regular.

Obsérvese que no basta revisar una descomposición particular de una cadena particular. El lema afirma que *existe* una descomposición que funciona, así que para refutarlo hay que mostrar que *ninguna* de las descomposiciones permitidas funciona. □

El ejemplo anterior es el clásico que se emplea para limitar las posibilidades de los AFD. La razón por la que un AFD no puede aceptar tal lenguaje es que no tiene forma de saber cuántos ni cuáles símbolos ha procesado en la cadena de entrada. Es decir, su memoria, que es el estado actual, es finita y no alcanza para contar una cantidad de ceros que puede ser arbitrariamente grande.

Así como ese lenguaje no es regular, muchos otros no lo son. Otros lenguajes no regulares son por ejemplo 0^n10^n, los palindromas y una infinidad más. Más ejemplos se presentan en el resto de las notas.

Esta limitante es la que lleva al estudio del siguiente modelo abstracto de computadora, que es el que acepta los Lenguajes Libres de Contexto, en la siguiente sección.

Capítulo 2

Lenguajes Libres de Contexto

Introducción

En el caso de los lenguajes regulares, estos fueron definidos en términos de una máquina abstracta (el autómata finito y sus variantes) y luego se mostró la equivalencia con un método algebraico para definirlos (las expresiones regulares). Claro, la presentación pude ser distinta: primero el modelo algebraico y luego el de la máquina.

Para la siguiente categoría de lenguajes, se invertirá el camino, presentando primero el modelo algebraico, que en este caso es en forma de gramática, y luego el modelo computacional (el autómata de pila). Los lenguajes regulares también se pueden definir en términos de una gramática muy simple, pero las expresiones regulares son el mecanismo preferido por su simplicidad y por estar más cercanas a lo que en la práctica se hace uso.

Como se mostrará, los lenguajes regulares son un subconjunto propio de los lenguajes libres de contexto. Ya se mostró un lenguaje (0^n1^n) que no es regular. Aquí se mostrará que es libre de contexto, junto con otros ejemplos.

Los lenguajes libres de contexto permiten definir y modelar la sintaxis de los lenguajes de programación. Son un elemento esencial en el diseño de compiladores y muchas componentes de software interactivo (aquel donde el usuario hace algo más que sólo dar CLICK).

2.1. Gramáticas Libres de Contexto

Una gramática es un conjunto de reglas con las que se pueden producir cadenas a partir de un alfabeto base. Las reglas se escriben haciendo uso de variables, el símbolo de producción $\rightarrow$ y símbolos terminales. Por ejemplo, si se dice que X es una variable y que puede sustituirse con un par de letras a,

entonces la regla de producción asociada es

$$X \to aa.$$

En este caso, la letra a es un símbolo del alfabeto base Σ. La X es la parte izquierda de la producción (una variable) y aa la parte derecha de la producción. La parte derecha puede contener tanto variables como símbolos terminales.

De manera formal, una *Gramática Libre de Contexto* (GLL) G consiste en 4 entidades $G = (V, T, P, S)$, donde

1. V un conjunto finito de variables,
2. T un conjunto finito de símbolos terminales (el alfabeto Σ), disjunto de V,
3. P las reglas de producción, de la forma
$$X \to \alpha \tag{2.1}$$
con $X \in V$ y $\alpha \in (V \cup T)^*$ y
4. $S \in V$ el símbolo inicial.

La aplicación de reglas de producción para generar una cadena se llama *derivación* y se distingue de la producción al utilizar una flecha de dos líneas $\Rightarrow$ a la que se puede agregar como subíndice la gramática empleada. Si no hay confusiones entre gramáticas, como en este caso, el subíndice en $\Rightarrow_G$ se puede suprimir de la derivación y escribir sólo $\Rightarrow$. En este caso, haciendo analogía con los lenguajes regulares, el conjunto de símbolos terminales T equivale al alfabeto Σ.

Veamos esto con ejemplos.

Ejemplo 46. *Sea $G = (\{S\}, \{0, 1\}, P, S)$ y las producciones P las reglas:*

$$S \to 0S1 |\ 01 \tag{2.2}$$

Partiendo del símbolo inicial, es posible generar cadenas como

$$S \Rightarrow 01 \tag{2.3}$$
$$S \Rightarrow 0S1 \Rightarrow 0011 \tag{2.4}$$
$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111 \tag{2.5}$$

y de esta forma se ve que el lenguaje generado es 0^n1^n, el Ejemplo 45 de un lenguaje que no es regular.

Es un lenguaje libre de contexto pues se obtiene a partir de una gramática libre de contexto.

Se emplea $\Rightarrow^*$ para resumir una derivación, omitiendo los pasos intermedios; el $*$ significa que se aplican cero o más pasos de derivación. En el ejemplo anterior, puede escribirse $S \Rightarrow^* 000111$

Ejemplo 47. *Un* palindroma *es una cadena simétrica con respecto a su centro, es decir, se lee lo mismo al derecho que al revés. En español algunas comunes son «Anita lava la tina» o «Dábale arroz a la zorra el abad». Claro, son palindromas una vez que no nos fijamos en espacios ni acentos. Para facilitar las cosas pensemos en los palindromas con los símbolos 0 y 1. Sea* $G = (\{S\}, \{0,1\}, P, S)$ *y las producciones* P *las reglas:*

$$S \to 0S0 \tag{2.6}$$
$$S \to 1S1 \tag{2.7}$$
$$S \to 0 \tag{2.8}$$
$$S \to 1 \tag{2.9}$$

Iniciando con el símbolo S*, se pueden generar cadenas aplicando producciones en un orden arbitrario. Por la forma de las producciones, es claro que las cadenas que se producen son simétricas, por lo que esta gramática produce palindromas. Esto muestra sólo una dirección: que todo lo que genera la gramática es un palíndromo. La otra dirección, que todo palíndromo de longitud impar sobre* $\{0,1\}$ *se genera, se prueba por inducción sobre la longitud: un palíndromo de longitud mayor que 1 tiene la forma* $0u0$ *o* $1u1$*, con* u *un palíndromo más corto. Además, todas las cadenas son de longitud impar, pues para producir palindromas de longitud par se requiere la producción adicional* $S \to \epsilon$ *que genera la cadena vacía.*

De esta manera, podemos derivar el palindroma 11011 *aplicando 3 producciones de la gramática, lo que se escribe:*

$$S \Rightarrow_G 1S1 \Rightarrow_G 11S11 \Rightarrow_G 11011$$

usando una flecha doble $\Rightarrow$ *para distinguir la derivación de las producciones. La derivación puede resumirse escribiendo* $S \Rightarrow^* 11011$ *.*

Las cadenas intermedias entre el símbolo inicial y la cadena derivada, que combinan símbolos terminales con variables, son llamadas *formas sentenciales*. Se puede decir que una forma sentencial es un elemento del conjunto $F = (V \cup T)^*$. Para una gramática fija, sus formas sentenciales posibles son un subconjunto de F.

Cuando una misma variable posee varias producciones distintas, es posible escribirlas como una sola, pero separando los lados derechos con una barra vertical. En el ejemplo anterior, las producciones pueden reescribirse como

$$S \to 0S0 \mid 1S1 \mid 0 \mid 1 \tag{2.10}$$

sin que la gramática cambie, sólo se trata de ahorrar líneas de texto.

El *lenguaje de una gramática* es el conjunto de cadenas que pueden derivarse a partir del símbolo inicial, esto es, dada la gramática $G = (V, T, P, S)$, el lenguaje de G es

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}. \tag{2.11}$$

2.1.1. Derivaciones izquierdas y derivaciones derechas

Dada una producción de la forma $X \to AB$, la definición de gramática no indica cuál de las variables sustituir primero, es decir, en la derivación, puede sustituirse A y luego B o al revés. Eso ocasiona que con para una misma cadena del lenguaje se pueden tener varias derivaciones distintas. Para evitar este problema es posible restringir la manera como se realizan las derivaciones.

Lo más sencillo es sustituir siempre la variable que está más a la izquierda, en lo que se conoce como *derivación izquierda*. De manera simétrica, si siempre se sustituye la variable más a la derecha, se obtiene una *derivación derecha*.

Ejemplo 48. *Sea G una gramática con producciones*

$$S \to AB \mid Bb \tag{2.12}$$
$$A \to aA \mid aa \tag{2.13}$$
$$B \to aBb \mid b \tag{2.14}$$

y debe entenderse que las mayúsculas son las variables y las minúsculas los terminales. Obsérvense las derivaciones siguientes:

1. $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaaB \Rightarrow aaab$
2. $S \Rightarrow AB \Rightarrow Ab \Rightarrow aAb \Rightarrow aaab$
3. $S \Rightarrow Bb \Rightarrow aBbb \Rightarrow abbb$

Las primeras dos producen la misma cadena, en un orden distinto de derivaciones. La primera derivación se obtiene sustituyendo siempre la variable más a la izquierda, por lo que es una derivación izquierda. *De la misma forma, la segunda derivación es una* derivación derecha.

2.1.2. Árboles de derivación

Una manera de ver gráficamente una derivación completa es con su árbol de derivaciones. Consiste en un diagrama en forma de árbol, con las sustituciones de variables como ramificaciones y los símbolos terminales como las hojas.

Para introducir esta idea, se puede partir de la definición de árbol como estructura de datos. Sea cada dato almacenado en una estructura básica llamada *nodo*.

Definición 17. *Un* árbol *T es un conjunto no vacío n de nodos, donde uno de los nodos es llamado raíz y el resto están organizados en subconjuntos disjuntos que son a su vez árboles, llamados subárboles o hijos de la raíz.*

Esta definición es en esencia recursiva pues se ha definido el tipo abstracto de datos árbol de $n4$ nodos en términos de árboles de menos de n nodos. Los nodos que ya no tiene hijos se llaman hojas.

Un árbol de derivación es una representación, preferentemente gráfica, del conjunto total de las derivaciones gramaticales que producen una cadena. Para el ejemplo 48, la cadena *aaab* tiene el árbol de derivaciones de la figura 2.1.

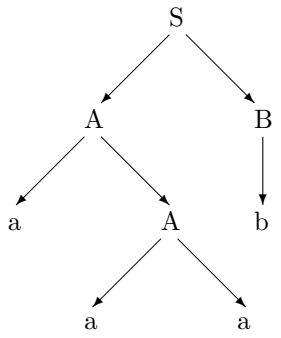


Figura 2.1: Árbol de derivación de la cadena *aaab*.

En ocasiones se prefiere dibujar simples segmentos de recta en vez de flechas. En los árboles de derivación, las hojas siempre son símbolos terminales de la gramática o incluso la cadena vacía. Las variables siempre son nodos intermedios.

Un mismo árbol corresponde a varias derivaciones, pero a una sola derivación izquierda y a una sola derivación derecha. La derivación izquierda sustituye las variables en el orden de un recorrido en preorden del árbol que visita los hijos de izquierda a derecha. La derivación derecha corresponde a un preorden que visita los hijos de derecha a izquierda; leída al revés, coincide con un recorrido en postorden, que es el orden en que trabajan los analizadores ascendentes.

2.1.3. Formas normales

Es de gran utilidad práctica que cada gramática tenga reglas de producción con el mismo formato, para facilitar su análisis y uso. Las siguientes son las más importantes, aunque en la literatura especializada hay más.

Forma Normal de Chomsky

Una gramática está en forma normal de Chomsky (FNC) cuando en sus producciones, cada variable se transforma en otras dos variables o en un terminal, es decir, las producciones son de la forma

$$A \to BC \tag{2.15}$$

o

$$A \to a \tag{2.16}$$

Si el lenguaje contiene a la cadena vacía, se permite además la producción $S \to \epsilon$, siempre que S no aparezca en el lado derecho de ninguna producción.

Toda gramática que no genere la cadena vacía se puede poner en FNC. Su uso tiene varias ventajas. Los árboles sintácticos son binarios y son fáciles de utlizar. Hay muchos algorimtos para convertir toda gramática en FNC.

Proposición 11. *Cada LLC tiene una gramática en Forma Normal de Chomsky que la genera. Toda gramática en Forma Nornal de Chomsky genera un LLC.*

Demostración. Sea G una GLC. La conversión se hace en varios pasos, cada uno de los cuales conserva el lenguaje generado, salvo posiblemente la cadena vacía.

1. Se simplifica la gramática como en la sección 2.1.4: se eliminan las producciones ϵ, las producciones unitarias y los símbolos inútiles, en ese orden.
2. En cada producción cuyo lado derecho tiene dos o más símbolos, cada terminal a se sustituye por una variable nueva C_a y se agrega la producción $C_a \to a$.
3. Cada producción $X \to Y_1 Y_2 \cdots Y_n$ con $n \geq 3$ variables se sustituye por producciones con la forma (2.15), generando variables intermedias nuevas:

$$\begin{aligned} X &\to Y_1 Z_1 \\ Z_1 &\to Y_2 Z_2 \\ &\vdots \\ Z_{n-3} &\to Y_{n-2} Z_{n-2} \\ Z_{n-2} &\to Y_{n-1} Y_n \end{aligned}$$

Al terminar, todas las producciones tienen la forma (2.15) o (2.16). El recíproco es inmediato: una gramática en FNC es, en particular, una GLC. □

Ejemplo 49. *Para la gramática del ejemplo 47, sólo hay que cambiar las dos primeras reglas de producción, pues las dos últimas ya están en Forma Normal de Chomsky. La regla $S \to 0S0$ genera las producciones*

$$\begin{aligned} S &\to C C_1 \\ C_1 &\to S C \end{aligned}$$

y la regla $S \to 1S1$ genera las producciones

$$\begin{aligned} S &\to U U_1 \\ U_1 &\to S U \end{aligned}$$

finalizando con las producciones $C \to 0$ y $U \to 1$.

Forma Normal de Greibach

Una gramática está en forma normal de Greibach cuando cada producción es de la forma

$$A \to a\alpha,$$

donde $a \in T$ y $\alpha \in V^*$ es una concatenación de cero o más variables. Como en la FNC, si el lenguaje contiene a la cadena vacía se permite además $S \to \epsilon$.

Como cada producción genera exactamente un símbolo terminal, cada paso de derivación produce un símbolo de la cadena. Por ello, una cadena w se deriva en exactamente $|w|$ pasos.

La conversión a esta forma es mucho más laboriosa y suele hacerse partiendo de la Forma Normal de Chomsky.

2.1.4. Simplificación de gramáticas

En esta sección veremos la manera de eliminar de una gramática aquello que no es útil, ya sea variables o terminales que no ayudan a generar cadenas, así como dos tipos de producciones que complican el análisis: las producciones ϵ y las producciones unitarias. En los tres casos el lenguaje generado no cambia, salvo que al eliminar las producciones ϵ se pierde, si estaba, la cadena vacía.

Símbolos inútiles

Un símbolo X es *útil* si aparece en alguna derivación de una cadena de terminales, es decir, si existe una derivación $S \Rightarrow^* \alpha X \beta \Rightarrow^* w$ con $w \in T^*$. Para ello X debe cumplir dos condiciones:

- ser *generador*: $X \Rightarrow^* w$ para alguna $w \in T^*$, y
- ser *alcanzable*: $S \Rightarrow^* \alpha X \beta$ para algunas formas sentenciales α y β.

Ambos conjuntos se calculan de manera inductiva, repitiendo hasta que ya no cambien. Todo terminal es generador, y si hay una producción $A \to \alpha$ en la que todos los símbolos de α son generadores (en particular, si $A \to \epsilon$), entonces A es generador. Por otra parte, S es alcanzable, y si A es alcanzable y $A \to \alpha$ es una producción, todos los símbolos de α son alcanzables.

Para eliminar los símbolos inútiles, primero se eliminan los que no son generadores, junto con todas las producciones en las que aparecen, y después los que no son alcanzables. El orden importa. Con la gramática

$$S \to AB \mid a, \qquad A \to b$$

la variable B no es generadora; al eliminarla queda $S \to a$, $A \to b$, y ahora A ya no es alcanzable, así que el resultado es $S \to a$. Si se hiciera en el orden contrario, todos los símbolos serían alcanzables al principio y la producción $A \to b$, que es inútil, se quedaría en la gramática.

Producciones ϵ

Una variable A es *anulable* si $A \Rightarrow^* \epsilon$. Si $A \to \epsilon$ es una producción, A es anulable, y si hay una producción $A \to B_1 B_2 \cdots B_k$ en la que todas las B_i son anulables, A también lo es.

Para eliminar las producciones ϵ, por cada producción $A \to X_1 X_2 \cdots X_k$ se agregan todas las versiones que se obtienen omitiendo cualquier subconjunto de los X_i anulables, excepto la que deja vacío el lado derecho. Después se eliminan todas las producciones de la forma $A \to \epsilon$. Por ejemplo, en $S \to aSb \mid \epsilon$ la variable S es anulable; de $S \to aSb$ se obtienen $S \to aSb \mid ab$, que generan $\{a^n b^n \mid n \geq 1\}$, el lenguaje original sin la cadena vacía. Si se quiere conservar ϵ, se agrega un nuevo símbolo inicial S_0 con $S_0 \to S \mid \epsilon$.

Producciones unitarias

Una *producción unitaria* es de la forma $A \to B$, con B una variable. Para eliminarlas se calculan las parejas (A, B) tales que $A \Rightarrow^* B$ usando sólo pro-

ducciones unitarias, lo que incluye las parejas (A, A). Por cada pareja (A, B) y cada producción no unitaria $B \to \alpha$, se agrega $A \to \alpha$; al final se eliminan todas las producciones unitarias. Por ejemplo, la gramática

$$E \to E + T \mid T, \qquad T \to T * F \mid F, \qquad F \to (E) \mid a$$

tiene las producciones unitarias $E \to T$ y $T \to F$, y las parejas (E, T), (E, F) y (T, F), además de (E, E), (T, T) y (F, F). Al eliminarlas se obtiene

$$E \to E + T \mid T * F \mid (E) \mid a, \qquad T \to T * F \mid (E) \mid a, \qquad F \to (E) \mid a.$$

2.1.5. Ejemplos de aplicaciones

Las GLC tienen una gran importancia en el campo de los lenguajes de programación o los lenguajes de marcado (como html o LaTeX). La sintaxis de cada lenguaje se define, en buena parte, mediante una gramática; algunas reglas, como declarar una variable antes de usarla, quedan fuera del alcance de las GLC (sección 2.5). Las expresiones regulares que usan los buscadores y los editores de texto corresponden a un caso particular, el de las gramáticas regulares.

Ejemplo 50. *Una gramática parecida a la del ejemplo 47 genera cadenas con delimitadores correctamente emparejados. Para un lenguaje como C o derivados, los delimitadores son los paréntesis, las llaves y los corchetes. Las reglas de producción son muy simples:*

$$S \to \{S\} \mid [S] \mid (S) \mid SS \mid \epsilon$$

La producción $S \to SS$ es la que permite poner grupos uno tras otro, como en ()[]*; sin ella sólo se generarían anidamientos como* ({[]}).

Ejemplo 51. *Las expresiones aritméticas son otro ejemplo sencillo. El caso de operaciones con números de un solo dígito se obtiene con la siguiente gramática.*

$$E \to E + E \mid E - E \mid E * E \mid E/E \mid (E) \mid D$$
$$D \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

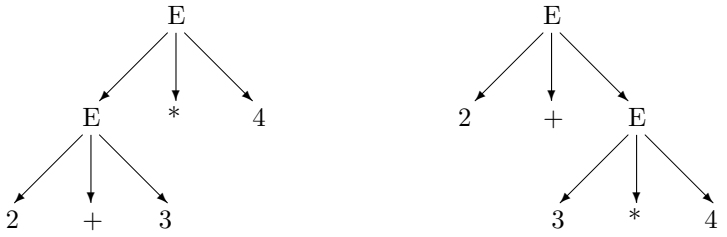
Es fácil modificarla para que los números tengan varios dígitos, por ejemplo agregando $N \to ND \mid D$ y usando N en lugar de D en las producciones de E.

Estos ejemplos caen directamente en lo que se conoce como analizadores sintácticos. Se estudiarán unas secciones más adelante.

2.1.6. Gramáticas ambiguas

Considérese la cadena $2 + 3 * 4$ con la gramática del ejemplo 51. Tiene dos derivaciones izquierdas distintas:

$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow^* 2 + 3 * 4 \tag{2.17}$$
$$E \Rightarrow E + E \Rightarrow^* 2 + E \Rightarrow 2 + E * E \Rightarrow^* 2 + 3 * 4 \tag{2.18}$$



(a) Agrupación $(2 + 3) * 4 = 20$.

(b) Agrupación $2 + (3 * 4) = 14$.

Figura 2.2: Dos árboles de derivación distintos para la misma cadena $2 + 3 * 4$; cada uno le da un significado distinto. Se omiten los nodos D.

A cada una le corresponde un árbol de derivación distinto, como se muestra en la figura 2.2. El primero agrupa la expresión como $(2 + 3) * 4 = 20$ y el segundo como $2 + (3 * 4) = 14$.

Esto significa que la gramática no proporciona medios para diferencias significados. Estos casos son conocidos como gramáticas ambiguas y su ambigüedad se debe a la existencia de más de un árbol de derivación distinto para una misma cadena, lo que no es conveniente en la construcción de compiladores.

Para eliminar la ambigüedad es necesario poder indicar la precedencia de las operaciones, o de las producciones, en general. Esto se hace agregando variables o reglas que obliguen tal precedencia. La gramática se puede construir como

$$
\begin{aligned}
E &\to E + T \mid E - T \mid T \\
T &\to T * F \mid T/F \mid F \\
F &\to (E) \mid D \\
D &\to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}
$$

En esta gramática, $*$ y $/$ quedan más abajo en el árbol que $+$ y $-$, por lo que se agrupan primero: tienen mayor precedencia. Además, la recursión por la izquierda hace que las operaciones del mismo nivel se agrupen de izquierda a derecha, de modo que $8 - 3 - 2$ se lee como $(8 - 3) - 2$. Esta gramática tiene recursividad izquierda; en la sección 2.3.1 se verá cómo eliminarla.

Agregar variables o reglas no es la única manera de resolver este problema, en ocasiones se puede especificar la forma de agrupar las operaciones.

No siempre es posible eliminar la ambigüedad: existen lenguajes para los que todas sus gramáticas son ambiguas, llamados *inherentemente ambiguos*. Además, no existe un algoritmo que decida, para cualquier GLC, si es ambigua; se trata de un problema *indecidible*, un concepto que se estudiará en el último capítulo.

Ejemplo 52. *La gramática*

$$S \to 0A \mid 1B$$
$$A \to 0AA \mid 1S \mid 1$$
$$B \to 1BB \mid 0S \mid 0$$

es ambigua pues la cadena 001101 tiene dos derivaciones izquierdas distintas:

$$S \Rightarrow 0A \Rightarrow 00AA \Rightarrow 001SA \Rightarrow 0011BA \Rightarrow 00110A \Rightarrow 001101$$
$$S \Rightarrow 0A \Rightarrow 00AA \Rightarrow 001A \Rightarrow 0011S \Rightarrow 00110A \Rightarrow 001101$$

Claro, si se puede encontrar una gramática equivalente que no sea ambigua, no hay problema. Casos irresolubles no tienen esta opción:

Ejemplo 53. *Un ejemplo es el lenguaje*

$$L = \{a^n b^n c^m d^m | n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n | n \geq 1, m \geq 1\}$$

para la cuál no hay ni una gramática que no sea ambigua.

Por fortuna, la gran mayoría de lenguajes útiles en la práctica no son inherentemente ambiguos, por lo que no ofrecen problemas para encontrar una gramática adecuada para la construcción de compiladores.

Se sabe que si una cadena con dos árboles de derivación distintos, también tiene dos derivaciones izquierdas distintas y viceversa.

Hay mucha más teoría desarrollada con respecto a las gramáticas libres de contexto, pero rebasa los alcances de este curso. Para mayor información, remítase el lector a libros especializados en el tema.

2.2. Autómatas de Pila

Un Autómata de Pila (AP) es un autómata al que se le da la posibilidad de llevar una memoria sencilla en forma de pila, estructura de datos tipo lista donde se agrega y retira información siempre por el mismo extremo.

De manera similar que con los AFD (página 16), un AP es una tupla de conjuntos, función y elementos $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ que consiste en lo siguiente:

1. Q un conjunto finito de estados,
2. Σ el alfabeto de la cinta de entrada,
3. Γ el alfabeto de la pila,
4. δ la función de transición, que a cada terna $(q, a, X) \in Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma$ le asocia un conjunto finito, posiblemente vacío, de parejas $(p, \gamma) \in Q \times \Gamma^*$,
5. $q_0 \in Q$ el estado inicial,
6. $Z_0 \in \Gamma$ el símbolo del fondo de la pila y
7. $F \subseteq Q$ el conjunto de los estados finales o de aceptación.

De esta manera, un AP es una tupla $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ junto con una cinta de entrada donde se escribe una cadena de caracteres que será procesada. La pila inicia con Z_0 como único elemento. Las transiciones posibles dependen del estado actual, del símbolo que se lee en la cinta de entrada (o de ninguno, si es una transición ϵ) y del símbolo que está en el tope de la pila, que se saca para revisar. Como δ puede ofrecer varias opciones, en general un AP es no determinista.

Las transiciones tienen la forma

$$(p, w) \in \delta(q, a, x) \tag{2.19}$$

que indica que, estando en el estado q, el AP puede procesar el caracter a de la entrada al mismo tiempo que saca el símbolo x del tope de la pila, pasar al estado p y colocar la cadena w en la pila, de modo que el primer símbolo de w queda en el tope. Si el símbolo a no está al inicio de la entrada que se está procesando o x no está en el tope de la pila, entonces la transición no se puede llevar a cabo.

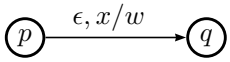
Una pregunta que es importante hacer es si la pila debe quedar vacía cuando el autómata termine de procesar una cadena como requisito para aceptarla. Este es un detalle importante que se estudiará en breve.

Hay varias estrategias que se siguen para procesar cadenas y trabajar con la pila. Dos de gran utilidad son las siguientes.

1. Una transición de la forma

   $$(q, w) \in \delta(p, \epsilon, x)$$

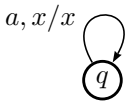
   que actúa sobre la pila (retira x del tope y agrega la cadena w) sin procesar la cadena de entrada. Es esta forma la que requiere que la función δ tenga como parte de su dominio a $\Sigma \cup \{\epsilon\}$ y no sólo a Σ.

2. La transición

   $$(q, x) \in \delta(q, a, x)$$

   procesa el símbolo a de la cadena de entrada y deja intacta la pila, pues saca x del tope y lo regresa.

Las anteriores dos transiciones aparecen comúnmente en diversas etapas de los autómatas de pila. Veamos estas estrategias en uso con algunos ejemplos.

Ejemplo 54. *El lenguaje no regular 0^n1^n puede ser generado por una gramática muy sencilla, mostrada en el ejemplo 46 (pág. 60). Cada cadena producida*

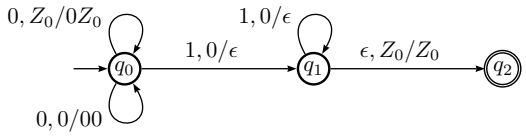


Figura 2.3: Autómata de pila que reconoce el lenguaje 0^n1^n. Se entiende que al inicio la pila sólo contiene a Z_0. Este autómata es completamente determinístico.

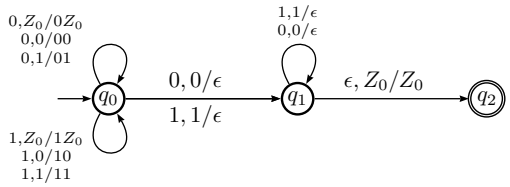


Figura 2.4: Autómata de pila que acepta palindromas de longitud par del alfabeto $\Sigma = \{0, 1\}$. Este autómata es no determinístico, pues el autómata no sabe dónde es la mitad de la cadena.

por esa gramática puede ser aceptada por un autómata de pila. Basta que el autómata procese todos los símbolos 0 agregándolos a la pila y cuando encuentre los símbolos 1, saque un 0 por cada 1 para verificar que sean la misma cantidad. Este autómata se muestra en la figura 2.3.

Es importante notar que cada transición del autómata de pila del ejemplo anterior es determinística. En ningún momento hay forma de tomar dos rutas distintas. No es el caso del autómata siguiente.

Ejemplo 55. *retomando los palindromas generados por la gramática del ejemplo 47, también pueden ser reconocidos por un autómata de pila que guarde la mitad de la cadena en la pila y luego vaya sacando cada símbolo para comprobar que es igual al que corresponde en los que quedan en la cinta de entrada. Esto se hace con el autómata de pila del diagrama 2.4, solo que para palindromas de longitud par.*

El estado q_0 guarda en la pila la primera mitad de la cadena, el estado q_1 comprueba si lo guardado en la pila es idéntico a lo que resta en la cinta de entrada.

En el caso del ejemplo 2.4, la transición que se selecciona depende al mismo tiempo del símbolo de la cinta de entrada y del que está en el tope de la pila, como se dijo en la definición. Con este autómata, hay que notar lo siguiente.

Supongamos que estamos en el estado q_0 y que ya procesamos los dos primeros símbolos de la cadena 00011000. En la cinta de entrada hay un 0 (el tercero) en la cabeza de lectura y en el tope de la pila otro 0 (el que se acaba de procesar). Se pueden usar dos transiciones, una que lo mantiene en q_0 y otra que lo lleva

a q_1. Quedarse en q_0 es lo correcto, pues aún no se llega a la mitad.

Este autómata tiene que adivinar *dónde está la mitad de la cadena. No es un defecto de este diseño en particular: se sabe que ningún autómata de pila determinista acepta este lenguaje. En la sección 2.2.5 se verá que los autómatas de pila deterministas aceptan menos lenguajes que los no deterministas.*

2.2.1. Descripción instantánea de los AP

Lo que ocurre con el AP en un momento arbitrario puede ser descrito con la tripleta (q, w, α), llamada descripción instantánea *del AP, donde*

1. *$q \in Q$ es el estado actual,*
2. *$w \in \Sigma^*$ es lo que resta de la cadena que se procesa y*
3. *$\alpha \in \Gamma^*$ el contenido de la pila, con su primer símbolo en el tope.*

De esta manera, con la descripción instantánea es posible determinar el estado exacto del autómata y los pasos sucesivos en el procesamiento de la cadena de entrada. Debe ser claro que al inicio, todo autómata de pila tiene la descripción instantánea (q_0, w, Z_0), es decir, el estado inicial, toda la cadena original w que se procesará y el símbolo Z_0 como contenido único de la pila.

Para indicar que se pasa de una descripción estándar a otra se emplea el símbolo $\vdash$.

La descripción instantánea del final del procesamiento dependerá de la cadena y si es se tiene o no éxito. Para un autómata que logra procesar toda su cadena y regresar la pila a su contenido inicial, tal descripción queda (q_f, ϵ, Z_0), donde se supone que q_f es un estado final. Si la pila queda completamente vacía, la descripción es (q, ϵ, ϵ).

Para la cadena 000111*, la sucesión de descripciones instantáneas del autómata de pila del ejemplo 2.3 son*

$$(q_0, 000111, Z_0) \vdash (q_0, 00111, 0Z_0) \vdash (q_0, 0111, 00Z_0) \vdash (q_0, 111, 000Z_0) \vdash (q_1, 11, 00Z_0) \vdash (q_1, 1, 0Z_0) \vdash (q_1, \epsilon, Z_0) \vdash (q_2, \epsilon, Z_0)$$

por lo que la cadena es aceptada.

Una cadena que no pertenece al lenguaje no alcanzará el estado final. Por ejemplo, 001 falla pues cuando se procesa el 1, se saca un cero de la pila, pero queda uno más, por lo que la última transición no se puede realizar.

En el ejemplo anterior, también es válido resumir la sucesión de descripciones instantáneas como

$$(q_0, 000111, Z_0) \vdash^* (q_2, \epsilon, Z_0) \tag{2.20}$$

donde el símbolo $\vdash^*$ indica que se aplican cero o más transiciones para pasar de la primera a la última.

Ejemplo 56. *En todo lenguaje de programación procedimental existen las sentencias de control* `if-else`*. Se entiende que debe haber un* `if` *para cada* `else`*,*

pero no puede haber `else` *sin* `if`*. Claro sentencias* `if` *sin* `else` *sí son permitidas. Si sólo se emplean las iniciales ie en vez de* `if else`*, se puede pensar en construciones de sentencias de control representadas con cadenas de símbolos* $\{i, e\}$*, dando lugar a el lenguaje* L_{ie} *que representa las construcciones correctas de sentencias de control* `if-else`*.*

Ejemplos de cadenas en este lenguaje son

$$\{i, ie, ii, iie, iei, ieie, iiee, iieie, iiieieiie, \ldots\}$$

y cadenas inválidas son

$$\{e, ei, iee, ieei, iiieeee, \ldots\}.$$

Una manera de describir este lenguaje es decir que consta de todas las cadenas de símbolos i *y* e *que comienzan con* i *y siempre hay al menos la misma cantidad de símbolos* i *que* e *cuando se cuenta desde el inicio de la cadena. Un autómata de pila que acepte este lenguaje es fácil de construir pues sólo requiere enviar a la pila cada* i *que llega y sacar una* i *cada vez que se procesa una* e*.*

En este caso, una cadena se acepta sin importar si la pila está vacía.

En el ejemplo anterior, el AP acepta por estado final aunque al terminar queden símbolos i en la pila, que corresponden a sentencias `if` sin `else`. Esto no es un problema: el AP es un modelo matemático y su pila no es memoria que haya que liberar. Lo que importa es fijar con precisión el criterio de aceptación, que es el tema de la siguiente sección.

Este lenguaje está relacionado con un problema real de los compiladores, el del `else` *colgante*: en `if a if b x else y`, la gramática del lenguaje debe indicar a cuál de los dos `if` pertenece el `else`. Una gramática que permita las dos lecturas es ambigua.

2.2.2. Lenguaje de un Autómata de Pila

Con las definiciones anteriores, ya podemos definir cuál es el lenguaje de un Autómata de Pila. Dado el autómata $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, hay dos maneras naturales de hacerlo. El lenguaje que acepta *por estado final* es

$$L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \alpha) \text{ con } p \in F \text{ y } \alpha \in \Gamma^*\}, \tag{2.21}$$

es decir, se acepta al terminar de leer la cadena en un estado final, sin importar lo que quede en la pila. El lenguaje que acepta *por pila vacía* es

$$N(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon) \text{ para algún } p \in Q\}.$$

Para un mismo AP, los dos lenguajes pueden ser distintos. Sin embargo, los dos criterios son igual de poderosos: para todo AP P existe otro AP P' tal que $N(P') = L(P)$, y viceversa. Por eso la clase de lenguajes aceptados por los AP no depende del criterio que se elija. Para los autómatas de pila deterministas sí hay diferencia, como se comenta en la sección 2.2.5.

2.2.3. Equivalencia entre GLL y AP

Ahora falta probar que las GLC son equivalentes a los AP, y de esta forma comprobaremos que el lenguaje de los AP son los Lenguajes Libres de Contexto (definidos como en la ecuación (2.11). Tal y como las LR equivalen a los AFD, en el sentido de que para cada LR hay un AFD y viceversa, también para cada GLC hay un AP y viceversa.

Primero, a partir de una gramática G se puede construir un Autómata de Pila P que simule la gramática. Es realmente sencilla la construcción. Sea la gramática $G = (V, T, P, S)$. El autómata se construye como

$$P = (\{q_0\}, T, T \cup V, \delta, q_0, S, \emptyset)$$

un autómata de un sólo estado que acepta *por pila vacía*. El símbolo inicial S de la gramática hace el papel de Z_0, el contenido inicial de la pila. Las transiciones se forman de la siguiente manera. Para cada producción

$$X \to \alpha$$

con α una combinación de variables y terminales, es decir, $\alpha \in (V \cup T)^*$, se construye la transición que no procesa ningún caracter

$$(q_0, \alpha) \in \delta(q_0, \epsilon, X)$$

lo que significa que si el autómata P encuentra la variable X en la pila, la retira y la sustituye por el lado derecho α de la producción, con el primer símbolo de α en el tope.

Aquí aparece el no determinismo: si X tiene varias producciones, el AP puede elegir cualquiera de ellas, y acepta si alguna sucesión de elecciones vacía la pila al terminar de leer la entrada. El AP simula así derivaciones izquierdas, pues siempre expande la variable que está en el tope, que es la que está más a la izquierda en la parte pendiente de la forma sentencial.

Para cada terminal a, se construye la transición

$$(q_0, \epsilon) \in \delta(q_0, a, a)$$

que procesa el símbolo a de la entrada cuando ese mismo símbolo está en el tope de la pila, y lo retira.

Ejemplo 57. *Retomando el ejemplo del languaje $a^n b^n$, generado por la gramática*

$$S \to aSb|ab$$

el autómata que lo simula se presenta en la figura 2.5.

Para la cadena aaabbb, el autómata procede con las descripciones instantaneas siguientes:

$$\begin{aligned}(q_0, aaabbb, S) &\vdash (q_0, aaabbb, aSb) \vdash (q_0, aabbb, Sb) \\ &\vdash (q_0, aabbb, aSbb) \vdash (q_0, abbb, Sbb) \\ &\vdash (q_0, abbb, abbb) \vdash (q_0, bbb, bbb) \\ &\vdash (q_0, bb, bb) \vdash (q_0, b, b) \vdash (q_0, \epsilon, \epsilon)\end{aligned}$$

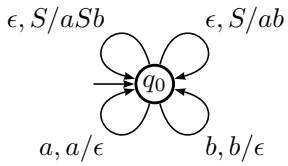


Figura 2.5: Autómata de pila que simula la gramática del lenguaje $a^n b^n$. Acepta por pila vacía.

Con esta sucesión de descripciones instantáneas, es evidente que lo que simula el autómata es la derivación (por la izquierda):

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb.$$

Esta construcción demuestra que todo lenguaje generado por una GLC es aceptado por algún AP. La dirección contraria también es cierta: todo lenguaje aceptado por un AP es libre de contexto. Su demostración es más laboriosa y no se presenta en estas notas; puede consultarse en los textos clásicos del tema, como el de Hopcroft, Motwani y Ullman.

2.2.4. Limitaciones de los AP

Así como se presentó el lema de bombeo para lenguajes regulares, existe un resultado equivalente para LLC, que se emplea para mostrar que algunos lenguajes no son libres de contexto.

Proposición 12 (Lema del bombeo para lenguajes libres de contexto)**.** [1] *Supongamos un lenguaje L que sea un LLC. Entonces existe una constante n, que depende del lenguaje L, tal que para toda cadena $z \in L$, con $|z| \geq n$, se puede descomponer z en cinco cadenas $z = uvwxy$ que cumplen con*

1. *$vx \neq \epsilon$,*
2. *$|vwx| \leq n$ y*
3. *para toda $k \geq 0$, la cadena $uv^k wx^k y$ también pertence a L.*

Ejemplo 58. *El lenguaje $L = \{a^n b^n c^n \mid n \geq 1\}$ no es libre de contexto.*

Demostración. Supongamos, para llegar a una contradicción, que L es libre de contexto, y sea n la constante del lema. Tomemos $z = a^n b^n c^n \in L$, que cumple $|z| = 3n \geq n$. Sea $z = uvwxy$ cualquier descomposición con $vx \neq \epsilon$ y $|vwx| \leq n$. Como entre la última a y la primera c hay n símbolos b, la subcadena vwx, de longitud a lo más n, no puede contener a la vez símbolos a y c. Por lo tanto, alguno de los tres símbolos no aparece en v ni en x. Con $k = 0$, la cadena uwy pierde al menos un símbolo (pues $vx \neq \epsilon$), pero conserva todos los del símbolo ausente, así que ya no tiene la misma cantidad de a, b y c. Entonces $uwy \notin L$, lo que contradice el lema. □

[1] También conocido como lema de Bar-Hillel.

Compárese con el ejemplo 45: el lenguaje 0^n1^n no es regular, pero sí es libre de contexto (un autómata de pila lo acepta, como se vio en la figura 2.3); en cambio, $a^nb^nc^n$ ni siquiera es libre de contexto.

Como en el caso del lema del bombeo para lenguajes regulares, hay lenguajes que no son libres de contexto y satisfacen el lema del bombeo.

Existen herramientas más poderosas para probar que un lenguaje no es LLC, pero ninguna de ellas es un criterio completo. El siguiente resultado es más fuerte que el lema del bombeo.

Proposición 13 (Lema de Ogden)**.** *Si un lenguaje L es libre de contexto, entonces existe un número k tal que para cualquier cadena $w \in L$ en la que se marquen k o más posiciones, la cadena puede separarse como $w = uxyzv$ y las subcadenas cumplen*

1. *las cadenas x y z, juntas, tienen al menos una posición marcada,*
2. *xyz tiene a lo más k posiciones marcadas y*
3. *$ux^iyz^iv \in L$ para cada $i \geq 0$.*

Si se marcan todas las posiciones de w, el lema de Ogden se reduce al lema del bombeo; la ventaja de poder marcar sólo algunas es que se controla en qué parte de la cadena cae la porción que se bombea.

2.2.5. Autómatas de Pila Determinísticos (APD)

Un autómata de pila es *determinista* si en cada descripción instantánea tiene a lo más una transición aplicable: para cada estado q, símbolo a y símbolo de pila X, el conjunto $\delta(q, a, X)$ tiene a lo más un elemento, y si $\delta(q, \epsilon, X)$ no es vacío, entonces $\delta(q, a, X)$ es vacío para todo $a \in \Sigma$. Por ejemplo, el autómata de la figura 2.3 es determinista.

A diferencia de lo que ocurre con los autómatas finitos, el no determinismo sí aumenta el poder de los autómatas de pila. El lenguaje $L = \{x^ny^n \mid n > 0\} \cup \{x^ny^{2n} \mid n > 0\}$ no es aceptado por ningún autómata de pila determinista, y tampoco lo es el de los palíndromos de longitud par del ejemplo 55, aunque ambos son libres de contexto.

Los lenguajes aceptados por estado final por los APD se denominan *Lenguajes Libres de Contexto Deterministas*. Es un conjunto más reducido que el de los Lenguajes Libres de Contexto, pero muy útil para el análisis sintáctico: todo lenguaje libre de contexto determinista tiene una gramática no ambigua, y es exactamente la clase de lenguajes que reconocen los analizadores LR(1). Además, a diferencia de los LLC en general, son cerrados bajo complemento.

Aún más, el conjunto de lenguajes se reduce si se exige que el APD acepte por pila vacía, porque entonces no puede aceptar una cadena y también una extensión de ella: al aceptar la primera, la pila ya quedó vacía y no puede continuar. Por ejemplo, ningún APD acepta por pila vacía el lenguaje regular 0^*.

2.3. Lenguajes y compiladores

Los lenguajes de programación y los compiladores con que trabajamos son dos importantísimas aplicaciones, por demás evidentes, de la teoría de lenguajes formales, particularmente de las gramáticas libres de contexto. Es la razón principal de su estudio.

La manera como se definen la mayoría de las características de un lenguaje es mediante gramáticas. Por ejemplo, en lenguaje C, las sentencias de iteración se declaran con las reglas de producción como

`while` (*Expr*) *Sentencia*

`do` *Sentencia* `while` (*Expr*);

`for` (*Decl Expr* ; *Expr* ; *Expr*) *Sentencia*

A su vez, *Expr* y *Sentencia* son variables de la gramática que tiene sus propias producciones.

Lo que hay en un compilador es un analizador sintáctico, cuyo objetivo es resolver un problema: determinar si la cadena w (el programa que escribimos) pertenece al lenguaje generado por la gramática del lenguaje y, de paso, construir su árbol de derivación, que es la estructura sobre la que trabajan las etapas siguientes del compilador. Esto se responde encontrando una derivación con las producciones de la gramática del lenguaje que lleven a obtener w, lo puede hacerse de varias maneras, ya sea partiendo del símbolo inicial S y llegando hasta la cadena w, lo que se conoce como *análisis descendente*. Otra forma es en sentido contrario, pertiendo de la cadena y encontrando la derivación en sentido inverso, conocido como *análisis ascendente*. También es posible combinar ambas estrategias.

Como la derivación se puede encontrar derivando por la izquierda, por la derecha o leyendo la cadena en ambos sentidos, esto da origen a diversos analizadores sintácticos, cada uno con características que lo hace deseable para propósitos específicos o restricciones de eficiencia.

Así como la tabla de transiciones de un AFD permite construir con gran facilidad un analizador léxico que reconoce cierto lenguaje, para un AP es posible definir una tabla que facilita el trabajo de un analizador sintáctico. El mayor problema en la práctica es el no determinismo. Existen algoritmos generales que deciden si una cadena pertenece a cualquier LLC, como el algoritmo CYK, que trabaja con la gramática en Forma Normal de Chomsky y tarda un tiempo proporcional a n^3 para una cadena de longitud n. Para un compilador eso es demasiado lento, pues se quiere analizar programas largos en un tiempo proporcional a su longitud. Por eso se usan gramáticas con las que el análisis puede hacerse de manera determinista.

Por lo general se parte de la gramática y se la una forma adecuada para que el proceso de análisis sea eficiente. Para el análisis descendente, el obstáculo principal es la recursividad izquierda, que siempre puede eliminarse sin cambiar el lenguaje. Se mostrará la manera de hecerlo y luego se presentan dos

de los analizadores más simples, que hace uso de una gramática donde no hay recursividad izquierda.

Entre los analizadores deterministas más usados están los LL, que leen la entrada de izquierda a derecha y construyen una derivación izquierda, y los LR, que también leen de izquierda a derecha, pero construyen una derivación derecha en sentido inverso (análisis ascendente). Los LR y su variante LALR son más poderosos, no tienen problema con la recursividad izquierda y son los que generan herramientas como Yacc y Bison; no se presentarán en estas notas. Como ambos tipos son deterministas, ninguno reconoce todos los LLC.

2.3.1. Eliminación de recursividad izquierda

Se dice que una producción es recursiva por la izquierda cuando la variable que la produce aparece en la primera posición en la forma sentencial producida, es decir, cuando son de esta forma:

$$X \to X\alpha \tag{2.22}$$

con α cualquier combinación de variables y terminales. Es claro que la intención de una producción de esa naturaleza es repetir α cuantas veces se requiera, pero a la hora de programarla, la variable X (que se convierte en una función del programa, se llama a sí misma de inmediato, lo que ocasiona un ciclo recursivo del que no se puede salir[2].

La producción (2.22) no puede estar sola, debe haber otra producción para la variable X. De no ser así, se estaría sustituyendo a sí misma recursivamente sin parar. Debe haber otra producción cuyo lado derecho no comienza con X, de la forma $X \to \beta$. De esta forma, la sustitución de X puede ser algo como

$$X \Rightarrow X\alpha \Rightarrow X\alpha\alpha \Rightarrow X\alpha\alpha\alpha \Rightarrow \beta\alpha\alpha\alpha$$

y se han aplicado 3 veces la producción (2.22) y $X \to \beta$ una vez al final. Esta derivación sugiere la idea de producir primero lo no recursivo y luego realizar las repeticiones. Para ello se crea una nueva variable T y las producciones de X se sustituyen por

$$X \to \beta T \tag{2.23}$$

$$T \to \alpha T \mid \epsilon \tag{2.24}$$

Así, X genera primero β y después T genera cualquier cantidad de repeticiones de α:

$$X \Rightarrow \beta T \Rightarrow \beta\alpha T \Rightarrow \beta\alpha\alpha T \Rightarrow \beta\alpha\alpha\alpha T \Rightarrow \beta\alpha\alpha\alpha,$$

que es el mismo resultado que antes. De esta forma se tiene una gramática equivalente a $X \to X\alpha \mid \beta$ que no es recursiva por la izquierda.

Incluso en el caso de que una variable tenga una producción como

$$A \to A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \beta_1 \mid \beta_2$$

[2]Hasta que la pila del sistema se llena y el programa es botado de la memoria.

será necesaria solo una variable extra para eliminar la recursividad:

$$A \to \beta_1 T \mid \beta_2 T$$
$$T \to \alpha_1 T \mid \alpha_2 T \mid \alpha_3 T \mid \epsilon.$$

Esta técnica elimina la recursividad izquierda *inmediata*. También puede haber recursividad izquierda indirecta, como en $A \to B\alpha$, $B \to A\beta$, donde $A \Rightarrow^+ A\beta\alpha$; para eliminarla hay un algoritmo general que primero sustituye unas variables en otras hasta volverla inmediata.

Por ejemplo, al aplicar esta técnica a la gramática no ambigua de las expresiones aritméticas, con a en lugar de D, se obtiene

$$E \to TE' \qquad E' \to +TE' \mid \epsilon$$
$$T \to FT' \qquad T' \to *FT' \mid \epsilon$$
$$F \to (E) \mid a$$

donde, por brevedad, se omitieron la resta y la división.

2.3.2. Analizadores Recursivos Descendentes

Los analizadores recursivos descendentes son los más sencillos de todos y se construyen casi de manera directa a partir de la gramática. Cada variable se convierte en una función y las producciones en el cuerpo de cada función. Considérese la producción

$$A \to aBcA \mid Ba \mid bCA \tag{2.25}$$

que son solo tres producciones de una gramática mayor. Esta variable se convierte de manera directa en una función que se muestra en el código 2.1, donde la cadena que se revisa está en el arreglo `Cad` y la variable `Pos` tiene la posición del símbolo que se está revisando. La función `Procesar` verifica que el símbolo enviado sea el que corresponde y avanza en la revisión de la cadena.

En el código 2.1, la última alternativa, $A \to Ba$, se elige cuando el símbolo actual no es a ni b. Esto sólo es correcto si ninguna cadena derivada de B empieza con a o con b; en general, para escoger la alternativa se usan los conjuntos First que se definen en la sección 2.3.3.

Es por ello que programar analizadores recursivos descendentes es tan sencillo, una vez que la gramática no tiene recursividad izquierda y que las alternativas de cada variable pueden distinguirse con el siguiente símbolo de la entrada. Si la gramática es muy grande, se requiere de programas adicionales para comprobar esas condiciones o para transformar la gramática.

2.3.3. Analizadores LL(1)

Las dos L del nombre se refieren a que estos analizadores leen la entrada (el programa) de izquierda a derecha (*Left-to-right*) y construyen una derivación izquierda (*Leftmost derivation*). Como en ciertas situaciones, se puede aplicar

Código 2.1: La función que procesa la producción (2.25).

```
1  void A()
2  {
3          if ( Cad[Pos] == 'a' ) {
4                  Procesar('a');
5                  B();
6                  Procesar('c');
7                  A();
8          } else if ( Cad[Pos] == 'b' ) {
9                  Procesar('b');
10                 C();
11                 A();
12         } else {
13                 B();
14                 Procesar('a');
15         }
16 }
```

más de una regla de derivación, observa un símbolo de la entrada por adelantado, es decir, sin procesarlo, para que sirva en la decisión de cuál regla utilizar. Es por eso del 1 entre paréntesis, porque se revisa un símbolo.

Es importante aclarar que se presentan este tipo de analizadores por su simplicidad, no por su generalidad. La sintaxis completa de los lenguajes de programación reales rara vez es LL(1). En C, por ejemplo, la expresión `(A)*B` es una conversión de tipo aplicada a `*B` si `A` es un nombre de tipo declarado con `typedef`, y una multiplicación si `A` es una variable. Para decidir, el analizador debe consultar la tabla de símbolos, información que una gramática libre de contexto no tiene. Los compiladores reales combinan técnicas como las de esta sección con ese tipo de información adicional.

En el tema de análisis léxico, sección 1.2.7, página 23, el proceso de análisis es guiado por la tabla de transiciones. En este caso hay una equivalencia pues se construye una tabla que permite ir decidiendo por la producción que será empleada al ir construyendo la derivación izquierda.

Para construir esta tabla guía se necesitan dos conjuntos. Para una cadena α de variables y terminales, First(α) es el conjunto de terminales con los que puede empezar una cadena derivada de α, más ϵ si $\alpha \Rightarrow^* \epsilon$. Para una variable A, Follow(A) es el conjunto de terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial, más el símbolo especial \$, que marca el fin de la entrada, si A puede quedar al final.

El conjunto First de cada símbolo X se calcula con las reglas siguientes, que se aplican repetidamente hasta que ningún conjunto cambie.

1. Si X es un símbolo terminal, entonces First(X) = $\{X\}$.
2. Si $X \to \epsilon$ es una producción, entonces ϵ se agrega a First(X).
3. Si existe la producción $X \to Y_1 Y_2 \cdots Y_k$, se agregan a First(X) los terminales de First(Y_1). Si $\epsilon \in$ First(Y_1), se agregan también los terminales de First(Y_2), y así sucesivamente: los terminales de First(Y_i) se agregan sólo

si ϵ está en $\text{First}(Y_1), \ldots, \text{First}(Y_{i-1})$. Si ϵ está en todos los $\text{First}(Y_i)$, se agrega ϵ a $\text{First}(X)$.

Para una cadena $\alpha = X_1 X_2 \cdots X_n$, $\text{First}(\alpha)$ se calcula con la misma idea de la regla 3: se agregan los terminales de $\text{First}(X_1)$, y los de $\text{First}(X_2)$ sólo si X_1 puede derivar ϵ, y así sucesivamente; $\epsilon \in \text{First}(\alpha)$ sólo si todos los X_i pueden derivar ϵ.

El conjunto Follow de cada variable se calcula con las reglas siguientes, también repetidas hasta que ningún conjunto cambie.

1. Se agrega \$ a $\text{Follow}(S)$.
2. Si hay una producción $A \to \alpha B \beta$, todos los terminales de $\text{First}(\beta)$ se agregan a $\text{Follow}(B)$.
3. Si hay una producción $A \to \alpha B$, o una producción $A \to \alpha B \beta$ con $\epsilon \in \text{First}(\beta)$, todo $\text{Follow}(A)$ se agrega a $\text{Follow}(B)$.

A partir de estos conjuntos se construye la *tabla guía* M, en inglés *parsing table*, con una fila por cada variable y una columna por cada terminal y por \$. Para cada producción $A \to \alpha$:

1. para cada terminal $a \in \text{First}(\alpha)$, se coloca $A \to \alpha$ en $M[A, a]$;
2. si $\epsilon \in \text{First}(\alpha)$, se coloca $A \to \alpha$ en $M[A, b]$ para cada $b \in \text{Follow}(A)$, incluido \$ si está.

Las casillas vacías corresponden a errores de sintaxis. La gramática es LL(1) si ninguna casilla tiene más de una producción; si alguna tiene dos, el analizador no puede decidir con un solo símbolo de anticipación.

El analizador usa una pila, que al inicio contiene $S\$$, con S en el tope, y lee la entrada $w\$$. En cada paso, sea X el tope de la pila y a el símbolo actual de la entrada:

- si $X = a = \$$, la cadena se acepta;
- si X es un terminal, se saca de la pila y se avanza en la entrada cuando $X = a$, y hay un error en caso contrario;
- si X es una variable y $M[X, a]$ contiene $X \to Y_1 Y_2 \cdots Y_k$, se saca X y se colocan $Y_k, \ldots, Y_2, Y_1$ en la pila, de modo que Y_1 quede en el tope; si $M[X, a]$ está vacía, hay un error.

Las producciones aplicadas, en orden, forman la derivación izquierda de w. En el fondo, este analizador es un autómata de pila determinista cuya función de transición está dada por la tabla.

Ejemplo 59. *Considérese la gramática de las expresiones aritméticas sin recursividad izquierda obtenida en la sección 2.3.1:*

$$E \to TE', \quad E' \to +TE' \mid \epsilon, \quad T \to FT', \quad T' \to *FT' \mid \epsilon, \quad F \to (E) \mid a.$$

Sus conjuntos First y Follow son

$$\begin{aligned} \mathit{First}(E) = \mathit{First}(T) = \mathit{First}(F) &= \{(, a\} & \mathit{Follow}(E) = \mathit{Follow}(E') &= \{), \$\} \\ \mathit{First}(E') &= \{+, \epsilon\} & \mathit{Follow}(T) = \mathit{Follow}(T') &= \{+,), \$\} \\ \mathit{First}(T') &= \{*, \epsilon\} & \mathit{Follow}(F) &= \{*, +,), \$\} \end{aligned}$$

y su tabla guía es la tabla 2.1. Ninguna casilla tiene más de una producción, así que la gramática es LL(1).

	a	$+$	$*$	$($	$)$	\$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to a$			$F \to (E)$		

Tabla 2.1: Tabla guía LL(1) de la gramática del ejemplo 59. Las casillas vacías son errores.

Pila	Entrada	Acción
$E\$$	$a+a\$$	$E \to TE'$
$TE'\$$	$a+a\$$	$T \to FT'$
$FT'E'\$$	$a+a\$$	$F \to a$
$aT'E'\$$	$a+a\$$	coincide a
$T'E'\$$	$+a\$$	$T' \to \epsilon$
$E'\$$	$+a\$$	$E' \to +TE'$
$+TE'\$$	$+a\$$	coincide $+$
$TE'\$$	$a\$$	$T \to FT'$
$FT'E'\$$	$a\$$	$F \to a$
$aT'E'\$$	$a\$$	coincide a
$T'E'\$$	$\$$	$T' \to \epsilon$
$E'\$$	$\$$	$E' \to \epsilon$
$\$$	$\$$	acepta

Tabla 2.2: Análisis LL(1) de la cadena $a+a$ con la tabla 2.1.

La tabla 2.2 muestra el análisis de la cadena $a+a$. La columna de la pila se escribe con el tope a la izquierda. La sucesión de producciones de la columna de acciones es la derivación izquierda $E \Rightarrow TE' \Rightarrow FT'E' \Rightarrow aT'E' \Rightarrow aE' \Rightarrow a+TE' \Rightarrow a+FT'E' \Rightarrow a+aT'E' \Rightarrow a+aE' \Rightarrow a+a$.

2.4. Propiedades de los LLC

2.4.1. Propiedades de decisión

Las dos preguntas de decisión más importantes son si un LLC L está vacío y si cierta cadena w pertenece a L.

La primera pregunta se responde encontrando una derivación a partir del símbolo incial hasta una cadena de símbolos terminales. Por lo general se eliminan primero las variables inútiles y al final del algoritmo se ve si sigue existiendo el símbolo inicial.

La segunda pregunta es la que más interesa en la práctica, pues es la que resuelve el *análisis sintáctico* de un compilador. Encontrar la derivación que produce una cadena equivale a saber que el programa no tiene errores de sintaxis,

es decir, que fue construido siguiendo correctamente las reglas gramaticales del lenguaje; eso no garantiza que no tenga otro tipo de errores. Para cualquier GLC existe un algoritmo que responde esta pregunta, por ejemplo el algoritmo CYK mencionado en la sección 2.3.

Otra pregunta es si un LLC es infinito. Primero se simplifica la gramática (sección 2.1.4) y se pone en Forma Normal de Chomsky; después se construye el grafo de dependencias, con una arista de A a B si hay una producción $A \to BC$ o $A \to CB$. El lenguaje es infinito si y sólo si ese grafo tiene un ciclo. La simplificación previa es indispensable: una producción como $A \to A$, o un ciclo entre variables inútiles, no hace infinito al lenguaje.

En cambio, hay preguntas sobre los LLC que no tienen solución algorítmica: no existe un algoritmo que decida, para dos GLC cualesquiera, si generan el mismo lenguaje, ni si una GLC es ambigua, ni si genera todas las cadenas de Σ^*. Son problemas *indecidibles*, un concepto que se estudiará en el último capítulo. Esto contrasta con los lenguajes regulares, para los que la equivalencia sí es decidible.

2.4.2. Propiedades de cerradura

Como ocurre con los lenguajes regulares, los libres de contexto son cerrados bajo ciertas operaciones, pero no las mismas que en el caso de los regulares.

Proposición 14. *Sean L, L_1 y L_2 lenguajes libres de contexto. Se cumple que*

- *$L_1 \cup L_2$ también es libre de contexto.*
- *$L_1 L_2$ también es libre de contexto.*
- *L^* es libre de contexto.*

Como contra parte, hay varias operaciones que en general no son cerradas. Por ejemplo la intersección de LLC. El ejemplo recurrente es el lenguaje $L(a^n b^n c^n)$ que no es libre de contexto, pero sí lo son los lenguajes $L(a^n b^m c^m)$ y $L(a^n b^n c^m)$ cuya intersección es el primero.

El complemento de un LLC no necesariamente es libre de contexto. La razón es sencilla: si los LLC fueran cerrados bajo complemento, como lo son bajo unión, también lo serían bajo intersección, pues $L_1 \cap L_2 = (L_1^C \cup L_2^C)^C$, y acabamos de ver que no lo son. En cambio, los LLC deterministas sí son cerrados bajo complemento.

Como caso curioso, la intersección entre un lenguaje regular y uno libre de contexto sí es libre de contexto. Esta propiedad es útil para demostrar que un lenguaje no es libre de contexto: si al intersectarlo con un lenguaje regular adecuado se obtiene un lenguaje que se sabe que no es libre de contexto, como $a^n b^n c^n$, el original tampoco puede serlo.

2.5. Limitaciones de los lenguajes libres de contexto

Como ya se observó antes, el lenguaje $a^n b^n c^n$ no es libre de contexto y de hecho hay una infinidad de lenguajes que no lo son. Para efectos prácticos, una limitación importante es que muchas reglas de los lenguajes de programación no son libres de contexto. Un ejemplo es la regla de declarar las variables antes de usarlas. El lenguaje $\{wcw \mid w \in \{a, b\}^*\}$, que modela un programa que declara un identificador w y más adelante lo usa, no es libre de contexto, lo que se prueba con el lema del bombeo. Algo parecido ocurre con verificar que los argumentos de una llamada coincidan en número y tipo con la definición de la función.

Por eso los compiladores complementan el análisis sintáctico con un *análisis semántico*, que usa información global, como la tabla de símbolos, para verificar las reglas que la gramática no puede expresar.

Capítulo 3

Lenguajes Recursivamente Enumerables

De los Lenguajes Regulares, aceptados por AFD o generados por ER, se pasó a los Lenguajes Libres de Contexto, aceptados por AP o generados por GLC. Ahora pasamos a un conjunto de lenguajes aún más amplio, aceptados por los autómatas más poderosos, llamados Máquinas de Turing, que equivalen a las gramáticas más generales, llamadas gramáticas no restringidas.

El enfoque cambia un poco pues las MT pueden verse no sólo como aceptadoras de cadenas de un lenguaje, sino como calculadoras de funciones. Se acepta que todo lo que puede calcular una computadora moderna lo puede calcular una MT, una afirmación conocida como *tesis de Church-Turing*. Es una tesis y no un teorema, porque *algoritmo* o *cálculo efectivo* no es un concepto matemático preciso; lo que la respalda es que todos los modelos de cómputo propuestos hasta ahora han resultado equivalentes a las MT o más débiles.

3.1. Máquinas de Turing

Las Máquinas de Turing[1] son el nivel más complejo de modelo abstracto de computadora. Al AFD se le agregó memoria en forma de pila y se consiguió una máquina mucho más poderosa, el autómata de pila. Ahora, al AFD no se le agrega una memoria tipo pila, sino que se le permite escribir en la cinta de entrada y avanzar en ambas direcciones, con lo que de nuevo gana mucho poder. De hecho, de acuerdo con la tesis de Church-Turing, cualquier cálculo que puede realizar la computadora más poderosa de la actualidad es posible con una MT, si se le da suficiente tiempo y cinta.

Una Máquina de Turing M consiste en los siguiente conjuntos:

[1] Propuestas por Alan Mathison Turing en 1936. Marvin Minsky consideraba el artículo de 1936 de Turing la presentación de la computadora moderna y algunas técnicas de programación.

	0	1	X	Y	$\not b$
q_0	(q_1, X, R)	–	–	(q_3, Y, R)	–
q_1	$(q_1, 0, R)$	(q_2, Y, L)	–	(q_1, Y, R)	–
q_2	$(q_2, 0, L)$	–	(q_0, X, R)	(q_2, Y, L)	–
q_3	–	–	–	(q_3, Y, R)	$(q_4, \not b, L)$
q_4	–	–	–	–	–

Tabla 3.1: Tabla de transiciones de la Máquina de Turing que acepta el lenguaje 0^n1^n. La construcción es similar a el caso de los AFD, cada renglón corresponde a un estado y cada columna a un posible símbolo en la cadena de entrada. Su diagrama de transiciones se muestra en la figura 3.1.

1. Q un conjunto finito de estados,
2. $\Sigma \subset \Gamma$ el alfabeto de la cinta de entrada,
3. Γ el alfabeto de la máquina,
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ la función de transición, que puede ser parcial,
5. $q_0 \in Q$ el estado inicial,
6. $B \in \Gamma$, $B \notin \Sigma$ el espacio en blanco y
7. $F \subseteq Q$ el conjunto de estados de aceptación.

De esta manera, una MT es una tupla $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ junto con una cinta dividida en celdas e infinita en ambos sentidos. Al inicio, la cadena de entrada ocupa celdas consecutivas, el resto de la cinta contiene el símbolo $\not b$ y la cabeza de lectura está sobre el primer símbolo de la entrada.

Las transiciones tienen la forma

$$\delta(p, a) = (q, b, Y) \tag{3.1}$$

que indica que, estando en el estado p, la MT procesa el símbolo a de la cinta de entrada, pasando al estado q (que puede ser el mismo estado p), escribiendo el símbolo b en la cinta y avanzando en la dirección Y (L =izquierda, R =derecha).

A diferencia de los anteriores autómatas, cuando la MT llega a un estado de aceptación $q \in F$, debe detener su ejecución, aceptando la cadena, no siendo válidas las transiciones a partir de q. Por eso, transiciones como la descrita en la ecuación (3.1), debe cumplirse que $p \notin F$. Por convención, también se supone que $q_0 \notin F$; de otro modo, la máquina aceptaría toda cadena sin leerla.

Ejemplo 60. *Consideremos el lenguaje $L = \{0^n1^n\}$, que siendo libre de contexto y por tanto aceptable con un AP, nos es útil para ilustrar el comportamiento de una MT. Sea la MT*

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, \not b\}, \delta, q_0, \not b, \{q_4\})$$

dada por la figura 3.1 y las transiciones dadas en la tabla 3.1.

El algoritmo funciona así: al inicio, la cabeza de lectura lee el primer 0, lo sustituye con una X y pasa al estado q_1, que va en busca de un 1, saltándose los

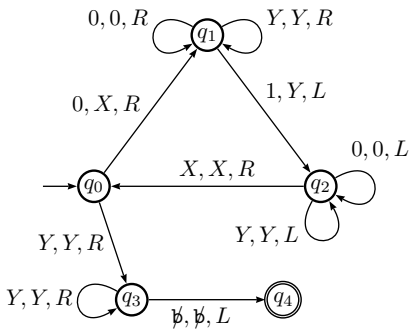


Figura 3.1: Máquina de Turing que acepta el lenguaje 0^n1^n. Su tabla de trasiciones se muestra en la tabla 3.1.

caracteres necesarios, que pueden ser un 0 o una Y. Al encontrar un 1, pasa al estado q_2, lo sustituye el 1 con una Y. El estado q_2 retrocede hasta encontrar la X más cercana. Al encontrarla pasa al estado q_0 y avanza esperando encontrar un 0 para repetir este procedimiento. De no encontrar ni un cero, es de esperarse que se encuentre con una Y, pasando al estado q_3, que se encarga de saltarse todas las Y. Al encontrar un $\not\square$, la máquina M se detiene en q_4.

El lector diligente puede probar con las cadenas 0011, 011 y 001 para estudiar el comportamiento en cada caso. ¿En qué estado se queda la MT en cada caso?

El ejemplo anterior sirve para mostrar la capacidad nueva de esta máquina abstracta, es fácil extender el ejemplo para cadenas de la forma $a^nb^nc^n$ o con tantos símbolos como se desee.

Escribir sobre la cinta de entrada y moverse en ambas direcciones le otorga un poder superior al de la pila como única memoria. Todo lo que se calcula con una computadora moderna es posible calcularlo con una MT, de acuerdo con la tesis de Church-Turing. Pasemos a otro ejemplo, un lenguaje que no es aceptado por ningún autómata de pila determinista.

Ejemplo 61. *El lenguaje $\{0^n1^n\}\cup\{0^n1^{2n}\}$ no es aceptado por ningún autómata de pila determinista. Una MT no tiene problemas. El autómata se muestra en la figura 3.2. En su primera parte trabaja idéntico que el de la figura 3.1.*

Los estados 1, 2 y 3 se encargan de revisar que los ceros sean la misma cantidad que los unos, como en las cadenas 01 o 000111. Al terminar esta etapa, el estado 4 verifica que no queden unos extras. Si los hay, pasa a los estados 5 y 6, que verifican que los 1 que queden sean tantos como los 0 que se marcaron con x. El estado 7 comprueba que ya no hay más símbolos x.

Es posible que una MT siga realizando operaciones de manera indefinida, quedando en un ciclo eterno, como ocurre en algunas ocasiones con los programas de computadora de los principiantes. En algunos textos la cinta sólo es infinita hacia la derecha; en ese modelo, si la MT intenta moverse a la izquierda de la primera celda, se detiene de manera anormal. Ambos modelos tienen el

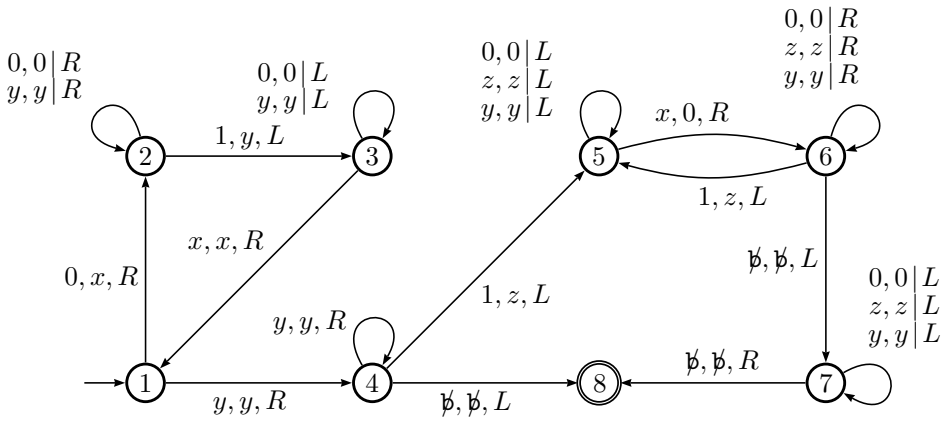


Figura 3.2: Máquina de Turing determinística que acepta el lenguaje $\{0^n1^n\} \cup \{0^n1^{2n}\}$.

mismo poder, pero en estas notas se usa la cinta infinita en ambos sentidos, lo que permite, por ejemplo, que una MT regrese hasta el blanco que está a la izquierda de la entrada, como lo hace el estado 7 de la figura 3.2.

3.1.1. Descripción instantánea de una MT

Como en el caso de los autómatas de pila, es muy conveniente una manera de describir el estado que guarda la MT en un momento arbitrario de su ejecución. Para los AP, tal descripción se dio en la sección 2.2.1.

Las MT pueden ser descritas mediante el contenido de su cinta, insertando el estado actual entre los símbolos, a la izquierda del símbolo que sigue por procesarse, es decir, queda de la forma

$$a_1 a_2 \dots a_{i-1} q_j a_i a_{i+1} \dots$$

que indica que la MT está en el estado q_j y que procesará el símbolo a_i. Debe ser claro que al inicio de su ejecución, al procesar la cadena $a_1 a_2 \dots a_n$, la MT comienza con la descripción instantánea

$$q_0 a_1 a_2 \dots a_n$$

y no es necesario describir el resto de la cinta, que contiene símbolos ␣. Utilizaremos el símbolo $\mapsto$ para indicar que de una descripción instantánea la MT ejecuta una transición y pasa a otra descripción instantánea.

Ejemplo 62. *Para la MT del ejemplo 60, que acepta el lenguaje* 0^n1^n*, la su-*

cesión de descripciones instantáneas para la cadena 0011 es como sigue:

$$\begin{aligned} &q_0 0011 \mapsto X q_1 011 \mapsto X 0 q_1 11 \mapsto X q_2 0 Y 1 \mapsto q_2 X 0 Y 1 \\ &\mapsto X q_0 0 Y 1 \mapsto X X q_1 Y 1 \mapsto X X Y q_1 1 \mapsto X X q_2 Y Y \\ &\mapsto X q_2 X Y Y \mapsto X X q_0 Y Y \mapsto X X Y q_3 Y \mapsto X X Y Y q_3 \not{b} \\ &\mapsto X X Y q_4 Y \end{aligned}$$

y la cadena es aceptada, ya que la ejecución termina con la MT en q_4, el estado de parada. El símbolo de espacio en blanco ($\not{b}$) se escribe para hacer explícito qué símbolo está por procesar la MT, pero sabemos que todo el resto de la cinta está lleno de tal símbolo.

3.1.2. Lenguaje de una MT

En el caso de las MT, como extensión de las otras máquinas abstractas, también se considera que una cadena en la cinta de entrada es aceptada cuando la MT llega a alguno de los estados finales. Sin embargo, por la forma como una MT opera, hay que considerar que una MT puede quedar en un ciclo infinito, además de detenerse en un estado no final, como les puede ocurrir a los AFD y AP.

Por lo general se prefiere que no haya transiciones que salgan de los estados finales. Se supone que al llegar a un estado final o de aceptación, la MT se detiene. Si una MT se detiene en un estado NO final, entonces se interpreta que la cadena ha sido rechazada, pero si queda ciclada, ignoramos la respuesta. Esto se reduce a los siguientes casos: Si w es la cadena en la cinta de entrada al inicio de la ejecución de la MT M, ocurre uno de los siguientes casos.

1. Si M se detiene en un estado final, $w \in L(M)$.
2. Si M se detiene en un estado que no es final, $w \notin L(M)$.
3. Si M se queda ciclada, entonces no tenemos una decisión.

Considerando lo anterior, el lenguaje de la MT M

$$L(M) = \{w \in \Sigma^* \mid q_0 w \mapsto^* \alpha p \beta \text{ con } p \in F \text{ y } \alpha, \beta \in \Gamma^*\} \tag{3.2}$$

donde $\mapsto^*$ indica cero o más pasos, y α y β representan cualquier contenido de la cinta, pues lo importante es el estado final p.

Ejemplo 63. *Consideremos ahora el lenguaje $L = \{0^n 1^n 2^n\}$, que NO es libre de contexto, por lo que no hay un AP que lo acepte. Una manera de conseguir que una MT revise si una cadena pertenece a este lenguaje es con el siguiente algoritmo. La cabeza de lectura se supone apuntando al primer símbolo, que debe ser un cero, aspi que se usará la transicion de q_0 a q_1, cambiando el 0 por a y moviendo la cabeza a la derecha. Este es el algoritmo:*

1. *En q_0, se cambia el primer símbolo (debe ser un 0) por a, se va al estado q_1 y se avanza a la derecha.*
2. *Si símbolo fuera una b, se va al paso 6, en el estado q_4.*

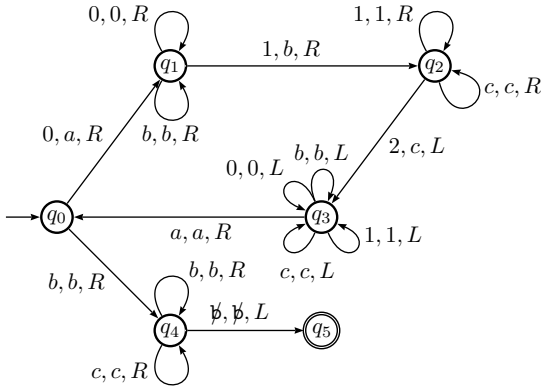


Figura 3.3: Máquina de Turing que acepta el lenguaje NO libre de contexto $0^n1^n2^n$.

3. *En q_1, cada 0 o b que se lee se vuelve a reescribir hasta encontrar un 1. Tal 1 se sustituye por b, se va al estado q_2 y la cabeza de lectura se mueve a la derecha.*
4. *En q_2, cada 1 o c que se lee se vuelve a reescribir hasta encontrar un 2. Ese dos se sustituye por c, se va a q_3 y se mueve a la izquierda.*
5. *En q_3, la cabeza de lectura se mueve a la izquierda, hasta encontrar una a, es decir, cada símbolo que se lee se vuelve a reescribir. Al concontrar la a, se reescribe, se va a q_0 y ahora se mueve a la derecha, regresando al paso 1.*
6. *En q_4, cada símbolo b o c que se lee se reescribe y se mueve a la derecha, hasta encontrar un $\not b$; entonces se pasa al estado de aceptación q_5. Si aparece un 0, un 1 o un 2, la máquina se detiene sin aceptar, pues no hay transición.*

El último paso (6) comprueba que no quedó ningún 0, 1 o 2 sin marcar. La MT que aplica este algoritmo es la de la figura 3.3.

Debe ser claro que es posible extenderse a lenguajes como $\{0^n1^n2^n3^n\cdots k^n\}$.

3.1.3. MT como calculadoras de funciones

Además de una máquina para aceptar cadenas, es decir, una máquina abstracta que resuelve un problema en el sentido de la ecuación (1.4) en la página 13, es conveniente ver a las MT como evaluadoras de funciones, de tal manera que para cierta entrada produzcan una salida distinta que la aceptación.

Pueden verse entonces como funciones *parciales* $MT : \mathbb{N} \to \mathbb{N}$: si la máquina no se detiene con la entrada a, la función no está definida en a. En este caso, si se calcula $f(a) = b$, el argumento a se coloca en la cinta al iniciar la ejecución y al terminar, en la cinta sólo aparece el resultado b.

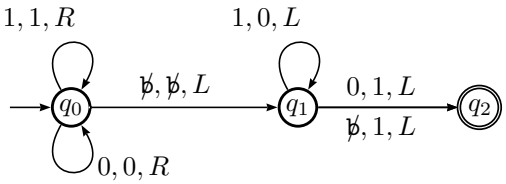


Figura 3.4: Máquina de Turing que suma 1 a su entrada escrita en sistema binario.

De la misma manera, se puede pensar que una MT recibe m argumentos, separándolos con algún símbolo especial y que con ellos calcula un resultado que incluye varios valores. Así, se modelan funciones de varias variables, en general el caso $MT : \mathbb{N}^m \to \mathbb{N}^n$. Veamos algunos ejemplos.

Ejemplo 64 (Contador)**.** *La idea es tener una MT que sume 1 a su entrada, dada en binario. Esa es la base de un contador, sumar uno en cada iteración. La idea es simple si se recuerda la numeración en binario:*

0
1
10
11
100

Lo que se requiere es comenzar desde el final de la cadena y sustituir cada 1 por cero hasta encontrar un 0 que debe cambiarse por uno. Si tal 0 no se encuentra, entonces se agrega un 1 al inicio de la cadena, como al pasar de 11 a 100. La idea es simple y la MT que lo hace es la de la figura 3.4.

Ejemplo 65. *Máquina de Turing que desplaza toda su cadena de entrada una celda a la derecha. Si el alfabeto de entrada consta sólo de un símbolo, el problema se reduce a eliminar el primer caracter y escribirlo al final de la cadena, pero nos interesa más el caso de $|\Sigma| > 1$.*

La idea esencial del algoritmo que se propone es tener estados suficientes para cada símbolo de el alfabeto. Se requiere un estado para ir acarreando a la derecha cada símbolo. Es la misma idea de hacerlo a mano: la cadena `|1101|` debe transformarse en `|␣1101|`, así que al inicio se lee el primer símbolo (1), se escribe ␣ y se pasa a el estado que sabe que se tiene que escribir ese símbolo, digamos q_2, avanzando a la derecha. Ahora, como sigue un 1, no se requiere sobreescribir, solo saltárselo hasta encontrar un símbolo distinto, aunque realmente se debiera leer el segundo 1 y escribir el que se leyó primero, pero no es necesario.

Al llegar al tercer símbolo, el 0, es momento de escribir un 1 en esa posición, pasar al estado que sabe que hay un 0 pendiente, digamos q_1, y avanzar en la

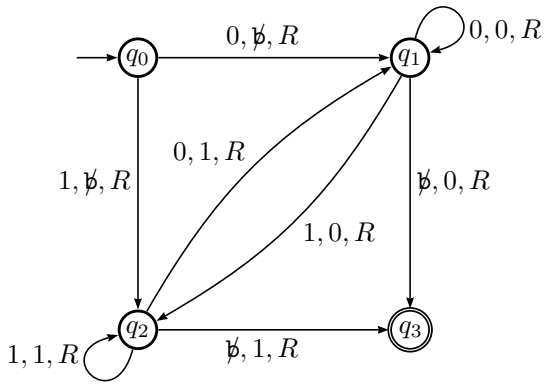


Figura 3.5: Máquina de Turing que desplaza la cadena de entrada una posición a la derecha.

cadena. El siguiente símbolo es un 1, así que se escribe el 0 y se pasa al estado q_2 de nuevo. Ahora, al final de la cadena, encontrarse con el primer espacio en blanco, se escribe el 1 final y se pasa al estado de parada. El diagrama se muestra en la figura 3.5.

Ejemplo 66. *Construyamos una MT que duplica la cadena de entrada, es decir, dada la cadena $w \in \Sigma^*$, al término de su ejecución la cinta tiene la cadena ww. Para duplicar una cadena sobre $\Sigma = \{0, 1\}$, la MT necesita un alfabeto de cinta más amplio, $\Gamma = \{0, 1, a, b, x, y, \not b\}$. La idea de ejecución es duplicar la cadena codificada. Cada 0 se copia como x y cada 1 se copia como y. Para distinguir los símbolos ya copiados se sustituyen con a y b respectivamente. Una secuencia de pasos para la cadena 1001 es la siguiente, donde el símbolo subrayado es el siguiente por copiar.*

$\underline{1}001$	Cadena original
$b\underline{0}01y$	Primer 1 se copia
$ba\underline{0}1yx$	Segundo símbolo copiado
$baa\underline{1}yxx$	Tercer símbolo copiado
$baaby\underline{x}xy$	Cuarto símbolo copiado; falta sustituir símbolos
10011001	

Una vez diseñado el algoritmo que realizará la tarea, hacer la MT es mero trámite.

Como puede verse de los ejemplos anteriores, lo primero para poder hacer una MT que realice alguna tarea se requiere tener idea precisa de cómo se realizará, es decir, un algoritmo que haga lo que queremos. Con cada detalle ya pensado, hacer la MT es bastante más sencillo.

Por otra parte, cuando poco a poco se tiene el diseño de MT que realizan tareas sencillas, es común combinarlas, empleándo una MT como rutina de otra. De esta manera, se realiza lo que se llama construcción modular de MT. El siguiente ejemplo lo ilustra.

Ejemplo 67. *Ahora construiremos una MT que suma dos números dados en binario. La entrada será un par de números separados por el símbolo +. El terminar la ejecución, en la cinta debe estar el resultado y la cabeza de lectura al inicio de la suma. Primeramente, desarrollemos la idea de cómo llevar a cabo la suma.*

Tenemos ya una MT que suma de 1 en 1. De manera similar, hacer una que reste uno también es fácil, por lo que aprovecharemos que se cumple la siguiente propiedad:

$$m + n = (m + 1) + (n - 1).$$

De esta manera, dadas las dos cantidades, mientras la segunda no sea cero, iremos sumando 1 a la primera y restando 1 a la segunda. Cuando la segunda sea 0, borraremos el signo + y los ceros del segundo número para dejar solo el resultado en la cinta, con la cabeza de lectura en el primer bit del resultado.

Por ejemplo: $2 + 3 = 3 + 2 = 4 + 1 = 5 + 0 = 5$. *En binario:*

$$\begin{aligned} 10 + 11 &= 11 + 10 \\ &= 100 + 01 \\ &= 101 + 00 \\ &= 101 \end{aligned}$$

y 101 *es lo único que aparece al final en la cinta. Tendiendo esta idea clara, se puede construir la MT.*

Para hacerlo de manera aún más modular, pensemos que esta MT que suma puede ser empleada por otra MT, así que hay que utilizar la cinta con cuidado. Pensemos que a la derecha de la suma tenemos todo el espacio necesario disponible y que a la izquierda, la cinta está ocupada por otra información, por lo que una suma que haga aumentar el tamaño del primer sumando (nuestro acumulador) corre el riesgo de sobre-escribir en la cinta.

Por ejemplo con la suma $111 + 1111 = 10110$, *en la que el primer sumando ha crecido dos posiciones a la derecha. Para evitar este problema, cada que se requiere un espacio adicional, toda la cadena de suma se recorre una posición a la derecha, empleando para ello una MT como la del ejemplo 65. La diferencia es que ahora aparece también el símbolo + como otro que hay que desplazar. Además, el cursor debe regresarse al inicio de la cadena para continuar con el proceso.*

Una MT como la diseñada es la de la figura 3.6. El estado etiquetado con S_{10+} *representa la MT que desplaza una posición a la derecha la cadena, regresando al inicio. En las transiciones que salen de ese estado, la dirección S indica que la cabeza no se mueve; es una extensión cómoda de la definición que no cambia el poder de las MT, pues equivale a moverse a la derecha y luego regresar a la izquierda.*

3.1.4. Máquinas de Turing no determinísticas

La definición de MT dada en el apartado anterior especifica un comportamiento determinístico. Como en los autómatas anteriores, es posible definir

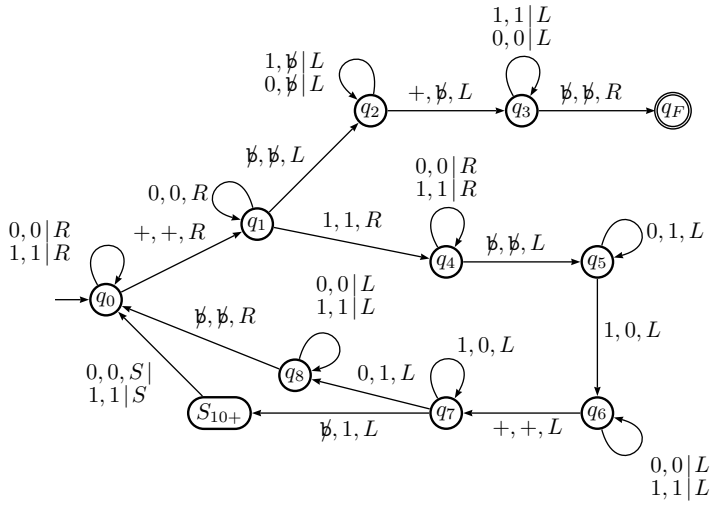


Figura 3.6: Máquina de Turing que suma dos números en binario, empleando una rutina de desplazamiento a la derecha para cuidar el espacio de la cinta. El estado etiquetado con S_{10+} es una MT que desplaza toda la cadena una posición a la derecha y regresando al inicio de la cadena.

Máquinas de Turing No determinísticas. Ahora, las transiciones serán de la forma

$$\delta(p, a) = \{(q_1, b_1, Y_1), (q_2, b_2, Y_2), \ldots, (q_k, b_k, Y_k)\} \tag{3.3}$$

dando diversos movimientos para el mismo par (p, a). Se puede ir a cualquiera de los estados q_i, escribiendo el símbolo b_i en la cinta y moviéndose en la dirección indicada por Y_i. Como en los AFN, puede pensarse que la MT explora todos estos caminos; acepta si alguno de ellos llega a un estado de aceptación.

A diferencia de los Autómatas de Pila, las Máquinas de Turing no ganan poder con el no determinismo, tal y como ocurre con los AFD. Claro, la simulación determinista conocida de una MT no determinista puede tardar exponencialmente más, pero hace el mismo trabajo. Si ese costo es inevitable es una de las preguntas abiertas más importantes de la computación, el problema P vs NP.

Ejemplo 68. *Consideremos el lenguaje* $\{0^n 1^m \mid n \leq m \leq 2n\}$*, en el que a cada 0 le corresponden uno o dos unos. Una MT no determinista puede aceptarlo con una idea parecida a la de la figura 3.2: cada vez que marca un 0 con* $\mathbf{x}$*, en el estado 2 tiene dos transiciones posibles al encontrar el siguiente 1. Un camino es cambiarlo por* $\mathbf{y}$ *y pasar directo al estado 4. El otro es cambiarlo por* $\mathbf{y}$*, pasar al estado 3 y cambiar el siguiente 1 también por* $\mathbf{y}$ *antes de ir al estado 4.*

De esta manera, la máquina adivina*, para cada 0, si le corresponden uno o dos unos, y acepta si alguna sucesión de elecciones marca todos los unos. La MT se muestra en la figura 3.7. El NO determinismo está en el estado dos, pues al encontrarse un 1, el autómata lo cambia por* $\mathbf{y}$ *y avanza a la izquierda y también*

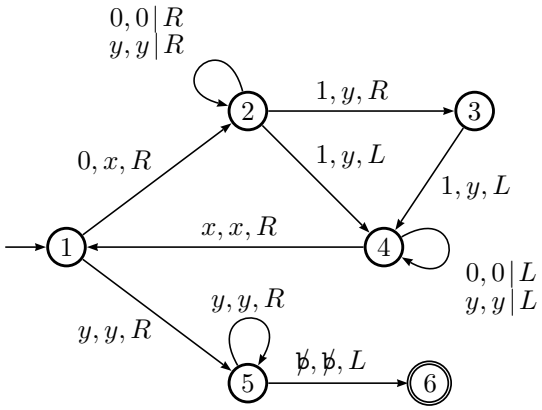


Figura 3.7: Máquina de Turing NO determinística que acepta el lenguaje $\{0^n 1^m \mid n \leq m \leq 2n\}$.

a la derecha.

Obsérvese que esta máquina no acepta el lenguaje $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$ *del ejemplo anterior, pues también acepta cadenas como 00111, en la que un 0 recibe un 1 y el otro dos. Para aceptar exactamente esa unión, la elección tendría que hacerse una sola vez, al principio, y mantenerse durante toda la ejecución.*

El no determinismo puede simplificar mucho el diseño de una MT, como en el ejemplo anterior.

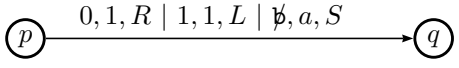
3.1.5. Máquinas de Turing con múltiples cintas

Agregar cintas a la MT permite organizar mejor los algoritmos y el procesamiento de los datos. Por ejemplo, se puede emplear una cinta para la entrada, otra para la salida y otras más para los cálculos intermedios.

El comportamiento de las transiciones es un poco más complejo pues en este caso hay que ver el símbolo que está en cada una de las k cintas de entrada para decidir qué hacer, es decir, para decidir si pasar a otro estado, escribir qué y moverse en una u otra dirección en cada cinta. Por ello las transiciones ahora tienen la forma

$$\delta(p, a_1, a_2, \ldots, a_k) = (q, (b_1, D_1), (b_2, D_2), \ldots, (b_k, D_k))$$

donde las a_i son símbolos que se leen en la respectiva cinta i, b_i es lo que se escribe y D_i la dirección que toma la cabeza de lectura en cada caso. En el caso de $k = 3$ cintas, la transición representada por un diagrama es la siguiente.



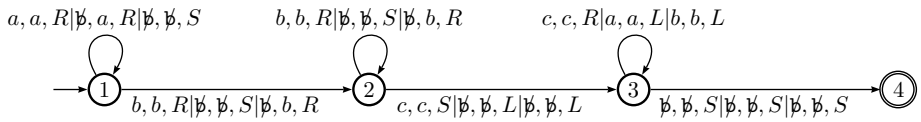


Figura 3.8: Máquina de Turing que acepta el lenguaje $a^n b^n c^n$ con tres cintas. La reducción en tamaño y tiempo de ejecución es notable.

Lo que ocurre en cada cinta queda separado por líneas verticales. La dirección S indica que la cabeza de esa cinta no se mueve. Esta transición se comporta así: estando en el estado p y estando las cabezas de lectura por procesar los símbolos 0, 1 y $\not b$, se mueve al estado q y escribe un 1 en la primera cinta, deja la segunda sin cambios y el símbolo a en la tercera.

Así como el no determinismo, varias cintas facilitan el diseño de algoritmos. Un ejemplo ilustra esto.

Ejemplo 69. *La figura 3.8 muestra une versión con 3 cintas de la MT de una sola cinta de la figura 3.3, que acepta el lenguaje $a^n b^n c^n$. Lo que hace el estado 1 es copiar los símbolos a en la segunda cinta hasta encontrar la primera b y comenzar a copiarlas en la tercera cinta en el estado 2. Al encontrar una c, se queda sobre ella y pasa al estado 3. Para el caso de la cadena aaabbbccc el estado de las cintas y las cabezas de lectura, aquí mostrados como cursores, sería:*

Cinta 1: $aaabbb\underline{c}cc$
Cinta 2: $aa\underline{a}$
Cinta 3: $bb\underline{b}$

Con los cursores en posición, basta moverlos de manera sincronizada, un símbolo a la vez para verificar que se trata de la misma cantidad, procesando la c de la primera cinta, la a de la segunda y la b de la tercera, avanzando (a la derecha) en la primera y retrocediendo (a la izquierda) en las otras dos. El proceso termina cuando se alcanza al mismo tiempo un espacio en blanco y pasando al estado de aceptación.

Como en el caso del no determinismo, las múltiples cintas no le dan poder adicional a las Máquinas de Turing. Cada MT multicinta puede ser simulada con una MT de una sola cinta. Lo que se hace es simular cada cinta en una porción distinta y cuando se requiere más espacio se utiliza una rutina que redistribuya la información, separando los contenidos de cada cinta simulada. El costo es moderado: si la MT de varias cintas hace t pasos, la simulación con una sola cinta hace del orden de t^2 pasos.

3.1.6. Lenguajes sensibles al contexto

Si vemos a las MT como aceptadoras de lenguajes, los lenguajes que aceptan son los *recursivamente enumerables*, y son exactamente los generados por

las *gramáticas no restringidas*, en las que una producción $\alpha \to \beta$ puede tener cualquier cadena de variables y terminales en ambos lados, con al menos una variable en α.

Entre los lenguajes libres de contexto y los recursivamente enumerables hay una clase intermedia, la de los *Lenguajes Sensibles al Contexto* (LSC). Son los que aceptan los *autómatas linealmente acotados*, MT que sólo pueden usar la porción de la cinta que ocupa la entrada, y los que generan las *Gramáticas Sensibles al Contexto* (GSC).

En una GSC, también hay variables, terminales, producciones y un símbolo inicial. Lo que cambia es la forma de las producciones. En las GLC, el lado izquierdo de cada producción constaba sólo de una variable. Ahora la variable puede estar acompañada de un contexto: las producciones son de la forma $\alpha A\beta \to \alpha\gamma\beta$, con γ no vacía, lo que significa que A puede sustituirse por γ sólo cuando está entre α y β. Por ejemplo, con la producción

$$aAb \to aBcb$$

la variable A puede cambiarse por Bc sólo si tiene una a a la izquierda y una b a la derecha. Una definición equivalente, más cómoda en la práctica, sólo pide que ninguna producción acorte la forma sentencial, es decir, que $|\alpha| \leq |\beta|$ en cada producción $\alpha \to \beta$. Las gramáticas de los ejemplos siguientes son de este tipo.

Ejemplo 70. *La gramática con las producciones*

$$\begin{aligned} A &\to aABc \mid abc \\ cB &\to Bc \\ bB &\to bb \end{aligned}$$

genera el lenguaje $\{a^n b^n c^n\}$ *que no es libre de contexto. Un ejemplo de derivación es el siguiente.*

$$\begin{aligned} A &\Rightarrow aABc \\ &\Rightarrow aaABcBc \\ &\Rightarrow aaabcBcBc \\ &\Rightarrow aaabBccBc \\ &\Rightarrow aaabbccBc \\ &\Rightarrow aaabbcBcc \\ &\Rightarrow aaabbBccc \\ &\Rightarrow aaabbbccc \end{aligned}$$

Es evidente cómo el contexto define la producción que se empleará.

Ejemplo 71. *Otra gramática para el mismo lenguaje es*

$$\begin{aligned} A &\to aAB \mid abc \\ bBc &\to bbcc \\ cB &\to Bc \end{aligned}$$

Tipo	Gramática	Autómata	Lenguajes
3	Regular: $A \to aB$, $A \to a$	Autómata finito	Regulares
2	Libre de contexto: $A \to \gamma$	Autómata de pila	Libres de contexto
1	Sensible al contexto: $\alpha A \beta \to \alpha\gamma\beta$	Linealmente acotado	Sensibles al contexto
0	No restringida: $\alpha \to \beta$	Máquina de Turing	Recursivamente enumerables

Tabla 3.2: La jerarquía de Chomsky. Cada tipo de gramática equivale al autómata de su renglón.

y lo genera con secuencias distintas, como puede constatarse intentando con la misma cadena aaabbbccc.

Estas gramáticas son más versátiles pero también mucho más complicadas de emplear y analizar. No se estudiarán en este curso.

3.1.7. La jerarquía de Chomsky

Con las MT se completa la jerarquía de lenguajes que propuso Noam Chomsky en 1956, que resume la tabla 3.2. Cada clase está contenida propiamente en la siguiente: 0^n1^n es libre de contexto pero no regular; $a^nb^nc^n$ es sensible al contexto pero no libre de contexto, y el lenguaje del problema de la parada es recursivamente enumerable pero no sensible al contexto, pues todo lenguaje sensible al contexto es decidible. Entre los sensibles al contexto y los recursivamente enumerables quedan los lenguajes decidibles, que se estudian en la siguiente sección.

3.2. Problemas de decisión

Con problemas de decisión se hace referencia a problemas donde se requiere tener siempre una respuesta, positiva o negativa. Una MT que llega a un estado de aceptación y termina acepta la cadena de su entrada, pero una MT que nunca termina, es decir que queda ciclada, jamás da una respuesta, por lo que la pertenencia de la entrada queda en una incertidumbre.

Muchos problemas pueden plantearse como problemas de decisión. Según se definió en la página 13, en la definición 9, un problema es determinar si una cadena w pertenece a un lenguaje L y es precisamente un problema de decisión. Las MT pueden emplearse para este fin. Más aún, pueden ser empleadas para simular el comportamiento de otra MT.

Los lenguajes mismos se clasifican en dos tipos: los decidibles y los reconocibles.

Definición 18. *Un lenguaje L es* reconocible, *o* recursivamente enumerable, *si existe una MT M tal que $L = L(M)$, es decir, que se detiene aceptando las cadenas de L y no acepta ninguna otra.*

Observe que, con las cadenas que no son del lenguaje, la MT puede detenerse sin aceptar o no detenerse nunca.

Definición 19. *Un lenguaje es* decidible *si existe una MT que siempre se detiene aceptando las cadenas del lenguaje y rechanzando aquellas que no pertenecen.*

En este caso, un lenguaje es decidible cuando existe al menos una MT que siempre se detiene en todas sus entradas. Estos lenguajes también son llamados recursivos.

Varias propiedades son sencillas de probar, en el caso de esta clasificación de lenguajes.

Proposición 15. *Si L_1 y L_2 son lenguajes decidibles, entonces también los son $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 L_2$.*

Demostración. Las pruebas son sencillas y consisten construir una nueva MT M a partir de las MT M_1 y M_2 que deciden a los lenguajes. Se deja tal construcción como ejercicio. □

El caso del complemento se deja para más adelante, cuando esté en un contexto más conveniente. Los lenguajes reconocibles cumplen las mismas propiedades, excepto el complemento, aunque las pruebas requieren más cuidado. Para la unión, por ejemplo, no basta simular M_1 y después M_2, porque M_1 podría no detenerse nunca; hay que simularlas alternando un paso de cada una.

3.2.1. Máquinas de Turing Universales

Las transiciones de una MT pueden codificarse como cadenas de símbolos, estableciendo una numeración binaria para los estados y símbolos.

La transición $\delta(q_2, a) = (q_3, b, L)$ pudiera codificarse como la cadena

$$0010100010010$$

donde los símbolos 1 separan cadenas de ceros que representan cada elemento de la transición. a se codifica con 0 y b con 00. $L = 0$ y $R = 00$, aunque no se emplea.

Una transición genérica $\delta(q_i, a_m) = (q_j, a_n, L)$ puede codificarse como cadenas de ceros separadas por unos como $0^i 1 0^m 1 0^j 1 0^n 10$

$$\overbrace{00\cdots0}^{i}1\overbrace{00\cdots0}^{m}1\overbrace{00\cdots0}^{j}1\overbrace{00\cdots0}^{n}10$$

donde la dirección L se codifica con un cero y R con dos ceros. De esta manera cada transición de una MT pude ser codificada con ceros y unos, colocando todas las transiciones en una misma cadena, separándo el código de cada transición con dos unos consecutivos para distinguirlas.

Definición 20. *Con tal cadena formada por la concatenación de transiciones (o de cadenas que codifican transiciones, para ser exactos), lo interesante es que esa cadena representa por sí sola a la MT original. Si M es tal Máquina de Turing, sea $\langle M \rangle$ su codificación, o* desripción canónica.

Ejemplo 72. *La MT que suma 1 a un número en binario, en la figura 3.4, se codifica como sigue. Sus estados q_0, q_1 y q_2 se numeran como 1, 2 y 3, y sus símbolos ␣, 0 y 1 también como 1, 2 y 3; así, por ejemplo, el estado q_0 se escribe 0 y el símbolo 1 se escribe 000.*
0100010100010011010010100100110101001010110010001001001011001001000100010110010100010010

Definición 21. *Si al lado de esta representación se coloca la cadena de entrada w para la MT, toda esta gran cadena $\langle M \rangle w$ se puede dar como entrada a una Máquina de Turing U cuya tarea sea simular el comportamiento de M. Tal MT U se llama* Máquina de Turing Universal.

Para simular otra MT, requiere varias cintas para facilitar el trabajo. En una están las transiciones codificadas, es decir, la descripción canónica de la MT simulada. En otra la entrada de dicha MT. Otra cinta es de control y se escribe el estado en el que se está en cada momento. La máquina universal avanza al parejo en la transición actual y en la cinta de control para verificar que se tata del mismo estado. De no ser así, busca la siguiente transición. Lo mismo para el símbolo que se lee.

Un trabajo arduo, pero esto modela de manera clara la ejecución de programas en memoria por parte de una computadora.

3.2.2. Máquina de Turing Universal restringida

Aunque la descripción canónica anterior es la que usan los textos clásicos, una MTU que la emplea para simular cualquier MT es bastante sofisticada. Se puede construir una MTU que imponga ciertas restricciones para que sea más sencilla. Tal MTU se describe en esta sección.

La MTU la llamaremos M_R y simulará el comportamiento de MT con pocos estados, etiquetados con letras minúsculas de la *a* a la *f*. La cadena de entrada constará sólo del alfabeto binario además del espacio en blanco, codificado aquí como la letra B, pero durante la ejecución se permite el uso de los símbolos 2, x y y. Solo se considera el caso de MT determinísticas con un solo estado final.

Esto permite codificaciones más cortas para las transiciones. Por ejemplo, la transición $\delta(a, 0) = (b, 1, R)$ queda representada con la cadena a0b1R. Las transiciones se separan con guiones. Para la simulación, M_R emplea tres cintas. La primera cinta contiene los códigos de las transiciones encerrados entre dos símbolos #. Es una cinta de lectura, excepto al final, cuando se coteja el estado final.

La segunda cinta contiene la cadena que será la entrada, encerrada entre dos letras B, que se interpreta como los espacios en blanco alrededor de la cadena. En esta cinta ocurre el comportamiento de la MT simulada. Si durante la simulación se require más espacio al final de la cadena, deben agregarse suficientes símbolos B. La cadena no puede crecer hacia el otro extremo de la cinta.

La tercera cinta contiene dos símbolos, que representan el estado inicial y el estado final. Durante la simulación, la cabeza de lectura está fija en este símbolo y sólo se mueve hasta el final. El estado inicial es cambiado cada que se pasa

de un estado a otro de la MT simulada. Al terminar la ejecución se espera que ambos símbolos sean iguales, lo que significa que la MT llegó al estado final.

Durante su ejecución, M_R lee una y otra vez las transiciones codificadas en la cinta 1. Cada que encuentra la transición correcta, la simula cambiando de estado en la cinta 3, escribiendo en la cinta 2 y moviendo la cabeza en la cinta 2. Luego regresa la cabeza de la cinta 1 al inicio para comenzar de nuevo buscando alguna transición para el nuevo estado y el nuevo símbolo de lectura.

Ejemplo 73. *Empleando la MT M_{++} que suma 1 a su entrada en binario, mostrada en la figura 3.4, su descripción queda de la siguiente manera.*

$$\langle M_{++} \rangle = \texttt{a0a0R-a1a1R-aBbBL-b1b0L-b0c1L-bBc1L}$$

Para esta codificación se ha sustituido ♭ por B, pues la MTU M_R en JFLAP así lo requiere. Si se simulará la suma de 1 a la cadena 1011, las cintas de M_R comenzarían con la entrada y durante la simulación podrá verse cómo la cinta

Cinta 1: #a0a0R-a1a1R-aBbBL-b1b0L-b0c1L-bBc1L#
Cinta 2: B1011B
Cinta 3: ac

representa la cinta de la MT simulada. Como es de esperarse, las cabezas de lectura inician en el primer símbolo de cada cinta.

La MTU M_R consta de 10 estados con suficientes transiciones para procesar el lenguaje binario y la tabla de transiciones de la MT simulada. De manera general, los estados q_0 y q_2 buscan una transición que comience con el estado actual y se pasa a q_1 donde se comprueba que el símbolo de la transición corresponde con el que está por procesarse en la cinta dos. Si no es ese regresa a q_0 y si lo es, pasa a q_3, en la ruta que procesa la transición terminando en q_6, donde se regresa a la primera transición para iniciar de nuevo.

En la tabla 3.3 se explica con detalle lo que hace cada transición. La figura 3.9 es un esquema general de la M_R, que puede descargarse en la liga `https://mancpato.github.io/assets/files/programacion/MTUR.jff`.

¿Cuántas Máquinas de Turing hay?

Con la codificación presentada de máquinas de Turing de la sección previa, es fácil responder esta pregunta. Basta escribir en binario todos los números naturales. Algunas de las cadenas serán representaciones correctas de alguna máquina de Turing. De esta manera, descartando las codificaciones que no corresponden a MT alguna, tendremos una lista de todas las MT posibles. Evidentemente, esta es una cantidad infinita y enumerable, aún cuando hay una infinidad de números cuya representación en binario no corresponde a ninguna MT.

Transición	**Objetivo**
S a q_0	Se salta los delimitadores de las cintas 1 y 2
q_0 a q_1	Si el estado de inicio de la transición coincide con el que indica la cinta 3
q_0 a q_2	Si el estado de inicio de la transición NO coincide con el que indica la cinta 3
q_2 a q_2	Avanza hasta encontrar el símbolo de nueva transición $(-)$
q_1 a q_3	Comprueba que el símbolo de lectura de la transición sea el indicado en la cinta 2, si coinciden se ha encontrado la transición correcta
q_3 a q_4	Cambia el estado actual de la cinta 3 por el nuevo estado indicado en la transición que se procesa
q_4 a q_5	Escribe en la cinta 2 el símbolo indicado en la transición que se lee
q_5 a q_6	Mueve la cabeza de lectura de la cinta 2 conforme se indica en la transición de la cinta 1
q_6 a q_6	Regresa la cabeza de lectura de la cinta 1 hasta la primera transición, para comenzar de nuevo
q_2 a q_7	Si ya no hay transición adecuada, el estado de la cinta 3 se copia en la cinta 1
q_7 a q_8	Si el estado copiado coincide con el estado final, la ejecución termina.

Tabla 3.3: Explicación de las transiciones de la Máquina de Turing Universal M_R.

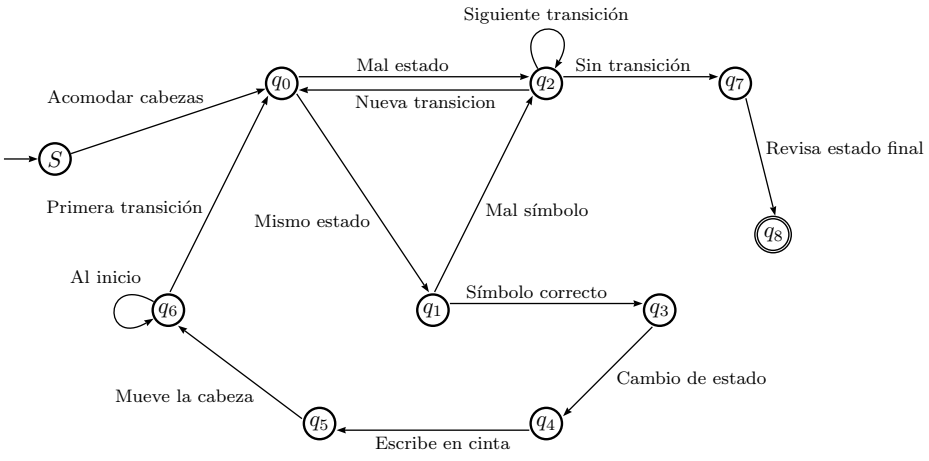


Figura 3.9: Esquema de la Máquina de Turing universal M_R.

¿Cuántas funciones hay?

> Esta sección está tomada de las notas Fundamentos Matemáticos y repetida aquí por conveniencia.

Un programa puede verse como una función que recibe una entrada y produce una salida. Tanto entrada como salida pueden codificarse como cadenas de bits, por lo que cada programa se interpreta como una función, posiblemente parcial, de $\mathbb{N}$ en $\mathbb{N}$.

¿Cuántas funciones hay? ¿Cuántos programas se pueden programar? Estas dos preguntas son de gran importancia en el mundo de la computación pues si son la misma cantidad, entonces es posible hacer un programa para cada función que se pueda definir, independientemente de si son 1 a 1, sobre, biyectivas o con cualquier estructura especial.

Considérese cualquier lenguaje de programación. Está basado en una cantidad finita de palabras reservadas y operadores. Con esos elementos, se pueden comenzar a enumerar los programas desde el más pequeño construible, aumentando la cantidad de bytes uno por uno de ser posible. Esto nos da una numeración para los posibles programas, por lo que son tantos programas como números naturales. Cada programa resuelve un único problema, aunque claramente la relación *(programa, problema)* no es inyectiva.

Por otra parte, contar las funciones posibles de $\mathbb{N}$ en $\mathbb{N}$ requiere algo más de esfuerzo. Una función no tiene restricciones de dominio y rango, por lo que se puede comenzar con las funciones que van de $\mathbb{N}$ en $\{0,1\}$. Con ese dominio y rango se definen las *funciones características*. Sea $A \subset \mathbb{N}$ y su función característica $f_A : \mathbb{N} \to \{0,1\}$ como

$$f_A(x) = \begin{cases} 1 & \text{si } x \in A \\ 0 & \text{si } x \notin A \end{cases}$$

que caracteríza a los elementos del conjunto A marcándolos con 1. ¿Cuántas funciones características hay? Tantas como subconjuntos se pueden definir para los números naturales, por lo que hay al menos tantas funciones como $|2^{\mathbb{N}}| = |\mathbb{R}|$. Y eso que sólo se está contando esa familia de funciones.

Debido a que $|\mathbb{N}| < |\mathbb{R}|$, eso significa que hay más funciones que programas. ★ Este resultado es de gran importancia pues indica que hay una infinidad de problemas que no tiene solución computable, o sea que hay problemas para los que no hay programas que los resuelvan. Se acaba así la intención de que la computadora resuelva todo. Todo lo anterior es la prueba de la contundente

Proposición 16. *Hay más problemas que posibles programas de computadora.*

Pero, ¿cuáles son esos problemas para los que no hay solución? ¿aparecen en la vida cotidiana? Son muchas las preguntas que falta responder y son de gran importancia tanto teórica como práctica.

3.2.3. Lenguajes recursivos

En una MT M con lenguaje $L(M)$, una cadena w es aceptada cuando M llega a un estado de aceptación al procesar w y se detiene. Pero si M procesa w y no llega a un estado de aceptación, se tiene dos casos: que M se detenga en un estado que no es de aceptación o que no se detenga jamás, es decir, que se quede en un ciclo infinito.

Definición 22. *Un Lenguaje Recursivo es aquel para el cual hay una MT que siempre se detiene con cualquier entrada, ya sea en un estado de aceptación o no.*

Es decir, los lenguajes recursivos son exactamente los decidibles; ambos nombres se usan indistintamente.

3.2.4. Lenguajes diagonal y universal

Consideremos el alfabeto $\{0, 1\}$. Sus cadenas pueden enumerarse ordenándolas primero por longitud y, entre las de la misma longitud, en orden lexicográfico: $\epsilon, 0, 1, 00, 01, 10, 11, 000, \ldots$ (en la tabla que sigue se omite la cadena vacía para simplificar). Sean $w_1, w_2, \ldots$ tales cadenas.

Como también las MT pueden enumerarse, codificándolas en binario como ya se vio, sean $M_1, M_2, \ldots$ tales máquinas. Por convención, una cadena que no codifica ninguna MT se considera el código de una MT que no acepta ninguna cadena; así, cada cadena w_i es el código de una máquina M_i. Ahora es posible construir una tabla donde se enumeren las cadenas (verticalmente) y las MT (horizontalmente), obteniendo una tabla de pertenencias. De esta manera, cada columna corresponde al lenguaje de una MT M_j, representando la pertenencia al lenguaje de cada cadena enumerada a la izquierda.

Lenguaje diagonal L_d

La tabla 3.4 muestra esta construcción. Cada fila i representa la cadena w_i e indica si esta cadena es aceptada (1) o no (0) por la MT M_j de la columna que corresponde, el problema ¿$w_i \in L(M_j)$?. En la tabla se han marcado los elementos de la diagonal, es decir, la pertenencia de la cadena w_i al lenguaje $L(M_i)$.

El *lenguaje diagonal*, representado como L_d es un lenguaje que no está en esa tabla y que se construye invirtiendo las pertenencias de la diagonal. De esta forma, la cadena w_i sí pertenecerá a L_d si no pertenece a $L(M_i)$ y no pertenecerá a L_d si pertenece a $L(M_i)$.

$$L_d = \{\ w_i \in (0+1)^* \mid w_i \notin L(M_i)\ \} \tag{3.4}$$

Proposición 17. *L_d NO es un lenguaje recursivamente enumerable.*

Demostración. Supongamos que existe una MT M tal que $L_d = L(M)$. Como las MT son enumerables, M debe ser una de las máquinas M_i de la tabla 3.4, donde están enlistadas todas las MT por lo que $M = M_j$ para alguna j.

$w_i \backslash M_j$	M_1	M_2	M_3	M_4	M_5	M_6	M_7	M_8	
0	1	0	1	0	1	0	1	1	
1	1	1	1	0	1	0	0	1	
00	0	1	0	1	1	0	1	0	
01	0	0	0	1	1	0	1	0	
10	0	1	0	0	0	0	0	1	
11	0	1	1	1	1	1	1	1	
000	1	0	0	1	0	0	0	0	
001	0	0	1	0	0	1	0	1	$\cdots$
010	1	0	1	1	0	1	1	1	
011	0	1	0	0	0	0	0	1	
					$\vdots$				

Tabla 3.4: Tabla de pertenencia de todas las cadenas de $(0+1)^*$ y todas las MT. Los elementos marcados sirven para definir el lenguaje diagonal L_d, que no es recursivamente enumerable.

Pero, ¿$w_j \in L_d$?. Si w_j está en L_d, entonces eso significa que $w_j \in L(M_j)$, lo que es una contradicción, por la definición (3.4), que dice que $w_i \notin L(M_i)$.

Si w_j no está en L_d, entonces debe pertenecer a $L(M_j)$, por la definición (3.4) lo que de nuevo es una contradicción porque $M = M_j$. □

Lenguaje de autoaceptación L_a

A partir de la misma tabla 3.4, es posible formar el *lenguaje de autoaceptación*, L_a. Este se construye con aquellas cadenas w_i que representan la codificación de la MT M_i y que aceptan su propio código como cadena de entrada.

En otras palabras, L_a es el conjunto de cadenas que representan MT que se aceptan como entrada. ¿Este lenguaje es reconocible o es decidible? Analicemos cada caso.

Sea w una cadena arbitraria de L_a. Una MT universal U puede simular el código $\langle w \rangle$ procesando la cadena w y como las máquinas representadas por cadenas de L_a aceptan su propio código, esta simulación terminará exitosamente. Así, la máquina U siempre termina la simulación de cadenas de L_a, por lo que el lenguaje es al menos reconocible.

Claro, esto no significa que L_a también sea decidible, pues no sabemos si U puede quedarse ciclada con otras cadenas que no estén en L_a. La respuesta a esta pregunta nos la da la siguiente proposición.

Proposición 18. *El complemento de un lenguaje decidible es también decidible.*

Demostración. Sea L un lenguaje decidible. Eso significa que hay al menos una MT, digamos M, que decide L, es decir, se detiene aceptando las cadenas de L y las que no son de L no las acepta pero también se detiene. A partir de M

se puede construir una MT que decida el lenguaje complemento L^C de manera sencilla.

Sea M_C una MT que emplea a M como subrutina invirtiendo sus respuestas. Cada vez que M_C recibe una cadena de $w \in L^C$, simula a M. Como w está en el complemento de $L = L(M)$, entonces M termina y no acepta a w, por lo que M_C informa que la cadena es aceptada. Si a M_C se le da una cadena que no está en L^C, entonces M la aceptará y M_C responderá que esa cadena no es aceptada. □

De esta manera, si L_a fuera decidible su complemento también lo sería. ¿Quién es el complemento de L_a? Es otro lenguaje, compuesto por las cadenas que representan MT que no aceptan su propio código, es decir, L_d, el lenguaje diagona, que no es recursivamente enumerable. Si L_a fuera decidible, también L_d lo sería, pero ni siquiera es reconocible. Por lo tanto, L_a no es decidible. Se trata de un ejemplo de un lenguaje reconocible, con complemento no reconocible.

En muchos textos se llama *lenguaje universal* al lenguaje $L_u = \{\langle M\rangle w \mid M \text{ acepta } w\}$, que es justamente el que reconoce la MT universal. L_a es un caso particular, en el que la entrada es el propio código de la máquina, así que L_u tampoco es decidible: si lo fuera, una MT que decide L_u también decidiría L_a, con sólo duplicar su entrada.

La situación de L_a es un caso particular de un resultado general.

Proposición 19. *Un lenguaje L es decidible si y sólo si L y su complemento L^C son reconocibles.*

Demostración. Si L es decidible, también lo es L^C, y todo lenguaje decidible es reconocible. Recíprocamente, sean M_1 y M_2 MT que reconocen L y L^C. Con una cadena w de entrada, se construye una MT M que simula a M_1 y a M_2 alternando un paso de cada una. Como w está en L o en L^C, alguna de las dos termina aceptando: si es M_1, M acepta, y si es M_2, rechaza. Así, M siempre se detiene y decide L. □

Por eso, como L_a es reconocible pero no decidible, su complemento no puede ser reconocible.

3.2.5. El problema de la parada

A partir de que hay más problemas que programas, se entiende que hay una infinidad (incontable) de problemas que no se pueden resolver con computadora. Sin embargo, encontrar tales problemas no es sencillo.

El problema de la parada es el ejemplo más conocido de un problema que no puede resolver ninguna MT y, por la tesis de Church-Turing, ninguna computadora. Su origen requiere una aclaración, porque es común atribuirlo directamente al artículo de Turing de 1936, y no es del todo exacto.

En ese artículo, Turing no habla de máquinas que se detienen. Sus máquinas calculaban números reales escribiendo una sucesión infinita de dígitos, y llamó *libres de círculos* a las que nunca dejan de escribirlos. Lo que demostró es que

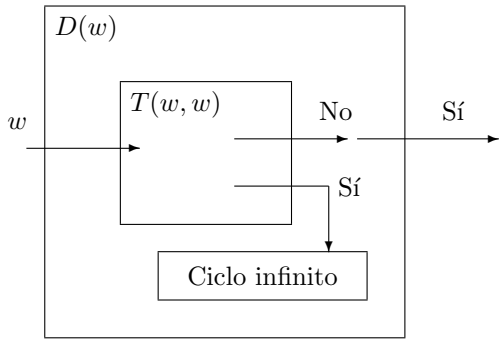


Figura 3.10: Esquema de la Máquina de Turing para el problema de la parada. Se comprueba que es imposible la construcción de la MT T.

no existe una máquina que decida si otra máquina es libre de círculos, ni una que decida si otra máquina llega a imprimir cierto símbolo. Con esos resultados respondió negativamente el *Entscheidungsproblem* de Hilbert: la pregunta de si existe un procedimiento mecánico para decidir si una fórmula de la lógica de primer orden es válida. Unos meses antes, Alonzo Church había llegado a la misma respuesta con un modelo distinto, el cálculo lambda.

La formulación que se presenta aquí, en términos de máquinas que se detienen o no, apareció después. Stephen Kleene la planteó prácticamente así en un libro de 1952, y el nombre de *problema de la parada* se debe a Martin Davis, que lo usó en sus clases desde 1952 y lo publicó en su libro de 1958. Los problemas de Turing y el de la parada están estrechamente relacionados y la técnica de demostración es la misma, la diagonalización, pero no son idénticos.

Supongamos que existe una máquina de Turing T que recibe como entrada dos cadenas iguales w, el código de cualquier MT. Lo que hace T es decidir si la MT codificada como w termina su ejecución utilizando como entrada la misma cadena w. De esta forma, $T(w, w)$ siempre sabe si toda otra MT termina o no cuando recibe como entrada su propio código.

A partir de T, se puede construir otra MT D que utiliza a la MT T para decidir qué hacer. La entrada de D se alimenta con w, el código de alguna MT. D copia w y pasa las dos cadenas a T, que hace su trabajo determinando si la máquina codificada con w termina al procesar su propio código. Con la respuesta de T, la máquina D hace lo siguiente. Si T dice que w termina, entonces D entra en un ciclo infinito. Pero si T dice que w NO termina, entonces D responde que Si.

Si para cualquier MT T, su código en binario es $\langle T \rangle$, entonces ¿que responde D cuando su entrada es $\langle D \rangle$? Solo hay dos casos.

Caso 1: Si T responde Sí cuando $w = \langle D \rangle$, entonces D entra en un ciclo infinito y T se habrá equivocado pues D no para.

Caso 2: Si T responde que D No para, entonces D termina de inmediato

respondiendo que Sí, lo que hace que T se haya equivocado de nuevo.

Con los dos razonamientos anteriores, es evidente que la existencia de una MT como T es imposible. Como implicación práctica inmediata, esto significa que no existe un algoritmo general que, para cualquier programa y cualquier entrada, determine si el programa terminará. Para programas particulares muchas veces sí puede saberse; lo imposible es un método que funcione para todos. La MT D se muestra como esquema en la figura 3.10.

La versión que se presentó es la de una MT que recibe su propio código como entrada. La versión general pregunta, dada cualquier pareja formada por una MT M y una cadena w, si M se detiene con w. También es indecidible: si se pudiera decidir la versión general, se podría decidir en particular el caso $w = \langle M \rangle$.

Este es un ejemplo académica e históricamente importante, que nos da muestra de limitaciones que tienen las MT y por tanto las computadoras que empleamos.

3.2.6. Reducciones

Una vez que se tiene un problema indecidible, es posible demostrar que otros también lo son sin repetir la diagonalización. La herramienta es la *reducción*: un problema A se reduce a un problema B si existe una MT que siempre se detiene y que transforma cada instancia x de A en una instancia $f(x)$ de B, de tal manera que la respuesta a x es sí exactamente cuando la respuesta a $f(x)$ es sí.

Si A se reduce a B y B fuera decidible, entonces A también lo sería: bastaría transformar x en $f(x)$ y aplicar la MT que decide B. Por lo tanto, si A es indecidible, B también lo es. Hay que cuidar la dirección: se reduce el problema que *ya se sabe* indecidible al problema que se quiere estudiar.

Ejemplo 74. *Sea V el problema de decidir, dada una MT M, si M se detiene cuando su entrada es la cadena vacía. Se reduce a V el problema de la parada. Dada una MT M y una cadena w, se construye la MT M_w que borra su entrada, escribe w en la cinta y después se comporta como M. Construir el código de M_w a partir de $\langle M \rangle$ y w es un proceso mecánico que siempre termina. Además, M_w se detiene con la cadena vacía si y sólo si M se detiene con w. Si V fuera decidible, el problema de la parada también lo sería; por lo tanto, V es indecidible.*

3.2.7. El teorema de Rice

El ejemplo anterior no es un caso aislado. Una *propiedad semántica* de las MT es una propiedad del lenguaje que aceptan, y no de cómo están construidas; por ejemplo, *aceptar la cadena vacía*, *aceptar un lenguaje finito* o *aceptar un lenguaje regular*. Una propiedad es *no trivial* si algunas MT la tienen y otras no.

Proposición 20 (Teorema de Rice)**.** *Toda propiedad semántica no trivial de las MT es indecidible: no existe una MT que, dado el código $\langle M \rangle$ de cualquier MT, decida si $L(M)$ tiene la propiedad.*

Idea de la demostración. Sea P una propiedad no trivial. Podemos suponer que el lenguaje vacío no la tiene; si la tiene, se razona con la propiedad contraria, que es decidible si y sólo si P lo es. Como P no es trivial, hay una MT M_P cuyo lenguaje sí la tiene. Se reduce a P el problema de la parada: dada una MT M y una cadena w, se construye la MT N que, con entrada x, primero simula a M con w y, si esa simulación termina, simula a M_P con x. Si M se detiene con w, entonces $L(N) = L(M_P)$, que tiene la propiedad; si no se detiene, N no acepta nada y $L(N) = \emptyset$, que no la tiene. Así, decidir P para N equivaldría a decidir si M se detiene con w. □

Es importante leer el teorema con precisión. Habla de propiedades del *lenguaje*, es decir, de lo que la máquina hace, y no de su *código*, es decir, de cómo está escrita: preguntas como si una MT tiene más de diez estados sí son decidibles. Además, *indecidible* no significa que la pregunta no pueda responderse para ninguna máquina, sino que no existe un algoritmo que la responda correctamente para *todas*. Tampoco debe confundirse *indecidible*, cuando no existe un algoritmo, con *costoso*, cuando existe pero tarda demasiado.

Por la tesis de Church-Turing, el teorema se aplica a los programas de cualquier lenguaje de programación de propósito general. Por ejemplo, no hay un algoritmo que decida si un programa arbitrario termina con toda entrada, ni si dos programas arbitrarios calculan la misma función.

Esto tiene una consecuencia directa en los compiladores. Un comprobador de tipos estático no puede responder la pregunta *¿este programa se comportará bien al ejecutarse?*, que es una propiedad semántica no trivial. Lo que responde es una pregunta más simple y decidible: *¿el programa cumple las reglas de tipos?* Como las dos preguntas no coinciden, hay dos errores posibles: una *falsa alarma*, cuando se rechaza un programa que al ejecutarse sería correcto, y una *falla no detectada*, cuando se acepta un programa que después falla. Un sistema que nunca acepta programas incorrectos se llama *sólido* (en inglés, *sound*), y uno que nunca rechaza programas correctos se llama *completo*. Por el teorema de Rice, un comprobador que siempre termina no puede ser ambas cosas para las propiedades interesantes de los programas, y cada lenguaje elige un compromiso distinto. Rust, por ejemplo, prefiere las falsas alarmas: rechaza un programa cuando no puede demostrar que cierto uso de la memoria es seguro. En C, en cambio, el compilador acepta programas que acceden fuera de los límites de un arreglo, lo que produce comportamiento indefinido. Por eso, que un programa compile no significa que sea correcto.

Entendiendo que hay una frontera tras la cual están los problemas que no se pueden resolver, uno de los objetivos de la Teoría de la Computación es estudiar los problemas que sí tienen solución, clasificándolos por su dificultad, en términos del consumo de recursos. Esa es el área conocida como *Complejidad Computacional* y es motivo de otra asignatura.